MLREML: linear mixed models by average-information REML for plant and animal breeding

User guide, theoretical background and worked examples with real data — version 1.0.0

Osval Antonio Montesinos López, Abelardo Montesinos-López, José Crossa, J. Cricelio Montesinos-López

October 2026

Table of Contents

1 Introduction 5
1.1 What MLREML is 5
1.2 Who this guide is for 5
1.3 Installation 5
1.4 Capabilities and their correspondence with ASReml 6
1.5 A first example 7
1.6 What is new in version 1.0.0 ..... **Error! Bookmark not defined.**
2 Theoretical foundations 8
2.1 The linear mixed model 9
2.2 Structures as direct products 9
2.3 Restricted likelihood 10
2.4 The average information algorithm 10
2.5 Estimation of fixed effects and prediction of random effects 11
2.6 Predicted means and standard errors of differences 12
2.7 Hypothesis tests 12
2.8 Functions of the variance parameters 13
2.9 Relationship matrices 13
2.10 Computational cost 14
2.11 Correlated effects of several factors: direct–maternal models 14
2.12 The sparse engine: REML with the mixed model equations 15
2.13 Kenward–Roger degrees of freedom 16
3 Model syntax 17
3.1 The function `mlreml()` 17
3.2 Random terms 18
3.3 Residual structures 20
3.4 Multivariate models 20
3.5 Parameter names, starting values and constraints 21

3.6 The fitted object 21
4 Example 1: an oat trial in an alpha-lattice design 22
4.1 The data 22
4.2 Varieties as fixed effects: adjusted means 22
4.3 Varieties as random effects: BLUP and heritability 24
4.4 Comparison of adjusted means and BLUPs 24
4.5 Is the incomplete-block design worthwhile? 25
4.6 Diagnostics 25
4.7 Verification with lme4 26
5 Example 2: spatial analysis of a wheat trial 27
5.1 The data 27
5.2 Complete-block model 28
5.3 AR1 × AR1 spatial model 28
5.4 Global trend 28
5.5 Sample variogram 29
5.6 Gain in precision 30
5.7 Estimated spatial trend 31
5.8 Irregularly spaced data 31
6 Example 3: multi-environment trials and factor analytic models 32
6.1 The data 32
6.2 The sequence of models 32
6.3 Interpreting the FA(1) model 34
6.4 Predictions by environment and overall performance 35
6.5 Recommendations for MET analysis 36
7 Example 4: genomic prediction (GBLUP) with 599 wheat lines 37
7.1 The data 37
7.2 Quality control and genomic matrix 37
7.3 GBLUP in one environment 38
7.4 Predicting unphenotyped lines 39
7.5 Cross-validation: pedigree, markers and both 39
7.6 Multi-environment GBLUP 40
7.7 Practical notes 42
8 Example 5: animal model with pedigree in dairy cattle 42
8.1 The data 42
8.2 Preparing the pedigree 42
8.3 Animal model for first lactation 43
8.4 Repeatability model with all lactations 45
8.5 Maternal effect 46

8.6 Bivariate analysis of milk and fat 46
8.7 Direct–maternal genetic effects 48
8.8 Large data sets: the sparse engine 51
9 Example 6: single-step GBLUP with genotyped and non-genotyped individuals 52
9.1 Setting 52
9.2 Aligning G with the pedigree and building H 53
9.3 Comparison by validation 53
9.4 Single-step with the general engine 54
9.5 Recommendations 54
10 Example 7: repeated measures and random regression 55
10.1 Residual correlation: growth of children 55
10.2 Linear random regression: sleep deprivation 56
10.3 Random regression with Legendre polynomials: growth curves 58
11 Example 8: diallel analysis 59
11.1 The data 59
11.2 The model 60
11.3 Interpretation 61
11.4 Predicted combining abilities 61
11.5 Model comparison 62
12 Example 9: unreplicated early-generation trial 62
12.1 The data 62
12.2 Models 63
12.3 Consequences for selection 64
12.4 Spatial trend and checks 65
13 Advanced use and troubleshooting 66
13.1 Iteration history 66
13.2 Starting values and restarts 67
13.3 Fixed parameters and profile likelihood 68
13.4 Known variance components 69
13.5 Aliased fixed effects 69
13.6 Parameters on the boundary 70
13.7 Ill-conditioned relationship matrices 70
13.8 Scale of the variables 70
13.9 Memory and time 71
13.10 Common problems 71
14 Simulation study: properties of the estimators 72
14.1 Complete block design with random genotypes 72
14.2 Size of the likelihood ratio test on the boundary 74

14.3 Spatial model 74
15 Numerical validation, performance and limitations 75
15.1 Comparison with other software 75
15.2 Computing time and memory 79
15.3 Strategies for large data sets 81
15.4 Limitations of version 1.0.0 81
16 Function summary 82
16.1 Syntax equivalences with ASReml-R 83
16.2 Quick guide to models 83
16.3 Post-fit functions 85
References 85
Appendix: function reference manual 88
16.4 `MLREML-package`: MLREML: linear mixed models by Average-Information REML 88
16.5 `mlreml`: Fit a linear mixed model by Average-Information REML 89
16.6 `summary.mlreml`: Print and summary methods for mlreml fits 92
16.7 `coef.mlreml`: Extract fixed and random effects 92
16.8 `wald`: Wald tests for fixed effects 93
16.9 `predict.mlreml`: Predicted means with standard errors and standard errors of differences 94
16.10 `vpredict`: Functions of variance parameters with delta-method standard errors 95
16.11 `lrt`: Likelihood ratio test between nested models 95
16.12 `vc_matrix`: Estimated covariance and correlation matrix of a structured component 96
16.13 `variogram`: Sample variogram of residuals of a field trial 96
16.14 `plot.mlreml`: Diagnostic plots 96
16.15 `pedigree_prepare`: Prepare a pedigree 97
16.16 `inbreeding_coef`: Inbreeding coefficients (Meuwissen and Luo, 1992) 97
16.17 `Amatrix`: Numerator (additive) relationship matrix 98
16.18 `Ainverse`: Sparse inverse of the additive relationship matrix 98
16.19 `snp_qc`: Quality control of a SNP matrix 98
16.20 `Gmatrix`: Genomic relationship matrix 99
16.21 `align_G`: Align a genomic matrix with the pedigree matrix 99
16.22 `Hmatrix`: Single-step relationship matrix H and its inverse 100
16.23 `bend`: Make a symmetric matrix positive definite (bending) 100
16.24 `gblup`: Fast GBLUP by spectral decomposition 101
16.25 `cv_gblup`: K-fold cross-validation of GBLUP 101

1 Introduction

1.1 What MLREML is

MLREML is an R package that fits linear mixed models by restricted maximum likelihood (REML) with the Average Information (AI) algorithm. It is designed for the typical problems of plant and animal breeding: estimation of variance and covariance components, heritabilities and genetic correlations, prediction of genetic values (BLUP, EBV and GEBV), analysis of multi-environment trials, spatial analysis of field experiments, models with pedigree and genomic information, multivariate models and longitudinal models.

The name combines the two ideas that organise the package: mixed linear models and their estimation by REML. Its syntax is inspired by that of ASReml (Gilmour et al., 2015; Butler et al., 2017), the reference program in this field, so that an ASReml user can read and write MLREML models with almost no additional learning. MLREML is, however, independent open-source software: it contains no ASReml code, does not depend on it and requires no licence.

The central idea of the package is the same one that makes ASReml powerful. A linear mixed model is defined not only by the effects it contains but by the way those effects and the residuals are distributed and correlated. Each random term in MLREML is therefore written as a direct product of variance structures: for example, `us(env):vm(gen, G)` declares genetic effects of each genotype in each environment, correlated among genotypes according to the genomic matrix **G** and among environments according to an unstructured covariance matrix estimated from the data.

MLREML also includes the covariance between direct and maternal genetic effects (and, in general, between random effects of several factors that share a relationship matrix) through the term `str()`; a second computing engine based on the mixed model equations with sparse matrices, for data sets with hundreds of thousands of records; and Kenward–Roger denominator degrees of freedom for the F tests of fixed effects.

1.2 Who this guide is for

This guide is written for researchers, lecturers and graduate students in quantitative genetics, plant and animal breeding, experimental statistics and biostatistics. It assumes basic knowledge of R and of linear models. The theory of mixed models is presented in full in Chapter 2, with the notation used throughout the document, so the guide can also be used as course material.

The document has three parts. The first (Chapters 1 to 3) describes the model, the estimation algorithm and the syntax. The second (Chapters 4 to 12) presents complete examples with public-domain real data: an alpha-lattice oat trial, a wheat trial with spatial analysis, a multi-environment maize trial, 599 CIMMYT wheat lines genotyped with markers, milk production records of Holstein cows with their pedigree, growth curves of chickens, reaction times of people in a sleep-deprivation study, a complete tobacco diallel and an unreplicated early-generation wheat trial. The third part (Chapters 13 to 16) covers advanced use, a simulation study, the numerical validation of the package against other programs, its limits and a summary of all functions; the complete function reference is given in the appendix.

1.3 Installation

The package is distributed as a source file `MLREML_1.0.0.tar.gz` available at https://github.com/osval78/MLREML. It depends on the packages `Matrix` (version 1.6-0 or later, which ships with R) and `Rcpp`, and on `methods`, `stats`, `graphics` and `utils`. The package

contains C++ code (the sparse inverse used by the sparse engine and the computation of inbreeding coefficients), so it is installed from source with a compiler: Rtools on Windows, the Xcode command line tools on macOS (`xcode-select --install`) and the usual development tools on Linux (for example, the `r-base-dev` package of Debian and Ubuntu):

```
install.packages("Rcpp")
install.packages("MLREML_1.0.0.tar.gz", repos = NULL, type = "source")
library(MLREML)
```

The examples in this guide use data from the packages `agridat` (Wright, 2024), `BGLR` (Pérez and de los Campos, 2014), `pedigreemm` (Vazquez et al., 2010) and `lme4` (Bates et al., 2015), and compare results with `lme4`, `nlme` (Pinheiro and Bates, 2000) and `pbkrtest` (Halekoh and Højsgaard, 2014). None of them is needed to use MLREML:

```
install.packages(c("agridat", "BGLR", "pedigreemm", "lme4", "nlme",
"pbkrtest"))
```

For fast linear algebra, R should be used with an optimised BLAS library (OpenBLAS, Intel MKL or Accelerate on macOS). The time of a fit with the dense engine is dominated by Cholesky factorisations of $n \times n$ matrices, and an optimised BLAS speeds them up 10 to 30 times relative to the reference BLAS. The sparse engine also benefits, because the dense blocks of its sparse factorisation and of its sparse inverse are computed with BLAS and LAPACK.

1.4 Capabilities and their correspondence with ASReml

The table below summarises the ASReml capabilities usually mentioned in plant and animal breeding and how to obtain them in MLREML. It states plainly what is fully implemented, what is partial and what is not available in this version.

ASReml capabilities and their MLREML equivalent.

Capability (ASReml)	In MLREML	Status
Linear mixed models, unbalanced data	`mlreml()`	Complete
REML with average information (AI)	AI algorithm with damping	Complete
Variance components with standard errors	`summary()`, inverse of the AI matrix	Complete
Heritability, correlations, functions of components	`vpredict()` (delta method)	Complete
Animal model with pedigree, EBV	`vm(animal, A)`, `Amatrix()`, `Ainverse()`	Complete
Inbreeding coefficients	`inbreeding_coef()` (Meuwissen and Luo)	Complete
GBLUP	`vm(gen, G)`, `Gmatrix()`, `gblup()`	Complete
Single-step GBLUP (H matrix)	`Hmatrix()`, `align_G()`, `bend()`	Complete
SNP quality control	`snp_qc()`	Basic
Multivariate models, genetic correlation	`cbind(y1, y2) ~ trait`, `us(trait)`	Complete

Capability (ASReml)	In MLREML	Status
Multi-environment trials (MET), G×E	`diag()`, `us()`, `fa(k)`	Complete
Heterogeneous residual variances	`residual = ~ diag(env):units`	Complete
Row × column spatial analysis	`residual = ~ ar1(row):ar1(col)`	Complete
Irregular spatial data	`expc(x)`, `sexp(x, y)`	Complete
Repeated measures, permanent environment	`id(animal)` + `vm(animal, A)`, `ar1()`	Complete
Random regression	`us(leg(x, k)):id(animal)`, `pol()`	Complete
Maternal effects	additional terms `id(dam)`, `vm(dam, A)`	Complete
Diallels (GCA, SCA, reciprocals)	`mm(p1, p2)`, `id(cross)`	Complete
BLUP, PEV, reliability	`coef(fit, "random")`	Complete
Predicted means and SED	`predict(fit, classify = ...)`	Complete
Incremental and conditional Wald tests	`wald()`	Complete (conditional tests respect marginality)
Model comparison (LRT, AIC, BIC)	`lrt()`, `AIC`, `BIC`	Complete
Sample variogram	`variogram()`	Complete
Direct–maternal genetic covariance	`str(animal, dam, K = A)`	Complete
Sparse-matrix methods (millions of records)	`engine = "sparse"` (mixed model equations)	Complete for `id`, `vm`, `diag`, `us` and `str()` terms
Kenward–Roger degrees of freedom	`wald(fit, ddf = "KR")`	Complete (dense engine)
Generalised linear mixed models	—	Not available

MLREML has two computing engines. The dense engine uses the **V**-based method: it is exact, simple and easy to verify, and allows very general structures without special code, but limits its comfortable use to about 5,000–6,000 records on a personal computer with 8 GB of memory. The sparse engine solves the mixed model equations with a sparse Cholesky factorisation and the sparse inverse of the coefficient matrix, as the programs used in national genetic evaluations do; it supports the structures most used with large data sets and fits animal models with hundreds of thousands of records on the same computer (Sections 2.12 and 15.2). For GBLUP with a single genetic effect, the function `gblup()` offers an exact alternative based on the spectral decomposition of **G**. Chapter 15 documents computing times and memory.

1.5 A first example

To show the general shape of an analysis, consider a simulated experiment with 30 genotypes evaluated in 4 complete blocks. The model is

$$y_{ij} = \mu + \beta_j + g_i + e_{ij}, \qquad g_i \sim N\left(0, \sigma_g^2\right), \qquad e_{ij} \sim N(0, \sigma_e^2),$$

with fixed blocks and random genotypes.

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)), gen = factor(rep(1:30,
4)))
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d, trace = FALSE)
summary(fit)

## MLREML fit (AI-REML)
## Observations: 120   Fixed-effect columns: 4   Iterations: 6 (converged)
## REML log-likelihood: -192.9292   AIC: 389.86   BIC: 395.37
##
## Variance parameters:
##                 estimate std.error z.ratio bound
## id(gen)!sigma2   1.9621    0.5686    3.45     P
## R!units!sigma2   0.7992    0.1212    6.60     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

The output shows the number of observations, the fixed-effect columns, the iterations, the restricted log-likelihood and the information criteria. It then gives each variance parameter with its approximate standard error (obtained from the inverse of the average information matrix), the z ratio and a boundary code. The broad-sense heritability on a genotype-mean basis and its standard error are obtained with `vpredict()`:

```
vpredict(fit, H2 ~ V1 / (V1 + V2 / 4))

##    estimate std.error
## H2   0.9076   0.02802

head(coef(fit, "random", "id(gen)"))

##   level     blup    pev     se reliability
## 1     1  0.14700 0.2407 0.4906      0.8773
## 2     2 -1.03768 0.2407 0.4906      0.8773
## 3     3 -0.29958 0.2407 0.4906      0.8773
## 4     4  1.02779 0.2407 0.4906      0.8773
## 5     5  0.47789 0.2407 0.4906      0.8773
## 6     6 -0.08843 0.2407 0.4906      0.8773
```

The second instruction returns the BLUPs of the genotypes with their prediction error variance (PEV), standard error and reliability $1 - \text{PEV}/\sigma_g^2$.

2 Theoretical foundations

This chapter presents the model fitted by MLREML, the restricted likelihood, the average information algorithm as implemented, and the quantities the package computes after the fit: estimators of fixed effects, predictors of random effects, prediction error variances, predicted means, hypothesis tests and functions of the variance parameters.

2.1 The linear mixed model

Let $\mathbf{y}$ be the vector of n observations. The general linear mixed model is

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{u}_k + \mathbf{e}, \qquad (2.1)$$

where $\boldsymbol{\beta}$ is the vector of p fixed effects with design matrix $\mathbf{X}$ ($n \times p$, of full rank after removing redundant columns), $\mathbf{u}_k$ is the vector of q_k random effects of term k with design matrix $\mathbf{Z}_k$, and $\mathbf{e}$ is the vector of residuals. It is assumed that

$$\mathbf{u}_k \sim N\big(\mathbf{0}, \mathbf{G}_k(\boldsymbol{\theta})\big), \qquad \mathbf{e} \sim N\big(\mathbf{0}, \mathbf{R}(\boldsymbol{\theta})\big), \qquad \mathrm{Cov}(\mathbf{u}_k, \mathbf{u}_l) = \mathbf{0}, \ \mathrm{Cov}(\mathbf{u}_k, \mathbf{e}) = \mathbf{0},$$

and that the matrices $\mathbf{G}_k$ and $\mathbf{R}$ depend on a vector of variance parameters $\boldsymbol{\theta}$ (variances, covariances, correlations, factor loadings). The marginal distribution of the observations is

$$\mathbf{y} \sim N(\mathbf{X}\boldsymbol{\beta}, \mathbf{V}), \qquad \mathbf{V}(\boldsymbol{\theta}) = \sum_{k=1}^{K} \mathbf{Z}_k \, \mathbf{G}_k \mathbf{Z}_k{}' + \mathbf{R}. \qquad (2.2)$$

All the flexibility of the model lies in the form of $\mathbf{G}_k$ and $\mathbf{R}$. In a traditional analysis of variance each $\mathbf{G}_k$ is $\sigma_k^2 \mathbf{I}$ and $\mathbf{R} = \sigma_e^2 \mathbf{I}$. In quantitative genetics, by contrast, $\mathbf{G}$ can be $\sigma_a^2 \mathbf{A}$ (pedigree), $\sigma_g^2 \mathbf{G}_{\text{genomic}}$, or $\boldsymbol{\Sigma}_E \otimes \mathbf{K}$ in a multi-environment trial; and $\mathbf{R}$ can be diagonal with different variances per environment, or have the separable autoregressive structure of a field trial.

2.2 Structures as direct products

MLREML builds each $\mathbf{G}_k$ as a direct (Kronecker) product of small matrices, one for each factor that defines the term:

$$\mathbf{G}_k = \sigma_k^2 \, \boldsymbol{\Sigma}_{k1} \otimes \boldsymbol{\Sigma}_{k2} \otimes \cdots \otimes \boldsymbol{\Sigma}_{kC}. \qquad (2.3)$$

For example, for genotype-within-environment effects with E environments and N genotypes, $\mathbf{u}$ has EN elements ordered with the genotype varying fastest, and $\mathbf{G} = \boldsymbol{\Sigma}_E \otimes \mathbf{K}$. The design matrix of the term is the face-splitting product (row-wise Khatri–Rao product) of the design matrices of each factor: if $\mathbf{z}_{c,i}{}'$ is row i of the design matrix $\mathbf{Z}_c$ of factor c, row i of $\mathbf{Z}_k$ is $\mathbf{z}_{1,i}{}' \otimes \mathbf{z}_{2,i}{}' \otimes \cdots$.

A simple algebraic property allows the contribution of each term to $\mathbf{V}$ to be computed without ever forming the complete $\mathbf{Z}_k$ or $\mathbf{G}_k$. For the face-splitting product,

$$\mathbf{Z}_k(\boldsymbol{\Sigma}_1 \otimes \boldsymbol{\Sigma}_2)\mathbf{Z}_k{}' = (\mathbf{Z}_1\boldsymbol{\Sigma}_1\mathbf{Z}_1{}') \circ (\mathbf{Z}_2\boldsymbol{\Sigma}_2\mathbf{Z}_2{}'), \qquad (2.4)$$

where $\circ$ is the Hadamard (element-wise) product. When factor c is a classification variable, $\mathbf{Z}_c$ has a single one per row and $\mathbf{Z}_c\boldsymbol{\Sigma}_c\mathbf{Z}_c{}'$ is simply the matrix $\boldsymbol{\Sigma}_c$ indexed by the levels of each observation: its (i,j) element is $\Sigma_c[\ell_c(i), \ell_c(j)]$, where $\ell_c(i)$ is the level of factor c in observation i. Thus a genomic matrix of 6,547 animals is used only in the rows and columns of the animals with records, and the term `us(env):vm(gen, G)` contributes the elements $\Sigma_E[e_i, e_j]\, K[g_i, g_j]$ to $\mathbf{V}$. Identity (2.4) is what allows MLREML to combine freely any structure with any other.

Residuals are treated in the same way. The default structure, `units`, is the identity over observations. The structure `diag(env):units` gives $R_{ij} = \sigma^2_{e(i)}\delta_{ij}$, different residual variances per environment. The structure `ar1(row):ar1(col)` gives

$$R_{ij} = \sigma^2 \, \rho_r^{|r_i - r_j|} \, \rho_c^{|c_i - c_j|}, \qquad (2.5)$$

the separable spatial model of Gilmour et al. (1997), where r_i and c_i are the row and column of plot i. Because the matrix is built from the coordinates of each observation, the trial need not be a complete grid: missing plots simply do not appear.

2.3 Restricted likelihood

The fixed effects are removed from the likelihood by error contrasts $\mathbf{K}'\mathbf{y}$, with $\mathbf{K}'\mathbf{X} = \mathbf{0}$, whose distribution does not depend on $\boldsymbol{\beta}$ (Patterson and Thompson, 1971). Up to a constant, the restricted log-likelihood is

$$\ell_R(\boldsymbol{\theta}) = -\frac{1}{2}\left[\log|\mathbf{V}| + \log|\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}| + \mathbf{y}'\mathbf{P}\mathbf{y} + (n-p)\log(2\pi)\right], \qquad (2.6)$$

with the projection matrix

$$\mathbf{P} = \mathbf{V}^{-1} - \mathbf{V}^{-1}\mathbf{X}(\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}. \qquad (2.7)$$

REML corrects the bias of maximum likelihood due to the estimation of the fixed effects; in a balanced design its estimators coincide with those of the analysis of variance when the latter are positive. The value reported by MLREML includes the constant $(n-p)\log(2\pi)$ and equals that of `lme4` and `nlme`, which makes it easy to compare programs (Chapter 15). Two REML likelihoods are comparable only if the models have exactly the same fixed effects.

2.4 The average information algorithm

Let $\mathbf{V}_i = \partial\mathbf{V}/\,\partial\theta_i$. The first derivatives of (2.6), which form the score vector, are

$$s_i = \frac{\partial \ell_R}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(\mathbf{P}\mathbf{V}_i) - \mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{y}\right]. \qquad (2.8)$$

The observed information matrix contains traces of products of four $n \times n$ matrices, which are very expensive, and the Fisher information contains $\operatorname{tr}(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j)$, also costly. Gilmour, Thompson and Cullis (1995) observed that the average of the two, when $\mathbf{V}$ is linear in the parameters, is

$$\mathcal{I}_{A,ij} = \frac{1}{2}\,\mathbf{y}'\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\mathbf{P}\mathbf{y}, \qquad (2.9)$$

which requires only matrix–vector products: if $\mathbf{w}_i = \mathbf{V}_i\mathbf{P}\mathbf{y}$ and $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_m]$, then $\mathcal{I}_A = 1/2\;\mathbf{W}'\mathbf{P}\mathbf{W}$. The average information matrix is positive semi-definite and is a Gauss–Newton-type approximation to the information also when $\mathbf{V}$ is not linear in $\boldsymbol{\theta}$ (autoregressive correlations, factor loadings). The update is

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} + \left[\mathcal{I}_A^{(t)}\right]^{-1}\mathbf{s}^{(t)}. \qquad (2.10)$$

2.4.1 Implementation in MLREML

In each iteration the dense engine of MLREML (i) builds $\mathbf{V}$ with identity (2.4); (ii) computes its Cholesky factor $\mathbf{V} = \mathbf{U}'\mathbf{U}$, $\log|\mathbf{V}| = 2\sum\log u_{ii}$, $\mathbf{V}^{-1}$ and $\mathbf{P}$; (iii) for each free parameter forms $\mathbf{V}_i$ — again through Hadamard products, replacing the matrix of the component by its derivative — and computes $tr(\mathbf{P}\mathbf{V}_i) = \sum_{jk} P_{jk}\,V_{i,jk}$ and $\mathbf{w}_i$; and (iv) obtains $\mathbf{s}$ and $\mathcal{I}_A$. The sparse engine obtains the same quantities from the mixed model equations (Section 2.12). The derivatives of every structure are analytical; for example, for an autoregressive structure $\partial\rho^d/\,\partial\rho = d\rho^{d-1}$

for every distance $d > 0$, including distances smaller than one, and zero for $d = 0$, for an unstructured matrix the derivative with respect to σ_{ab} is $\mathbf{E}_{ab} + \mathbf{E}_{ba}$, and for a factor analytic model $\boldsymbol{\Lambda}\boldsymbol{\Lambda}' + \boldsymbol{\Psi}$ the derivative with respect to the loading λ_{lm} is $\mathbf{e}_l\boldsymbol{\lambda}_m{}' + \boldsymbol{\lambda}_m\mathbf{e}_l{}'$.

The pure Newton step (2.10) can leave the parameter space (negative variances, correlations larger than one, matrices that are not positive definite) or decrease the likelihood far from the optimum. MLREML includes three safeguards:

1. **Levenberg–Marquardt damping.** The step is computed as $\left(\mathcal{I}_A + \lambda\,\mathrm{diag}(\mathcal{I}_A)\right)^{-1}\mathbf{s}$. It starts with $\lambda = 0$ (pure AI step); if the candidate is not valid or reduces ℓ_R, λ is multiplied by 10. When λ is large the step approaches a short gradient step, which always increases the likelihood. After an accepted step, λ is reduced.
2. **Boundary constraints.** Variances are kept above a very small positive lower bound; the correlation of `cs` within $(-1/(L-1) + 10^{-4}, 0.999)$, because the $L \times L$ compound symmetry matrix is positive definite only for $\rho > -1/(L-1)$; and autoregressive correlations within $(-0.99, 0.999)$ when all distances are integers and within $(10^{-6}, 0.999)$ otherwise, because ρ^d is not defined for $\rho < 0$ and non-integer d. A parameter that ends at its bound is marked with the code `B` in the output, as in ASReml. A variance that is on its bound and whose score, or whose Newton step, points outside the parameter space is held on the bound and removed from the AI system, and the step of the other parameters is computed conditionally (an active-set method). Clipping the joint step instead could leave all the parameters practically still at a point that is not a maximum.
3. **Semi-definite projection.** If an unstructured matrix (`us`) ceases to be positive definite, it is projected onto the cone of positive semi-definite matrices by replacing its negative eigenvalues with a small positive value. This lets the algorithm approach solutions on the boundary (for example, genetic correlations close to one), which are frequent in multi-environment trials.

Convergence is declared when the change in ℓ_R is smaller than `tol` (by default 10^{-6}) and the Newton decrement $\mathbf{s}'\mathcal{I}_A^{-1}\mathbf{s}$, computed without the parameters held at their bound, is smaller than 10^{-3}, or the relative change in the parameters is negligible. In the latter case, if the Newton decrement is still large — which happens when an unstructured matrix converges to a singular matrix — a warning recommends checking the estimates and considering a reduced structure.

Starting values. The variance v_p of the ordinary least-squares residuals of the fixed-effects model is computed and split equally among the random terms and the residual. Level-specific variances (structures `diag`, `us`, `fa`) start at the observed residual variance within each level times the same proportion, which gives values on the right scale in multivariate models with traits measured in different units. Autoregressive correlations start at the value giving a correlation of 0.5 at the minimum distance between levels. Users can give their own starting values with the argument `start` and fix parameters with `fix`.

Standard errors. The inverse of $\mathcal{I}_A$ at the optimum approximates the asymptotic covariance matrix of $\widehat{\boldsymbol{\theta}}$ and gives the standard errors in the table of components. For parameters on the boundary this approximation is not valid and the standard error should be ignored.

2.5 Estimation of fixed effects and prediction of random effects

Given $\widehat{\boldsymbol{\theta}}$, the generalised least-squares estimator (empirical BLUE) of the fixed effects and its variance are

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}\mathbf{X}'\mathbf{V}^{-1}\mathbf{y}, \qquad \mathrm{Var}(\widehat{\boldsymbol{\beta}}) = \mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \qquad (2.11)$$

and the best linear unbiased predictor (empirical BLUP) of each random term is

$$\widehat{\mathbf{u}}_k = \mathbf{G}_k\mathbf{Z}_k{}'\mathbf{V}^{-1}\left(\mathbf{y} - \mathbf{X}\widehat{\boldsymbol{\beta}}\right) = \mathbf{G}_k\mathbf{Z}_k{}'\mathbf{P}\mathbf{y}. \qquad (2.12)$$

These expressions are the solution of Henderson's (1975) mixed model equations,

$$\begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{X}'\mathbf{R}^{-1}\mathbf{Z} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{X} & \mathbf{Z}'\mathbf{R}^{-1}\mathbf{Z} + \mathbf{G}^{-1} \end{bmatrix} \begin{bmatrix} \widehat{\boldsymbol{\beta}} \\ \widehat{\mathbf{u}} \end{bmatrix} = \begin{bmatrix} \mathbf{X}'\mathbf{R}^{-1}\mathbf{y} \\ \mathbf{Z}'\mathbf{R}^{-1}\mathbf{y} \end{bmatrix},$$

but the dense engine of MLREML evaluates them directly in the form (2.12), which does not require inverting $\mathbf{G}$. Singular relationship matrices (for example, a genomic matrix with fewer markers than individuals, or genetically identical lines) therefore cause no problems, provided $\mathbf{V}$ is positive definite. The sparse engine (Section 2.12) solves the equations themselves and needs $\mathbf{G}^{-1}$.

When a term uses a relationship matrix with more levels than those observed — for example, the matrix $\mathbf{A}$ of a pedigree with ancestors without records, or the genomic matrix of unphenotyped selection candidates — (2.12) also predicts those levels, because $\mathbf{G}_k\mathbf{Z}_k{}'$ includes their covariances with the observed individuals.

Prediction error variance. The inverse of the coefficient matrix of Henderson's equations, $\mathbf{C}$, gives the joint covariance of the estimation and prediction errors. Its blocks, in terms of $\mathbf{V}$, are

$$\mathbf{C}_{11} = (\mathbf{X}'\mathbf{V}^{-1}\mathbf{X})^{-1}, \quad \mathbf{C}_{21} = -\mathbf{G}\mathbf{Z}'\mathbf{V}^{-1}\mathbf{X}\mathbf{C}_{11}, \quad \mathbf{C}_{22} = \mathrm{Var}(\widehat{\mathbf{u}} - \mathbf{u}) = \mathbf{G} - \mathbf{G}\mathbf{Z}'\mathbf{P}\mathbf{Z}\mathbf{G}. \qquad (2.13)$$

The diagonal of $\mathbf{C}_{22}$ is the PEV of each BLUP. Reliability, widely used in animal breeding, is $r_i^2 = 1 - PEV_i/G_{ii}$, the squared correlation between the true and the predicted genetic value. The dense engine of MLREML computes the PEV by blocks of rows of $\mathbf{G}\mathbf{Z}'$, without forming $\mathbf{G}$, so the memory needed is proportional to n times the block size; the sparse engine takes them from the diagonal of the sparse inverse of $\mathbf{C}$ (Section 2.12).

2.6 Predicted means and standard errors of differences

A prediction is a linear combination $\boldsymbol{\tau} = \mathbf{L}\boldsymbol{\beta} + \mathbf{M}\mathbf{u}$ (Welham et al., 2004). The function `predict()` builds $\mathbf{L}$ by averaging the rows of the design matrix over the levels of the fixed factors not in the classification (simple averaging) and evaluating covariates at their mean; $\mathbf{M}$ selects the BLUPs of the random terms whose factors are all in the classification, and the other random terms are ignored because their expectation is zero. The prediction error variance is

$$\mathrm{Var}(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = \mathbf{L}\mathbf{C}_{11}\mathbf{L}' + \mathbf{L}\mathbf{C}_{12}\mathbf{M}' + \mathbf{M}\mathbf{C}_{21}\mathbf{L}' + \mathbf{M}\mathbf{C}_{22}\mathbf{M}', \qquad (2.14)$$

and the standard error of the difference between two predictions is $\mathrm{SED}_{ab} = \sqrt{v_{aa} + v_{bb} - 2v_{ab}}$. The average SED is the basis of the generalised heritability of Cullis, Smith and Coombes (2006), $H_C^2 = 1 - \overline{\mathrm{SED}}^2/\left(2\sigma_g^2\right)$, useful in unbalanced trials.

2.7 Hypothesis tests

Fixed effects. The Wald statistic of a hypothesis $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$, with $q = rank(\mathbf{L})$, is $W = \left(\mathbf{L}\widehat{\boldsymbol{\beta}}\right)'(\mathbf{L}\mathbf{C}_{11}\mathbf{L}')^{-1}\mathbf{L}\widehat{\boldsymbol{\beta}}$, asymptotically χ_q^2. The function `wald()` gives two versions for each term T of the fixed formula. The *incremental* statistic adjusts T for the terms that precede it in the formula (sequential or "type I" tests): if $\mathbf{U}'\mathbf{U} = \mathbf{C}_{11}^{-1} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{X}$ is the Cholesky factorisation, $\mathbf{L}$ is formed by the rows of $\mathbf{U}$ of the columns of T, and $W_T = \sum_{j\in T}\left(\mathbf{U}\widehat{\boldsymbol{\beta}}\right)_j^2$. The *conditional* statistic

adjusts T for all the terms that do not contain it, respecting the principle of marginality as ASReml does ("type II" tests): in `y ~ A * B` the main effect `A` is adjusted for `B` but not for `A:B`, and the interaction is adjusted for both main effects. It is computed as the incremental statistic after reordering the columns so that T follows the terms that do not contain it and precedes those that contain it. Adjusting each term for all the others, including the interactions that contain it, would test, with treatment contrasts, the effect of `A` at the reference level of `B`. Each statistic is accompanied by an F test. By default (`ddf = "auto"`), fits of the dense engine use the method of Kenward and Roger (1997), which adjusts the covariance matrix of $\widehat{\boldsymbol{\beta}}$ and the F statistic for the estimation of the variance parameters and gives the denominator degrees of freedom of each test (Section 2.13). With `ddf = "residual"`, and always for fits of the sparse engine, $F = W/q$ with $n - p$ denominator degrees of freedom; this approximation is adequate with many data and liberal in small experiments.

Variance parameters. Two nested models with the same fixed effects are compared with the likelihood ratio $LR = 2(\ell_{R,1} - \ell_{R,0})$. When testing whether a single variance is zero, the parameter lies on the boundary and the null distribution is a 50:50 mixture of χ^2_0 and χ^2_1 (Self and Liang, 1987; Stram and Lee, 1994), so the p-value is half that of a χ^2_1. The function `lrt()` applies this correction. Non-nested models are compared with AIC $= -2\ell_R + 2m$ and BIC $= -2\ell_R + m\log(n-p)$, where m is the number of variance parameters.

2.8 Functions of the variance parameters

Heritabilities, repeatabilities and genetic correlations are non-linear functions $f(\boldsymbol{\theta})$. Their approximate standard error is obtained by the delta method,

$$\mathrm{Var}\left[f(\widehat{\boldsymbol{\theta}})\right] \approx \nabla f(\widehat{\boldsymbol{\theta}})' \, \mathcal{I}_A^{-1} \, \nabla f(\widehat{\boldsymbol{\theta}}), \qquad (2.15)$$

which is what `vpredict()` does, with the gradient computed by central differences. The parameters are named `V1`, `V2`, … in the order of the table of components.

2.9 Relationship matrices

Pedigree. The additive relationship between two individuals is twice their coancestry coefficient. The matrix $\mathbf{A}$ is built with the tabular method: if s and d are the parents of i, $A_{ji} = 1/2\,(A_{js} + A_{jd})$ for $j < i$ and $A_{ii} = 1 + 1/2\,A_{sd}$, where $F_i = 1/2\,A_{sd}$ is the inbreeding coefficient. The inverse is built directly with Henderson's (1976) rules with inbreeding (Quaas, 1976): with $b_i = [1/2 - 1/4\,(F_s + F_d)]^{-1}$ (using $F = -1$ for unknown parents), b_i is added to element (i,i), $-b_i/2$ to (i,s), (s,i), (i,d), (d,i), and $b_i/4$ to (s,s), (d,d), (s,d), (d,s). The coefficients F are computed with the algorithm of Meuwissen and Luo (1992).

Genomic. With the matrix $\mathbf{M}$ of genotypes coded $0/1/2$ and allele frequencies p_j, VanRaden (2008) defines $\mathbf{W} = \mathbf{M} - 2\mathbf{p}'$ and $\mathbf{G} = \mathbf{W}\mathbf{W}'/\left[2\sum_j p_j\,(1-p_j)\right]$. The standardised alternative divides each column of $\mathbf{W}$ by $\sqrt{2p_j(1-p_j)}$ and the product by the number of markers. GBLUP with $\sigma^2_g\mathbf{G}$ is equivalent to ridge regression on the markers (Meuwissen et al., 2001; VanRaden, 2008).

Single-step. When only some individuals are genotyped, Legarra et al. (2009) combine pedigree and genomics in

$$\mathbf{H} = \mathbf{A} + \begin{bmatrix}\mathbf{A}_{12}\mathbf{A}_{22}^{-1}\\ \mathbf{I}\end{bmatrix}(\mathbf{G} - \mathbf{A}_{22})[\mathbf{A}_{22}^{-1}\mathbf{A}_{21} \quad \mathbf{I}], \qquad \mathbf{H}^{-1} = \mathbf{A}^{-1} + \begin{bmatrix}\mathbf{0} & \mathbf{0}\\ \mathbf{0} & \tau\mathbf{G}^{-1} - \omega\mathbf{A}_{22}^{-1}\end{bmatrix}, \qquad (2.16)$$

where index 2 corresponds to the genotyped individuals. To put $\mathbf{G}$ and $\mathbf{A}_{22}$ on the same scale, $\mathbf{G}$ is aligned as $\alpha + \beta\mathbf{G}$ so that the means of its diagonal and of all its elements match those of $\mathbf{A}_{22}$ (Christensen et al., 2012), and it is blended with a small proportion of $\mathbf{A}_{22}$ to ensure it is invertible.

2.10 Computational cost

The dominant step of each iteration of the dense engine is the factorisation and inversion of $\mathbf{V}$, with cost $O(n^3)$ in time and $O(n^2)$ in memory. Each parameter adds $O(n^2)$ operations for its trace and its vector $\mathbf{w}_i$. Prediction of the BLUPs and their PEV costs $O(qn^2)$ with memory $O(n \times \text{block})$. The cost of the sparse engine is that of the sparse Cholesky factorisation of the coefficient matrix of the mixed model equations and of its sparse inverse, both proportional to $\sum_i |S_i|^2$, where $|S_i|$ is the number of non-zero elements below the diagonal in column i of the Cholesky factor, plus one solve with the factor per parameter; its memory is proportional to the number of non-zero elements of the factor (Section 2.12). Chapter 15 presents measured times.

2.11 Correlated effects of several factors: direct–maternal models

In many species the phenotype of an individual depends on its own genes (direct genetic effect) and on the genes of its dam, expressed through the maternal environment she provides: milk yield, mothering ability, uterine capacity (maternal genetic effect). The two effects of the same animal are usually correlated, and the model of Willham (1963) treats them jointly:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_d\mathbf{a}_d + \mathbf{Z}_m\mathbf{a}_m + \cdots + \mathbf{e}, \qquad Var\begin{pmatrix}\mathbf{a}_d \\ \mathbf{a}_m\end{pmatrix} = \begin{pmatrix}\sigma_d^2 & \sigma_{dm} \\ \sigma_{dm} & \sigma_m^2\end{pmatrix} \otimes \mathbf{A} = \boldsymbol{\Sigma} \otimes \mathbf{A}, \qquad (2.17)$$

where $\mathbf{a}_d$ and $\mathbf{a}_m$ contain the direct and maternal breeding values of all the animals in the pedigree, $\mathbf{Z}_d$ links each record to the animal that produced it and $\mathbf{Z}_m$ links it to the dam of that animal. Both vectors are indexed by the same animals, but a record receives the direct value of one animal and the maternal value of another, so the term cannot be written as a direct product of components and identity (2.4) does not apply. Its contribution to $\mathbf{V}$ is a sum of three matrices,

$$\mathbf{Z}(\boldsymbol{\Sigma} \otimes \mathbf{A})\mathbf{Z}' = \sigma_d^2\,\mathbf{Z}_d\mathbf{A}\mathbf{Z}_d' + \sigma_m^2\,\mathbf{Z}_m\mathbf{A}\mathbf{Z}_m' + \sigma_{dm}(\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m' + \mathbf{Z}_m\mathbf{A}\mathbf{Z}_d'), \qquad \mathbf{Z} = [\mathbf{Z}_d, \mathbf{Z}_m], \qquad (2.18)$$

which is linear in the three parameters; their derivatives $\mathbf{V}_i$ are the three matrices of (2.18). The (i, j) element of $\mathbf{Z}_d\mathbf{A}\mathbf{Z}_m'$ is the relationship between the animal of record i and the dam of record j. Records whose dam is unknown have a row of zeros in $\mathbf{Z}_m$: they contribute to the estimation of the direct effects and are kept in the analysis.

MLREML fits this model with the term `str(f1, f2, ..., K = K, cov = "us")`, which generalises (2.17) to $m \geq 2$ factors whose levels are the rows of the same matrix $\mathbf{K}$: $\mathbf{u} = (\mathbf{u}_1', \ldots, \mathbf{u}_m')'$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$ and $Var(\mathbf{u}) = \boldsymbol{\Sigma} \otimes \mathbf{K}$, with $\boldsymbol{\Sigma}$ unstructured (`cov = "us"`, $m(m+1)/2$ parameters) or diagonal (`cov = "diag"`). The BLUP of effect a and its prediction error variance follow from (2.12) and (2.13):

$$\hat{\mathbf{u}}_a = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{K}\mathbf{Z}_b'\mathbf{P}\mathbf{y}, \qquad PEV_{a,l} = \Sigma_{aa}K_{ll} - \mathbf{g}_{a,l}'\mathbf{P}\mathbf{g}_{a,l}, \qquad \mathbf{g}_{a,l}' = \sum_{b=1}^{m} \Sigma_{ab}\ \mathbf{k}_l'\mathbf{Z}_b', \qquad (2.19)$$

where $\mathbf{k}_l'$ is row l of $\mathbf{K}$. Besides the direct heritability $h_d^2 = \sigma_d^2/\sigma_P^2$ and the direct–maternal genetic correlation $r_{dm} = \sigma_{dm}/(\sigma_d\sigma_m)$, the total heritability of Willham (1972), $h_T^2 = (\sigma_d^2 + 1.5\sigma_{dm} + 0.5\sigma_m^2)/\sigma_P^2$, measures the response to selection on the phenotype. The maternal genetic variance is identifiable only if the pedigree links dams with records of their own or of relatives, and estimates of σ_{dm} are often negative and imprecise (Meyer, 1992); a maternal permanent-environment effect is added, if necessary, as a separate term `id(dam)`.

2.12 The sparse engine: REML with the mixed model equations

The method of the previous sections works with the $n \times n$ matrix $\mathbf{V}$ and is exact and general, but its cost grows with n^3. The programs used for national genetic evaluations (ASReml, DMU, WOMBAT, BLUPF90) work instead with Henderson's mixed model equations, whose coefficient matrix is sparse. MLREML includes this second **sparse engine**. Let $\mathbf{T} = [\mathbf{X}, \mathbf{Z}]$, $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_K]$ and $\mathbf{G} = \text{blockdiag}(\mathbf{G}_1, \ldots, \mathbf{G}_K)$. The coefficient matrix of the equations is

$$\mathbf{C} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{T} + \begin{bmatrix}\mathbf{0} & \mathbf{0}\\ \mathbf{0} & \mathbf{G}^{-1}\end{bmatrix}, \qquad \mathbf{C}\begin{bmatrix}\widehat{\boldsymbol{\beta}}\\ \widehat{\mathbf{u}}\end{bmatrix} = \mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \qquad (2.20)$$

Because $|\mathbf{V}||\mathbf{X}'\mathbf{V}^{-1}\mathbf{X}| = |\mathbf{R}||\mathbf{G}||\mathbf{C}|$, the restricted log-likelihood (2.6) can be written without $\mathbf{V}$:

$$\ell_R = -\frac{1}{2}[\log|\mathbf{R}| + \log|\mathbf{G}| + \log|\mathbf{C}| + \mathbf{y}'\mathbf{P}\mathbf{y} + (n-p)\log(2\pi)], \qquad \mathbf{y}'\mathbf{P}\mathbf{y} = \mathbf{y}'\mathbf{R}^{-1}\mathbf{y} - \left[\widehat{\boldsymbol{\beta}}', \widehat{\mathbf{u}}'\right]\mathbf{T}'\mathbf{R}^{-1}\mathbf{y}. \qquad (2.21)$$

The value of ℓ_R is the same as that of the dense engine. With $\hat{\mathbf{e}} = \mathbf{y} - \mathbf{T}\left[\widehat{\boldsymbol{\beta}}', \widehat{\mathbf{u}}'\right]'$, the score (2.8) becomes

$$s_i = -\frac{1}{2}\left[tr(\mathbf{R}^{-1}\mathbf{R}_i) + tr(\mathbf{G}^{-1}\mathbf{G}_i) + tr(\mathbf{C}^{-1}\mathbf{C}_i) + \hat{\mathbf{e}}'\frac{\partial\mathbf{R}^{-1}}{\partial\theta_i}\hat{\mathbf{e}} + \widehat{\mathbf{u}}'\frac{\partial\mathbf{G}^{-1}}{\partial\theta_i}\widehat{\mathbf{u}}\right], \qquad (2.22)$$

with $\mathbf{R}_i = \partial\mathbf{R}/\,\partial\theta_i$, $\mathbf{G}_i = \partial\mathbf{G}/\,\partial\theta_i$ and $\mathbf{C}_i = \partial\mathbf{C}/\,\partial\theta_i$, which is blockdiag$(\mathbf{0}, \partial\mathbf{G}^{-1}/\,\partial\theta_i)$ for a parameter of $\mathbf{G}$ and $\mathbf{T}'(\partial\mathbf{R}^{-1}/\,\partial\theta_i)\mathbf{T}$ for a parameter of $\mathbf{R}$.

Sparse inverse. The trace $tr(\mathbf{C}^{-1}\mathbf{C}_i)$ needs only the elements of $\mathbf{C}^{-1}$ in the positions where $\mathbf{C}_i$ is not zero, all of which lie in the pattern of the Cholesky factor of $\mathbf{C}$. Let $\mathbf{C} = \mathbf{L}\mathbf{L}'$ after a fill-reducing permutation of rows and columns, and let S_i be the rows of the non-zero elements of column i of $\mathbf{L}$ below the diagonal. The elements Z_{ji} of $\mathbf{C}^{-1}$ in the pattern of $\mathbf{L}$ are obtained, column by column from the last to the first, with the recursions of Takahashi et al. (1973) and Erisman and Tinney (1975), used in animal breeding by Misztal and Perez-Enciso (1993):

$$Z_{ji} = -\sum_{k\in S_i}\frac{L_{ki}}{L_{ii}}Z_{kj} \quad (j \in S_i), \qquad Z_{ii} = \frac{1}{L_{ii}^2} - \sum_{k\in S_i}\frac{L_{ki}}{L_{ii}}Z_{ki}. \qquad (2.23)$$

Only elements in the pattern of $\mathbf{L}$ are needed on the right-hand side, because that pattern is closed: if k and j belong to S_i, element (k, j) belongs to the pattern. The cost is proportional to $\sum_i |S_i|^2$, the same order as that of the factorisation itself, and the memory equals that of $\mathbf{L}$. The diagonal of the block of the random effects gives the prediction error variances, and the block of the fixed effects gives $Var(\widehat{\boldsymbol{\beta}})$, at no additional cost.

Average information. The working variates $\mathbf{w}_i = \mathbf{V}_i\mathbf{P}\mathbf{y}$ of (2.9) are computed without $\mathbf{V}$, since $\mathbf{P}\mathbf{y} = \mathbf{R}^{-1}\hat{\mathbf{e}}$ and $\mathbf{Z}'\mathbf{P}\mathbf{y} = \mathbf{G}^{-1}\widehat{\mathbf{u}}$: $\mathbf{w}_i = \mathbf{Z}\mathbf{G}_i\mathbf{G}^{-1}\widehat{\mathbf{u}}$ for a parameter of $\mathbf{G}$ and $\mathbf{w}_i = \mathbf{R}_i\mathbf{R}^{-1}\hat{\mathbf{e}}$ for a parameter of $\mathbf{R}$. Since $\mathbf{P} = \mathbf{R}^{-1} - \mathbf{R}^{-1}\mathbf{T}\mathbf{C}^{-1}\mathbf{T}'\mathbf{R}^{-1}$,

$$\mathcal{I}_{A,ij} = \frac{1}{2}\mathbf{w}_i'\mathbf{P}\mathbf{w}_j = \frac{1}{2}\left(\mathbf{w}_i'\mathbf{R}^{-1}\mathbf{w}_j - \mathbf{t}_i'\mathbf{C}^{-1}\mathbf{t}_j\right), \qquad \mathbf{t}_i = \mathbf{T}'\mathbf{R}^{-1}\mathbf{w}_i, \qquad (2.24)$$

which needs one solve with the factor of $\mathbf{C}$ per parameter (Johnson and Thompson, 1995; Gilmour et al., 1995).

Structures. For a term $\mathbf{G}_k = \mathbf{K}_1 \otimes \boldsymbol{\Sigma} \otimes \mathbf{K}_2$, where $\boldsymbol{\Sigma}$ is the only matrix with parameters (or σ^2 for a term without a variance structure) and $\mathbf{K}_1$, $\mathbf{K}_2$ are identities or relationship matrices, $\mathbf{G}_k^{-1} = \mathbf{K}_1^{-1} \otimes \boldsymbol{\Sigma}^{-1} \otimes \mathbf{K}_2^{-1}$; $\log|\mathbf{G}_k|$ follows from $\log|\mathbf{A} \otimes \mathbf{B}| = b\log|\mathbf{A}| + a\log|\mathbf{B}|$ for matrices of orders a and b; and $\mathbf{G}_i\mathbf{G}^{-1}\hat{\mathbf{u}} = (\mathbf{I} \otimes \boldsymbol{\Sigma}_i\boldsymbol{\Sigma}^{-1} \otimes \mathbf{I})\hat{\mathbf{u}}$ involves only the small matrix $\boldsymbol{\Sigma}$. The relationship matrices are needed only through their inverses, so $\mathbf{A}^{-1}$ built with Henderson's rules (Section 2.9) can be given directly. A `str()` term is $\boldsymbol{\Sigma} \otimes \mathbf{K}$ with $\mathbf{Z} = [\mathbf{Z}_1, \ldots, \mathbf{Z}_m]$. The residual matrix is block diagonal, with one block per record: $\sigma_e^2\mathbf{I}$ (`units`), $diag\left(\sigma^2_{e(i)}\right)$ (`diag(f):units`) or, in multivariate models, the submatrix of $\boldsymbol{\Sigma}_e$ of the traits observed in each record (`us(trait):units`).

Implementation. $\mathbf{C}$ is assembled with a fixed pattern (the union of all the positions that can be non-zero), so that the fill-reducing ordering and the symbolic factorisation of CHOLMOD (Chen et al., 2008), available in R through the `Matrix` package, are computed once and only the numerical factorisation is repeated in each iteration. The recursions (2.23) are programmed in C++ and use a sorted merge of each column with the columns of its rows, without searches. The candidate points of the damped steps need only the factorisation, because $\log|\mathbf{C}| = 2\sum_i \log L_{ii}$. Predicted means use $Var(\hat{\boldsymbol{\tau}} - \boldsymbol{\tau}) = [\mathbf{L}, \mathbf{M}]\mathbf{C}^{-1}[\mathbf{L}, \mathbf{M}]'$, equivalent to (2.14), obtained by solving with the factor of $\mathbf{C}$.

The sparse engine supports random terms built from `id`, `vm`, `diag` and `us` components (with factors, `pol()`, `leg()` or `mm()` bases), `str()` terms, and the residual structures `units`, `diag(f):units` and `us(trait):units`. Structures with correlation parameters (`cs`, `ar1`, `expc`, `sexp`) and `fa` are fitted with the dense engine. With `engine = "auto"`, the default, MLREML uses the sparse engine when the data have more than 5,000 records and the model is supported, and the dense engine otherwise. Both engines give the same likelihood, estimates, BLUPs and prediction error variances up to rounding error (Section 15.1). Because the sparse engine needs $\mathbf{G}^{-1}$, the relationship matrices must be positive definite; a singular genomic matrix must be bent or blended (Section 13.7) or fitted with the dense engine.

2.13 Kenward–Roger degrees of freedom

The covariance matrix $\boldsymbol{\Phi} = \left(\mathbf{X}'\widehat{\mathbf{V}}^{-1}\mathbf{X}\right)^{-1}$ of (2.11) treats the variance parameters as known. In small experiments it underestimates the variance of $\widehat{\boldsymbol{\beta}}$, and F tests with $n - p$ denominator degrees of freedom reject a true hypothesis too often. Kenward and Roger (1997) proposed an adjusted covariance matrix,

$$\boldsymbol{\Phi}_A = \boldsymbol{\Phi} + 2\boldsymbol{\Phi}\left\{\sum_{i,j} W_{ij}\left(\mathbf{Q}_{ij} - \mathbf{P}_i\boldsymbol{\Phi}\mathbf{P}_j - \frac{1}{4}\mathbf{R}_{ij}\right)\right\}\boldsymbol{\Phi}, \qquad (2.25)$$

with $\mathbf{P}_i = -\mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{Q}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_i\mathbf{V}^{-1}\mathbf{V}_j\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{R}_{ij} = \mathbf{X}'\mathbf{V}^{-1}\mathbf{V}_{ij}\mathbf{V}^{-1}\mathbf{X}$, $\mathbf{V}_{ij} = \partial^2\mathbf{V}/\,\partial\theta_i\,\partial\theta_j$, and $\mathbf{W}$ the inverse of the expected information matrix, whose elements are $\frac{1}{2}tr\left(\mathbf{P}\mathbf{V}_i\mathbf{P}\mathbf{V}_j\right)$. For a hypothesis $\mathbf{L}\boldsymbol{\beta} = \mathbf{0}$ with $q = rank(\mathbf{L})$ they use the statistic

$$F = \frac{\lambda}{q}\left(\mathbf{L}\widehat{\boldsymbol{\beta}}\right)'\left(\mathbf{L}\boldsymbol{\Phi}_A\mathbf{L}'\right)^{-1}\mathbf{L}\widehat{\boldsymbol{\beta}}, \qquad (2.26)$$

and choose the scale λ and the denominator degrees of freedom m so that the mean and the variance of F approximately match those of an $F(q, m)$ distribution. With $\mathbf{\Theta} = \mathbf{L}'(\mathbf{L}\mathbf{\Phi}\mathbf{L}')^{-1}\mathbf{L}$,

$$A_1 = \sum_{i,j} W_{ij}\ tr(\mathbf{\Theta}\mathbf{\Phi}\mathbf{P}_i\mathbf{\Phi})tr\big(\mathbf{\Theta}\mathbf{\Phi}\mathbf{P}_j\mathbf{\Phi}\big), \qquad A_2 = \sum_{i,j} W_{ij}\ tr\big(\mathbf{\Theta}\mathbf{\Phi}\mathbf{P}_i\mathbf{\Phi}\mathbf{\Theta}\mathbf{\Phi}\mathbf{P}_j\mathbf{\Phi}\big),$$

$$B = \frac{A_1 + 6A_2}{2q}, \quad g = \frac{(q+1)A_1 - (q+4)A_2}{(q+2)A_2}, \quad c_1 = \frac{g}{3q + 2(1-g)}, \quad c_2 = \frac{q-g}{3q+2(1-g)}, \quad c_3 = \frac{q+2-g}{3q+2(1-g)},$$

$$E^* = \left(1 - \frac{A_2}{q}\right)^{-1}, \quad V^* = \frac{2}{q}\,\frac{1 + c_1 B}{(1 - c_2 B)^2(1 - c_3 B)}, \quad \rho = \frac{V^*}{2E^{*2}}, \quad m = 4 + \frac{q+2}{q\rho - 1}, \quad \lambda = \frac{m}{E^*(m-2)}. \qquad (2.27)$$

In balanced designs in which the F test is exact, the method reproduces it: $\lambda = 1$ and m equals the classical degrees of freedom (for example, those of the whole-plot error in a split-plot design).

MLREML computes (2.25)–(2.27) for every incremental and conditional test of `wald()` (Section 2.7); the matrix $\mathbf{L}$ of each test is given by the rows of the Cholesky factor used to compute the Wald statistic. All free variance parameters are included except those on the boundary of the parameter space, which are treated as known. The second derivatives $\mathbf{V}_{ij}$ are zero for the structures that are linear in their parameters (`id`, `vm`, `diag`, `us`, `str()`) and are included analytically for the others (`cs`, `ar1`, `expc`, `sexp`, loadings of `fa`, and products of two components with parameters, such as `ar1(row):ar1(col)` with its scale). The adjustment uses the $n \times n$ derivative matrices and is available for fits of the dense engine stored with `store = TRUE`; when the expected information would need more than 2 GB of memory, the average information matrix is used in its place. With the sparse engine, intended for large data sets in which the adjustment is negligible, `wald()` uses $n - p$ degrees of freedom. The results agree with those of the `pbkrtest` package (Halekoh and Højsgaard, 2014) to five or more significant digits (Section 15.1).

3 Model syntax

3.1 The function `mlreml()`

The general call is

```
fit <- mlreml(fixed, random = NULL, residual = NULL, data,
              start = NULL, fix = NULL, maxit = 50, tol = 1e-6,
              pev = TRUE, trace = TRUE, store = TRUE,
              engine = c("auto", "dense", "sparse"))
```

Arguments of `mlreml()`.

Argument	Meaning
`fixed`	Two-sided formula for the response and the fixed effects, with the usual R syntax (`y ~ rep + gen`).
`random`	One-sided formula with the random terms (`~ gen + rep:block`).
`residual`	One-sided formula with a single term for the residual structure. Default `~ units`.

Argument	Meaning
`Data`	Data frame. Rows with a missing response or predictor are removed, except for missing values of the factors of a `str()` term (Section 2.11).
`start`	Named vector of starting values of the variance parameters.
`Fix`	Names of parameters held fixed at their starting values.
`maxit`, `tol`	Maximum number of iterations and tolerance for the change in ℓ_R.
`Pev`	Compute the prediction error variances of the BLUPs.
`trace`	Show the iteration history.
`store`	Keep the matrices needed by `predict()` and by the Kenward–Roger adjustment of `wald()`; `FALSE` saves memory.
`engine`	Computing engine: `"dense"` (variance matrix **V**), `"sparse"` (mixed model equations) or `"auto"` (sparse with more than 5,000 records when the model is supported, dense otherwise). See Section 2.12.

Fixed effects are handled as in `lm()`: factors generate treatment contrasts, interactions are written with `:` or `*`, and linearly dependent columns are removed automatically (the "aliased" effects of ASReml).

3.2 Random terms

Each random term is a product of **components** separated by colons. Each component consists of a **structure** applied to a **basis**. The basis defines the level to which each observation belongs and the structure defines the covariance (or correlation) matrix of the effects of that basis.

3.2.1 Bases

Bases of a component.

Basis	Description
`F`	A factor or classification variable in the data. If numeric, its distinct values are the levels, sorted.
`pol(x, k)`	Raw polynomial of order k with intercept: columns $1, x, \ldots, x^k$.
`leg(x, k)`	Normalised Legendre polynomials of order k of x rescaled to $[-1,1]$, as in random regression models in animal breeding.
`mm(f1, f2, ...)`	Multi-membership factor: each observation belongs to the levels of `f1` and of `f2`. Used for the general combining ability of the two parents.
`units`	The observation units (one level per record; in multivariate models, one level per original record).

3.2.2 Structures

The table below lists the available structures, the matrix they generate for a component with L levels and their number of parameters. **Correlation** structures have no scale of their own; **variance** structures do.

Variance structures.

Structure	$L \times L$ matrix	Parameters	Type
`id(f)` or `f`	$\mathbf{I}_L$	0	correlation

Structure	$L \times L$ matrix	Parameters	Type
`vm(f, K)`	known matrix $\mathbf{K}$ (A, G, H, kinship)	0	correlation
`cs(f)`	$(1-\rho)\mathbf{I} + \rho\mathbf{J}$	1	correlation
`ar1(f)`	$\rho^{\lvert x_a - x_b \rvert}$ (actual distances if the levels are numbers)	1	correlation
`expc(x)`	$\rho^{\lvert x_a - x_b \rvert}$ with irregular continuous x	1	correlation
`sexp(x, y)`	$\rho^{d_{ab}}$ with d the Euclidean distance in the plane	1	correlation
`diag(f)`	$\mathrm{diag}(\sigma_1^2, \ldots, \sigma_L^2)$	L	variance
`us(f)`	unstructured matrix	$L(L+1)/2$	variance
`fa(f, k)`	$\mathbf{\Lambda\Lambda}' + \mathbf{\Psi}$, $\mathbf{\Lambda}$ of size $L \times k$	$Lk + L$ (minus $k(k-1)/2$ zeros)	variance

A term made only of correlation components receives a scale parameter σ^2. A term can have at most one variance component, which then carries the scale; for example, in `us(env):vm(gen, G)` the `us` matrix is the genetic covariance among environments on the scale of the data and `vm` provides the correlations among genotypes. Two variance components in the same term are not identifiable (multiplying one by c and dividing the other by c does not change the model), and MLREML rejects them with an error message.

In `fa(f, k)` with $k > 1$, the loadings above the diagonal of $\mathbf{\Lambda}$ are fixed at zero to remove the rotational indeterminacy. The estimated loadings can be rotated afterwards for interpretation; `vc_matrix()` also returns the principal-component rotated solution.

Correlated terms. The term `str(f1, f2, ..., K = K, cov = "us")` is not a product but a sum of components (Section 2.11): it defines one random effect for each of the factors `f1`, `f2`, …, whose levels are the rows of the matrix `K`, with $Var(\mathbf{u}) = \mathbf{\Sigma} \otimes \mathbf{K}$, where $\mathbf{\Sigma}$ is unstructured (`cov = "us"`, the default) or diagonal (`cov = "diag"`). The factors must take values in `rownames(K)`; a missing value (`NA`, `""` or `"0"`) means that the record receives no effect from that factor, and the record is kept. If `K` is omitted, the identity matrix is used. The parameters are named as those of `us()`: for example, `str(animal, dam, K = A)!dam:animal` is the direct–maternal covariance.

3.2.3 Examples of terms

Examples of random terms.

Term	Model
`~ gen`	Independent genotypes, one variance.
`~ rep:block`	Incomplete blocks within replicates.
`~ vm(animal, A)`	Animal model: $\mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A})$.
`~ vm(animal, Ai), Ai <- Ainverse(ped)`	The same model with $\mathbf{A}$ given by its sparse inverse.
`~ vm(animal, A) + animal`	Repeatability model: genetic value plus permanent environment.
`~ str(animal, dam, K = A)`	Direct and maternal genetic effects with their covariance, $\mathbf{\Sigma} \otimes \mathbf{A}$.
`~ str(animal, dam, K = A) + id(dam)`	The same with a maternal permanent-environment effect.

Term	Model
`~ vm(gen, G)`	GBLUP.
`~ env:gen`	Homogeneous G×E interaction (compound symmetry).
`~ diag(env):id(gen)`	A different genetic variance in each environment, no covariance.
`~ us(env):vm(gen, G)`	Genomic MET with unstructured genetic covariance among environments.
`~ fa(env, 2):id(gen)`	MET with a two-factor factor analytic model.
`~ us(trait):vm(animal, A)`	Additive genetic covariance matrix among traits.
`~ us(leg(dim, 2)):vm(animal, A)`	Genetic random regression of order 2.
`~ mm(p1, p2) + cross`	Diallel: GCA of the parents and SCA of the cross.
`~ units`	*Nugget* effect (additional independent error) in spatial models.

The matrices given to `vm()` and `str()` are looked up in the environment where the fixed formula was created, usually the workspace. They must have row names containing all levels of the factor; they may include additional levels. A relationship matrix may also be given through its inverse if it carries the attribute `INVERSE = TRUE`, as the result of `Ainverse()` does: the sparse engine uses it directly and the dense engine inverts it.

3.3 Residual structures

The `residual` argument accepts a single term with the same syntax. The most common structures are:

Residual structures.

Residual	**R**
`~ units`	$\sigma_e^2\mathbf{I}$ (default)
`~ diag(env):units`	A different residual variance in each environment
`~ ar1(row):ar1(col)`	Separable spatial model (2.5)
`~ diag(env):ar1(row):ar1(col)`	Spatial by environment with own variances and common correlations
`~ ar1(time):id(animal)`	Autocorrelated repeated measures within each individual
`~ expc(time):id(animal)`	Same, with irregular times
`~ sexp(x, y)`	Isotropic spatial model with irregular coordinates
`~ us(trait):units`	Residual covariance among traits of the same record (multivariate default)

When the residual is spatial, a random term `units` is usually added to represent independent measurement error.

3.4 Multivariate models

If the left-hand side of `fixed` is `cbind(y1, y2, ...)`, MLREML stacks the data in long format, creates the factor `trait` with one level per trait and the basis `units` identifying each original record. The fixed-effects formula is written in terms of `trait`:

```
mlreml(cbind(weight, height) ~ trait + trait:sex,
       random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = mydata)
```

Records with some trait missing are kept for the observed traits, which is one of the advantages of multivariate REML analysis.

3.5 Parameter names, starting values and constraints

Each parameter is named `term!component_level`; residual parameters start with `R!`. For example, `us(env):vm(gen, G)!env_E2:E1` is the genetic covariance between environments E2 and E1, and `R!diag(env):units!env_E1` is the residual variance of E1. These names are used in `start` and `fix`. To fix the residual variance at 1 and estimate the rest:

```
fit0 <- mlreml(y ~ 1, random = ~ gen, data = d, trace = FALSE)
fit1 <- mlreml(y ~ 1, random = ~ gen, data = d,
               start = c("R!units!sigma2" = 1), fix = "R!units!sigma2")
```

Fixing all parameters makes it possible to evaluate the likelihood and the BLUPs with known components, for example published components or components estimated in another data set.

3.6 The fitted object

`mlreml()` returns a list of class `"mlreml"`. Its main elements are:

Elements of the `mlreml` object.

Element	Content
`varcomp`	Table of variance parameters: estimate, standard error, z ratio, boundary code.
`theta`, `vcov_theta`	Parameter vector and its approximate covariance ($\mathcal{I}_A^{-1}$).
`beta`, `vcov_beta`	Fixed effects and their covariance matrix (with the sparse engine and more than 1,000 fixed effects, a diagonal matrix with the variances; `wald()` computes the complete matrix when it needs it).
`blups`	List with one data frame per random term: level, BLUP, PEV, standard error and reliability (for a `str()` term, also the column `effect`).
`fitted`, `residuals`	Fitted values ($\mathbf{X}\hat{\boldsymbol{\beta}} + \mathbf{Z}\hat{\mathbf{u}}$) and conditional residuals.
`loglik`, `AIC`, `BIC`	Restricted likelihood and information criteria.
`history`, `converged`	Iteration history and convergence flag.
`data`	Data used (in long format if the model is multivariate).
`engine`	Engine used, `"dense"` or `"sparse"`.
`sparse`	Sparse engine only: number of equations (`N`), non-zero elements of the Cholesky factor (`nnzL`) and, with `store = TRUE`, the factor itself, used by `predict()` and `wald()`.

The available methods are `summary()`, `print()`, `coef()` (fixed or random), `fitted()`, `residuals()`, `vcov()`, `logLik()`, `AIC()`, `BIC()`, `plot()` and `predict()`. The functions `wald()`, `vpredict()`, `lrt()`, `vc_matrix()` and `variogram()` complete the inference.

4 Example 1: an oat trial in an alpha-lattice design

4.1 The data

The data set `john.alpha` in the `agridat` package comes from John and Williams (1995). It contains 24 oat varieties evaluated in an alpha design with 3 replicates, each divided into 6 incomplete blocks of 4 plots. The response is yield (t/ha). It is a classic example of the recovery of inter-block information: since each block contains only 4 of the 24 varieties, comparisons between varieties that do not share a block depend on the block variance.

```
library(agridat)
data(john.alpha)
d <- john.alpha
str(d)
```

```
## 'data.frame':    72 obs. of  7 variables:
##  $ plot : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ block: Factor w/ 6 levels "B1","B2","B3",..: 1 1 1 1 2 2 2 2 3 3 ...
##  $ gen  : Factor w/ 24 levels "G01","G02","G03",..: 11 4 5 22 21 10 20 2 23
14 ...
##  $ yield: num  4.12 4.45 5.88 4.58 4.65 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...
```

```
with(d, table(rep, block))[, 1:6]
```

```
##     block
## rep  B1 B2 B3 B4 B5 B6
##   R1  4  4  4  4  4  4
##   R2  4  4  4  4  4  4
##   R3  4  4  4  4  4  4
```

4.2 Varieties as fixed effects: adjusted means

When the aim is to compare varieties, they are treated as fixed and blocks as random. The model is

$$y_{ijk} = \mu + r_j + \tau_i + b_{k(j)} + e_{ijk}, \qquad b_{k(j)} \sim N(0, \sigma_b^2), \qquad e_{ijk} \sim N(0, \sigma_e^2).$$

```
fit_f <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = d, trace =
FALSE)
summary(fit_f)
```

```
## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 26   Iterations: 5 (converged)
## REML log-likelihood: -32.4492   AIC: 68.90   BIC: 72.56
##
## Variance parameters:
##                   estimate std.error z.ratio bound
## rep:block!sigma2  0.06195   0.03656    1.69     P
## R!units!sigma2    0.08522   0.02185    3.90     P
```

```
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
wald(fit_f)
```

```
##          term df wald.inc    F.inc ddf.inc     p.inc wald.con    F.con
ddf.con
## 1 (Intercept)  1  4338.49 4338.495   11.52 3.307e-16  4338.49 4338.495
11.52
## 2         rep  2    18.42    9.212   11.52 4.078e-03    18.42    9.212
11.52
## 3         gen 23   125.30    5.363   35.50 4.495e-06   125.30    5.363
35.50
##       p.con   p.chisq
## 1 3.307e-16 0.000e+00
## 2 4.078e-03 9.981e-05
## 3 4.495e-06 4.588e-16
```

The conditional Wald test for gen tests the hypothesis that all varieties are equal, adjusted for replicates. With 72 plots and 26 fixed-effect columns the residual degrees of freedom, 46, overstate the information about the varieties, part of which comes from the comparison of incomplete blocks. The Kenward–Roger method gives 35.5 denominator degrees of freedom for gen and 11.5 for rep, whose information comes mostly from the 18 incomplete blocks, and slightly reduces the F statistic of gen (5.363 instead of 5.448). The p-values of the χ^2 test (p.chisq) ignore this uncertainty and are too small. The conditional statistic of the intercept equals its incremental statistic, since every term contains the intercept.

The adjusted means of the varieties and their standard errors of differences are obtained with predict():

```
pf <- predict(fit_f, classify = "gen")
head(pf$pvals[order(-pf$pvals$predicted.value), ], 8)
```

```
##    gen predicted.value std.error
## 1  G01           5.108    0.1955
## 5  G05           5.037    0.1955
## 15 G15           4.969    0.1955
## 19 G19           4.840    0.1955
## 21 G21           4.795    0.1955
## 14 G14           4.776    0.1955
## 13 G13           4.758    0.1955
## 12 G12           4.755    0.1955
```

```
pf$avsed
```

```
## [1] 0.2647
```

```
range(pf$sed[upper.tri(pf$sed)])
```

```
## [1] 0.2574 0.2699
```

The SED is not constant: two varieties that share blocks are compared more precisely than two that never occur together. This is the information a complete-block analysis would ignore. An approximate least significant difference at 5 % is $t_{0.975,\mathrm{df}} \times \overline{\mathrm{SED}}$:

```
df_error <- nrow(d) - fit_f$p
qt(0.975, df_error) * pf$avsed
```

```
## [1] 0.5329
```

4.3 Varieties as random effects: BLUP and heritability

In a breeding programme the varieties are a sample from a population and the aim is to predict their genetic value. With random varieties one obtains BLUPs, which shrink the means towards the overall mean in proportion to their uncertainty.

```
fit_r <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace =
FALSE)
summary(fit_r)
```

```
## MLREML fit (AI-REML)
## Observations: 72   Fixed-effect columns: 3   Iterations: 4 (converged)
## REML log-likelihood: -46.5969   AIC: 99.19   BIC: 105.90
##
## Variance parameters:
##                  estimate std.error z.ratio bound
## gen!sigma2        0.14290   0.05208    2.74     P
## rep:block!sigma2  0.07022   0.03734    1.88     P
## R!units!sigma2    0.08162   0.02029    4.02     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
pr <- predict(fit_r, classify = "gen")
head(pr$pvals, 4)
```

```
##   gen predicted.value std.error
## 1 G01           4.981    0.1766
## 2 G02           4.484    0.1766
## 3 G03           3.695    0.1766
## 4 G04           4.486    0.1766
```

Broad-sense heritability on a variety-mean basis can be computed with the classical formula for $r = 3$ replicates or with the generalised heritability of Cullis et al. (2006), which uses the average SED of the BLUPs and is appropriate for unbalanced designs:

```
vpredict(fit_r, H2_classical ~ V1 / (V1 + V3 / 3))
```

```
##              estimate std.error
## H2_classical   0.8401   0.06254
```

```
sg2 <- fit_r$theta[1]
H2_cullis <- 1 - pr$avsed^2 / (2 * sg2)
H2_cullis
```

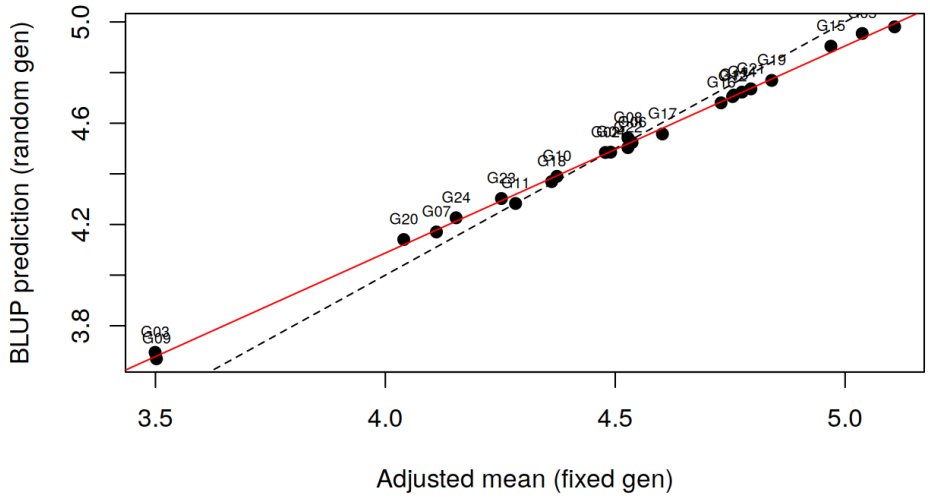
```
## gen!sigma2
##     0.8092
```

4.4 Comparison of adjusted means and BLUPs

The next figure compares the adjusted means (fixed varieties) with the BLUPs (random varieties). The slope smaller than one is the BLUP shrinkage.

```
plot(pf$pvals$predicted.value, pr$pvals$predicted.value, pch = 19,
     xlab = "Adjusted mean (fixed gen)", ylab = "BLUP prediction (random gen)")
abline(0, 1, lty = 2)
abline(lm(pr$pvals$predicted.value ~ pf$pvals$predicted.value), col = "red")
```

```
text(pf$pvals$predicted.value, pr$pvals$predicted.value, pf$pvals$gen, pos = 3,
cex = 0.6)
```



Adjusted means (fixed varieties) against BLUP predictions (random varieties) in the oat trial.

The ranking of the varieties is almost identical, but the BLUPs are closer to the mean. In unbalanced designs the BLUP can also change the ranking, because varieties with less information are shrunk more.

4.5 Is the incomplete-block design worthwhile?

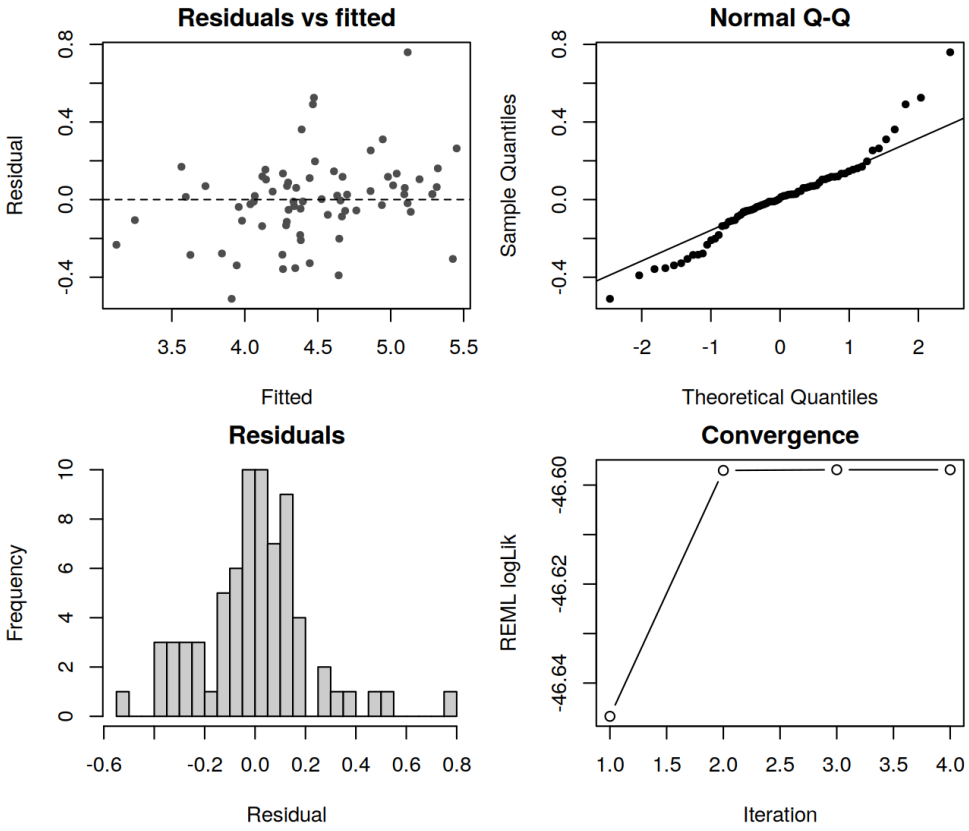
To find out whether the incomplete blocks capture variation, the model with and without `rep:block` is compared by a likelihood ratio test. Both models have the same fixed effects, so their REML likelihoods are comparable, and since a variance on the boundary is being tested the 50:50 mixture is used.

```
fit_r0 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
lrt(fit_r0, fit_r)

##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        8.606 0.001675 105.8 99.19 110.3 105.9
```

4.6 Diagnostics

```
plot(fit_r)
```



Diagnostic plots of the model with random varieties.

The conditional residuals show no trend with the fitted values and no important departures from normality. The bottom-right panel shows the convergence of the AI algorithm, which needs few iterations in this simple model.

4.7 Verification with lme4

```
library(lme4)
l <- lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)
data.frame(MLREML = unname(fit_r$theta),
           lme4 = as.data.frame(VarCorr(l))$vcov,
           row.names = c("gen", "rep:block", "residual"))
```

```
##              MLREML    lme4
## gen         0.14290 0.14290
## rep:block 0.07022 0.07022
## residual   0.08162 0.08162
```

```
c(MLREML = fit_r$loglik, lme4 = as.numeric(logLik(l)))
```

```
## MLREML   lme4
##  -46.6  -46.6
```

The two programs agree to six decimal places in the likelihood.

5 Example 2: spatial analysis of a wheat trial

5.1 The data

The data `gilmour.serpentine` come from the paper by Gilmour, Cullis and Verbyla (1997), which established spatial analysis with separable autoregressive structures as standard practice in Australian crop trials. They are 108 wheat varieties in a randomised complete block design with 3 replicates, on a grid of 22 rows by 15 columns (330 plots). The response is yield.

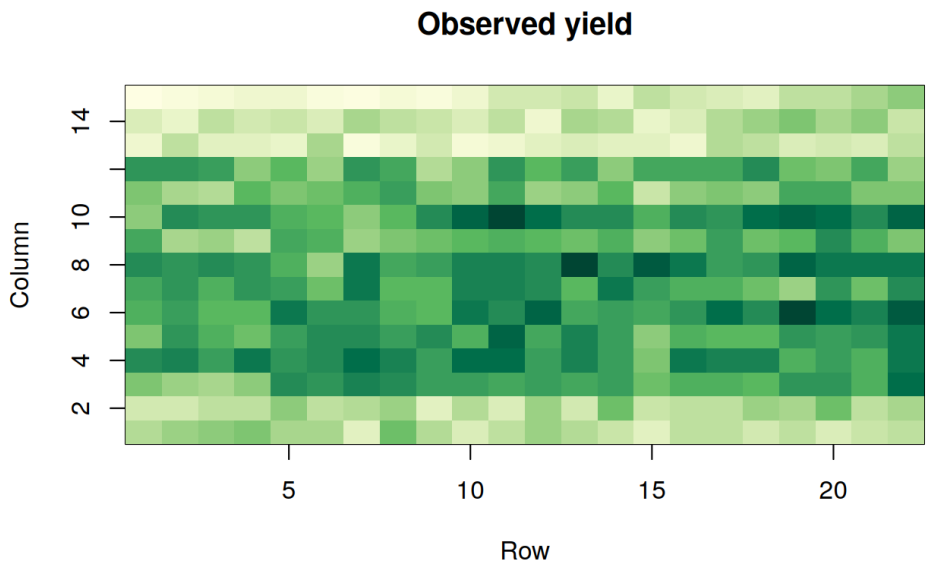
```
data(gilmour.serpentine)
d <- gilmour.serpentine
str(d)
```

```
## 'data.frame':    330 obs. of  5 variables:
##  $ col  : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ row  : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep  : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ gen  : Factor w/ 107 levels "(WWH*MM)*WR*",..: 4 10 15 17 21 32 33 34 72
74 ...
##  $ yield: int  483 526 557 564 498 510 344 600 466 370 ...
```

```
c(rows = length(unique(d$row)), columns = length(unique(d$col)))
```

```
##    rows columns
##      22      15
```

```
Ym <- tapply(d$yield, list(d$row, d$col), mean)
image(1:22, 1:15, Ym, col = hcl.colors(30, "YlGn", rev = TRUE),
      xlab = "Row", ylab = "Column", main = "Observed yield")
```



Observed yield on the trial grid (rows × columns). Darker colours indicate higher yield.

The figure above already suggests a fertility gradient along the rows and local variation that the complete replicates cannot absorb.

5.2 Complete-block model

```
m1 <- mlreml(yield ~ rep, random = ~ gen, data = d, trace = FALSE)
summary(m1)$varcomp
```

```
##                 estimate std.error z.ratio bound
## gen!sigma2          1934     947.3   2.042     P
## R!units!sigma2     13329    1263.6  10.548     P
```

5.3 AR1 × AR1 spatial model

Following Gilmour et al. (1997), the residual is modelled as a separable autoregressive process in rows and columns plus independent error (*nugget*), which in MLREML is the random term `units`:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}_g\mathbf{g} + \boldsymbol{\eta} + \boldsymbol{\xi}, \qquad \boldsymbol{\xi} \sim N\left(\mathbf{0}, \sigma^2_{\xi}\, \boldsymbol{\Sigma}_r(\rho_r) \otimes \boldsymbol{\Sigma}_c(\rho_c)\right), \qquad \boldsymbol{\eta} \sim N\left(\mathbf{0}, \sigma^2_{\eta}\mathbf{I}\right).$$

```
m2 <- mlreml(yield ~ rep, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m2)
```

```
## MLREML fit (AI-REML)
## Observations: 330   Fixed-effect columns: 3   Iterations: 16 (converged)
## REML log-likelihood: -1853.6433   AIC: 3717.29   BIC: 3736.24
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                   2.583e+03 4.764e+02    5.42     P
## units!sigma2                 1.702e+03 2.356e+02    7.22     P
## R!ar1(row):ar1(col)!sigma2   3.954e+04 2.093e+04    1.89     P
## R!ar1(row):ar1(col)!row_cor 9.903e-01 5.309e-03  186.55     P
## R!ar1(row):ar1(col)!col_cor 8.202e-01 6.620e-02   12.39     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

Both models have the same fixed effects, so they can be compared with AIC and BIC:

```
data.frame(model = c("complete blocks", "AR1 x AR1 + nugget"),
           logLik = c(m1$loglik, m2$loglik), AIC = c(m1$AIC, m2$AIC), BIC =
c(m1$BIC, m2$BIC))
```

```
##                model logLik  AIC  BIC
## 1    complete blocks  -2043 4091 4099
## 2 AR1 x AR1 + nugget  -1854 3717 3736
```

The correlation between neighbouring rows is very high, indicating a smooth trend along the rows, while the correlation between columns is lower. The improvement in AIC is large.

5.4 Global trend

Gilmour et al. (1997) recommend adding a linear trend as a fixed effect when the variogram shows a semi-variance that keeps increasing. The numeric covariate `row` is included in the fixed part and tested with Wald:

```
m3 <- mlreml(yield ~ rep + row, random = ~ gen + units,
             residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
summary(m3)$varcomp
```

```
##                               estimate std.error z.ratio bound
## gen!sigma2                   2.581e+03 4.761e+02   5.421     P
## units!sigma2                 1.697e+03 2.351e+02   7.218     P
## R!ar1(row):ar1(col)!sigma2   4.194e+04 2.267e+04   1.850     P
## R!ar1(row):ar1(col)!row_cor 9.905e-01 5.225e-03 189.573     P
## R!ar1(row):ar1(col)!col_cor 8.303e-01 6.455e-02  12.863     P
```

```
wald(m3)
```

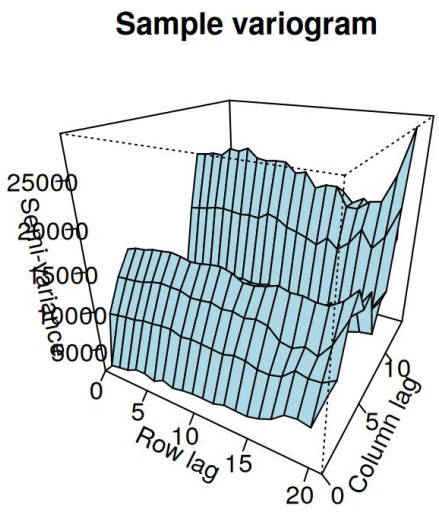
```
##          term df wald.inc   F.inc ddf.inc   p.inc wald.con   F.con ddf.con
p.con
## 1 (Intercept)  1  16.7395 20.0048   3.443 0.01547  16.7395 20.0048   3.443
0.01547
## 2         rep  2   1.8132  0.7871  11.472 0.47829   1.8166  0.7886  11.472
0.47766
## 3         row  1   0.6921  0.5961   9.759 0.45838   0.6921  0.5961   9.759
0.45838
##     p.chisq
## 1 4.288e-05
## 2 4.032e-01
## 3 4.055e-01
```

In this trial the linear trend is not significant (p = 0.46 with the Kenward–Roger method, which gives only 9.8 denominator degrees of freedom for this test): the autoregression between rows, with a correlation close to one, already represents the gradient. Models `m2` and `m3` have different fixed effects, so their REML likelihoods are not comparable and the decision rests on the Wald test. We keep the linear term below only to illustrate its use; the results with `m2` are practically the same.

5.5 Sample variogram

The variogram of the spatial residuals is the most useful diagnostic tool. For the complete-block model it is computed from the conditional residuals; a variogram that increases with distance indicates unmodelled spatial correlation.

```
v1 <- variogram(m1, "row", "col")
```



Sample variogram of the residuals of the complete-block model.

```
head(v1[order(v1$row.lag, v1$col.lag), ], 6)
```

```
##     row.lag col.lag gamma npairs
## 22        0       1  8657    308
## 44        0       2 12548    286
## 66        0       3 15256    264
## 88        0       4 10720    242
## 110       0       5 11402    220
## 132       0       6  9477    198
```

The semi-variance of the complete-block model increases with the row and column lags and does not level off within the trial; that is the spatial correlation the AR1 × AR1 model represents.

5.6 Gain in precision

The practical consequence of a better model is a more precise comparison of the varieties. To compare precisions, the model with fixed varieties is fitted and the average SED computed:

```
m1f <- mlreml(yield ~ rep + gen, data = d, trace = FALSE)
m3f <- mlreml(yield ~ rep + row + gen, random = ~ units,
              residual = ~ ar1(row):ar1(col), data = d, trace = FALSE)
c(SED_blocks = predict(m1f, "gen")$avsed, SED_spatial = predict(m3f,
"gen")$avsed)
```

```
##  SED_blocks SED_spatial
##       93.66       38.81
```

The spatial analysis reduces the error of the comparisons, which is equivalent to evaluating the varieties with more replicates without increasing the cost of the trial. The generalised heritability increases accordingly:

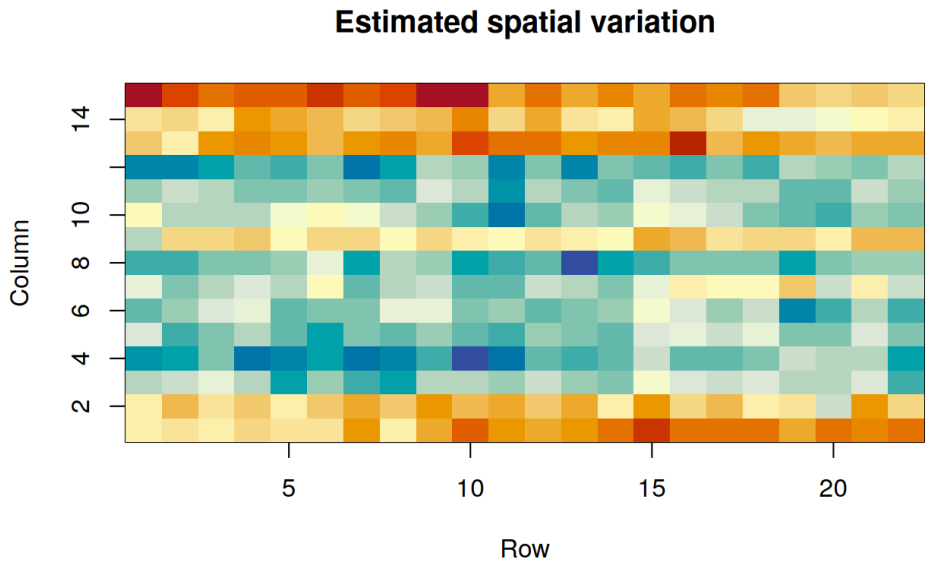
```
H2 <- function(fit) 1 - predict(fit, "gen")$avsed^2 / (2 * fit$theta[1])
c(blocks = H2(m1), spatial = H2(m3))
```

```
##  blocks.gen!sigma2 spatial.gen!sigma2
##             0.3080             0.7698
```

5.7 Estimated spatial trend

The spatial component predicted by the model is the difference between the data and the fixed part plus the genetic effects. The next figure shows it on the grid.

```
gb <- coef(m3, "random", "gen")
geff <- gb$blup[match(as.character(m3$data$gen), gb$level)]
spatial <- m3$y - m3$fixed_fitted - geff
Em <- tapply(spatial, list(m3$data$row, m3$data$col), mean)
image(1:22, 1:15, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Row", ylab =
"Column",
      main = "Estimated spatial variation")
```



Residuals of the model with genetic and fixed effects only (estimate of the spatial component).

5.8 Irregularly spaced data

When the units are not on a regular grid (trees in a stand, GPS-referenced plots, missing plants), the structure `sexp(x, y)` defines an isotropic exponential correlation $\rho^{d_{ij}}$ with the actual Euclidean distance. As an example we use the volume data of 1,600 Douglas firs (Johnson, 1998, in `agridat`), from which a random sample of 450 trees is taken to obtain an irregular spatial pattern.

```
data(johnson.douglasfir)
set.seed(11)
df <- johnson.douglasfir[sample(nrow(johnson.douglasfir), 450), ]
df$lv <- log(df$volume + 1)
f_iid <- mlreml(lv ~ 1, data = df, trace = FALSE)
f_exp <- mlreml(lv ~ 1, random = ~ units, residual = ~ sexp(row, col), data =
df, trace = FALSE)
summary(f_exp)$varcomp
```

```
##                                 estimate std.error z.ratio bound
## units!sigma2                      5.8841    2.4044  2.4473     P
```

```
## R!sexp(row, col)!sigma2              0.8822    2.3803  0.3706     P
## R!sexp(row, col)!sexp(row,col)_cor   0.3581    0.6771  0.5289     P
```

```
lrt(f_iid, f_exp, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        1.031  0.5971 2141 2144 2145 2156
```

The correlation per unit of distance and the improvement in likelihood indicate that nearby trees have similar volumes, because of competition and microsite. The correlation between two trees at distance d is $\hat{\rho}^d$; for example, at distances of 1, 3 and 5 positions:

```
rho <- f_exp$theta[grep("_cor$", names(f_exp$theta))]
unname(rho^c(1, 3, 5))
```

```
## [1] 0.358088 0.045917 0.005888
```

6 Example 3: multi-environment trials and factor analytic models

6.1 The data

The data set `besag.met` (Besag and Higdon, 1999; in `agridat`) contains the yield of 64 maize varieties evaluated in 6 locations (*counties*), each with 3 replicates divided into 8 incomplete blocks of 8 plots: 1,152 plots in total. It is a typical multi-environment trial (MET) of a breeding programme.

```
data(besag.met)
d <- besag.met
str(d)
```

```
## 'data.frame':    1188 obs. of  7 variables:
##  $ county: Factor w/ 6 levels "C1","C2","C3",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ row   : int  1 1 1 1 1 1 1 1 1 1 ...
##  $ col   : int  1 2 3 4 5 6 7 8 9 10 ...
##  $ rep   : Factor w/ 3 levels "R1","R2","R3": 1 1 1 1 1 1 1 1 1 1 ...
##  $ block : Factor w/ 8 levels "B1","B2","B3",..: 2 2 2 2 2 2 2 2 5 5 ...
##  $ yield : num  170 130 129 131 132 ...
##  $ gen   : Factor w/ 64 levels "G01","G02","G03",..: 42 58 26 10 34 50 2 18
45 61 ...
```

```
round(tapply(d$yield, d$county, mean), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

```
round(tapply(d$yield, d$county, sd), 1)
```

```
## C1 C2 C3 C4 C5 C6
## NA NA NA NA NA NA
```

Means and standard deviations differ widely among locations, anticipating heterogeneous genetic and residual variances.

6.2 The sequence of models

The reference MET analysis (Smith, Cullis and Thompson, 2001; 2005) models the genetic effects in each environment, $\mathbf{u} = (\mathbf{u}_1', \ldots, \mathbf{u}_E')'$, with

$$\mathrm{Var}(\mathbf{u}) = \boldsymbol{\Sigma}_E \otimes \mathbf{I}_N,$$

and compares structures for the $E \times E$ matrix of genetic covariances among environments $\boldsymbol{\Sigma}_E$. All structures share the fixed effects (location means), the design (replicates and blocks within location) and a residual variance specific to each location.

Structures for the genetic covariance among environments.

Model	$\boldsymbol{\Sigma}_E$	Genetic parameters
Diagonal	$\mathrm{diag}(\sigma^2_{g1}, \ldots, \sigma^2_{gE})$	E
FA(1)	$\boldsymbol{\lambda}\boldsymbol{\lambda}' + \boldsymbol{\Psi}$	$2E$
FA(2)	$\boldsymbol{\Lambda}\boldsymbol{\Lambda}' + \boldsymbol{\Psi}$	$3E - 1$
Unstructured	full	$E(E+1)/2$

The diagonal model assumes no genetic correlation among environments, which in practice is never true. The unstructured model is the most general, but with many environments it has too many parameters and is often hard to estimate. Factor analytic models are the middle ground: with few factors they describe most of the genetic covariance and make the genotype × environment interaction interpretable.

```
f_diag <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
                 residual = ~ diag(county):units, data = d, trace = FALSE)
f_fa1 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
f_fa2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 2):id(gen),
                residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 200)
f_us <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
us(county):id(gen),
               residual = ~ diag(county):units, data = d, trace = FALSE, maxit
= 100)
cmp <- data.frame(model = c("diag", "FA(1)", "FA(2)", "US"),
                  parameters = sapply(list(f_diag, f_fa1, f_fa2, f_us),
function(f) sum(f$free)),
                  logLik = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
"loglik"),
                  AIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "AIC"),
                  BIC = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`, "BIC"),
                  converged = sapply(list(f_diag, f_fa1, f_fa2, f_us), `[[`,
"converged"))
cmp
```

```
##   model parameters logLik  AIC  BIC converged
## 1  diag         14  -4849 9726 9796      TRUE
## 2 FA(1)         20  -4814 9668 9769      TRUE
## 3 FA(2)         25  -4812 9675 9801      TRUE
## 4    US         29  -4812 9681 9827      TRUE
```

The diagonal model is clearly the worst: ignoring the genetic correlation among locations costs 34.9 log-likelihood units relative to FA(1), with only 6 additional parameters. According to AIC the best model is FA(1). The likelihood increases from FA(1) to FA(2) and to the

unstructured model, as it must because the models are nested, but the increases are small for the number of parameters added. The likelihood is very flat in the direction of the second factor, a common situation in MET analysis that also occurs in ASReml. Several specific variances of the FA(2) model are on the boundary; thanks to the active-set treatment of the boundary (Section 2.4.1), they do not stall the joint AI step and the model converges. The log-likelihood of FA(2) lies between those of FA(1) and the unstructured model and does not change the choice of model.

The LRT between FA(1) and the unstructured model (both with the same fixed effects) is:

```
lrt(f_fa1, f_us, boundary = FALSE)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  9        4.712  0.8587 9668 9681 9769 9827
```

6.3 Interpreting the FA(1) model

`vc_matrix()` returns the estimated genetic covariance matrix, its correlation matrix, the loadings, the specific variances and the percentage of the genetic variance of each environment explained by the factors.

```
vfa <- vc_matrix(f_fa1, "fa(county, 1):id(gen)")
round(vfa$cor, 2)
```

```
##      C1   C2   C3   C4   C5   C6
## C1 1.00 0.86 0.68 0.86 0.86 0.40
## C2 0.86 1.00 0.79 1.00 1.00 0.46
## C3 0.68 0.79 1.00 0.79 0.79 0.36
## C4 0.86 1.00 0.79 1.00 1.00 0.46
## C5 0.86 1.00 0.79 1.00 1.00 0.46
## C6 0.40 0.46 0.36 0.46 0.46 1.00
```

```
data.frame(loading = round(vfa$loadings[, 1], 2), psi = signif(vfa$psi, 3),
           pct_explained = round(vfa$pct_variance, 1))
```

```
##    loading      psi pct_explained
## C1    6.49 1.46e+01          74.3
## C2    5.15 8.44e-07         100.0
## C3    7.57 3.47e+01          62.3
## C4    5.34 7.30e-07         100.0
## C5    8.90 6.70e-07         100.0
## C6    2.35 2.05e+01          21.3
```

```
summary(f_fa1)$varcomp[grepl("fa1|var_", rownames(summary(f_fa1)$varcomp)), ]
```

```
##                                        estimate std.error   z.ratio bound
## fa(county, 1):id(gen)!county_fa1_C1 6.491e+00     1.366 4.752e+00     P
## fa(county, 1):id(gen)!county_fa1_C2 5.149e+00     1.319 3.904e+00     P
## fa(county, 1):id(gen)!county_fa1_C3 7.569e+00     1.567 4.830e+00     P
## fa(county, 1):id(gen)!county_fa1_C4 5.345e+00     1.419 3.765e+00     P
## fa(county, 1):id(gen)!county_fa1_C5 8.898e+00     1.392 6.391e+00     P
## fa(county, 1):id(gen)!county_fa1_C6 2.355e+00     1.710 1.377e+00     P
## fa(county, 1):id(gen)!county_var_C1 1.458e+01    16.469 8.855e-01     P
## fa(county, 1):id(gen)!county_var_C2 8.438e-07    16.874 5.001e-08     B
## fa(county, 1):id(gen)!county_var_C3 3.472e+01    21.181 1.639e+00     P
## fa(county, 1):id(gen)!county_var_C4 7.300e-07    19.599 3.725e-08     B
```

```
## fa(county, 1):id(gen)!county_var_C5 6.700e-07    16.851 3.976e-08     B
## fa(county, 1):id(gen)!county_var_C6 2.053e+01    29.724 6.907e-01     P
```

Several specific variances ψ are on the boundary (code `B`). In those locations the common factor explains all the genetic variance, and the estimated genetic correlation among them is one. This result is frequent in trials with few replicates and does not indicate a fitting error; it indicates that the data cannot distinguish the specific variance from zero. Standard errors of parameters on the boundary are not valid.

```
vus <- vc_matrix(f_us, "us(county):id(gen)")
op <- par(mfrow = c(1, 2), mar = c(3, 3, 2, 1))
for (M in list(US = vus$cor, FA1 = vfa$cor)) {
  image(1:6, 1:6, M[, 6:1], zlim = c(-1, 1), col = hcl.colors(21, "RdBu"),
        axes = FALSE, xlab = "", ylab = "")
  axis(1, 1:6, rownames(M), cex.axis = 0.7); axis(2, 1:6, rev(rownames(M)),
cex.axis = 0.7, las = 1)
  text(rep(1:6, 6), rep(6:1, each = 6), sprintf("%.2f", M), cex = 0.6)
}
```



Genetic correlations among locations estimated with the unstructured model (left) and with FA(1) (right).

```
par(op)
```

6.4 Predictions by environment and overall performance

The predictions of each variety in each location combine the fixed location mean with the BLUP of the variety in that location. The classification `county:gen` includes the factor analytic term because both of its factors are in the classification:

```
pr <- predict(f_fa1, classify = "county:gen", sed = FALSE)
P <- with(pr$pvals, tapply(predicted.value, list(gen, county), mean))
round(head(P[order(-rowMeans(P)), ], 10), 1)
```

```
##        C1   C2    C3    C4    C5   C6
## G05 160.6 64.8 100.0 153.7 130.6 92.7
## G54 159.5 64.8 101.6 153.7 130.5 91.3
## G45 157.4 63.1 100.1 151.9 127.5 89.8
## G55 158.7 62.6 101.1 151.4 126.8 88.8
## G61 155.7 61.4 101.0 150.1 124.6 91.6
## G32 157.1 61.8 100.2 150.6 125.4 88.5
```

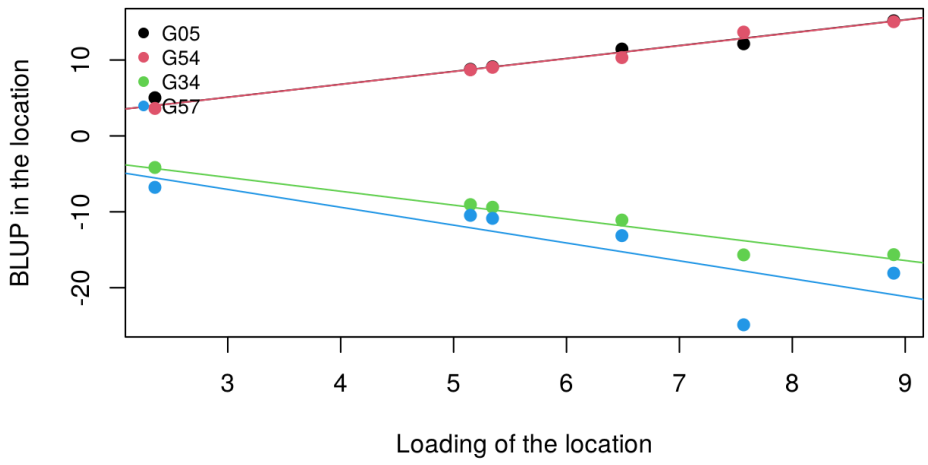
```
## G51 157.3 62.0  96.1 150.8 125.7 91.2
## G30 155.4 60.9  98.0 149.7 123.8 89.1
## G22 155.0 61.2  94.0 149.9 124.2 91.0
## G52 155.8 60.6  96.1 149.4 123.3 89.8
```

A natural measure of overall performance in an FA(1) model with positive loadings is the factor score of each variety multiplied by the mean loading (Smith and Cullis, 2018). Since $\mathbf{u}_i = \boldsymbol{\lambda} f_i + \boldsymbol{\delta}_i$, the score can be estimated by least squares from the BLUPs of each variety:

```
U <- matrix(coef(f_fa1, "random", "fa(county, 1):id(gen)")$blup, ncol = 6,
            dimnames = list(levels(d$gen), levels(d$county)))
lam <- vfa$loadings[, 1]
score <- drop(U %*% lam / sum(lam^2))
OP <- mean(lam) * score
head(sort(OP, decreasing = TRUE), 10)

##    G05    G54    G55    G45    G32    G61    G51    G30    G09    G52
## 10.149 10.131  8.381  8.330  7.407  7.220  6.954  6.159  5.860  5.709

top <- names(sort(OP, decreasing = TRUE))[c(1, 2, 63, 64)]
plot(range(lam), range(U[top, ]), type = "n", xlab = "Loading of the location",
     ylab = "BLUP in the location")
for (k in seq_along(top)) {
  points(lam, U[top[k], ], pch = 19, col = k)
  abline(0, score[top[k]], col = k)
}
legend("topleft", top, col = seq_along(top), pch = 19, bty = "n", cex = 0.8)
```



Latent regression plot: BLUPs of four varieties in each location against the loading of the location. The lines have slope equal to the score of each variety.

Varieties with a large positive slope are superior in locations with high loadings; the scatter of the points around the line is the part of the interaction not explained by the common factor.

6.5 Recommendations for MET analysis

1. Start with heterogeneous residual and genetic variances (`diag`) to check that the base model fits.

2. Fit FA(1) and increase the number of factors while AIC or the LRT justify it and the percentage of genetic variance explained is low in some environments.
3. Use `us` only with few environments; with more than 8–10 environments it often fails to converge or converges to singular matrices.
4. Check parameters on the boundary and increase `maxit` when the algorithm does not converge; an FA model that has not converged in the likelihood may still have stable predictions, which can be checked by comparing the BLUPs between iterations.

7 Example 4: genomic prediction (GBLUP) with 599 wheat lines

7.1 The data

The `wheat` data set of the `BGLR` package (Crossa et al., 2010; Pérez and de los Campos, 2014) contains 599 CIMMYT wheat lines genotyped with 1,279 DArT markers (coded 0/1), their pedigree relationship matrix and grain yield evaluated in four environments (E1 to E4). It is one of the most widely used data sets to compare genomic selection methods.

```
library(BGLR)
data(wheat)
ids <- paste0("L", 1:599)
M <- wheat.X; rownames(M) <- ids
A <- wheat.A; dimnames(A) <- list(ids, ids)
Y <- wheat.Y; rownames(Y) <- ids
dim(M); table(M)

## [1]  599 1279

## M
##      0      1
## 336588 429533

round(cor(Y), 2)

##       1     2     4     5
## 1  1.00 -0.02 -0.19 -0.12
## 2 -0.02  1.00  0.66  0.41
## 4 -0.19  0.66  1.00  0.39
## 5 -0.12  0.41  0.39  1.00
```

7.2 Quality control and genomic matrix

DArT markers are dominant and coded 0/1. To build **G** we recode them as 0/2 (the lines are inbred, so every locus is homozygous) and apply quality control:

```
qc <- snp_qc(2 * M, maf = 0.05, callrate = 0.9)
qc$report

##   markers_in removed_callrate removed_maf markers_out individuals_in
individuals_out
## 1       1279                0          96        1183            599
599

G <- Gmatrix(qc$M, method = "VanRaden")
summary(diag(G))
```

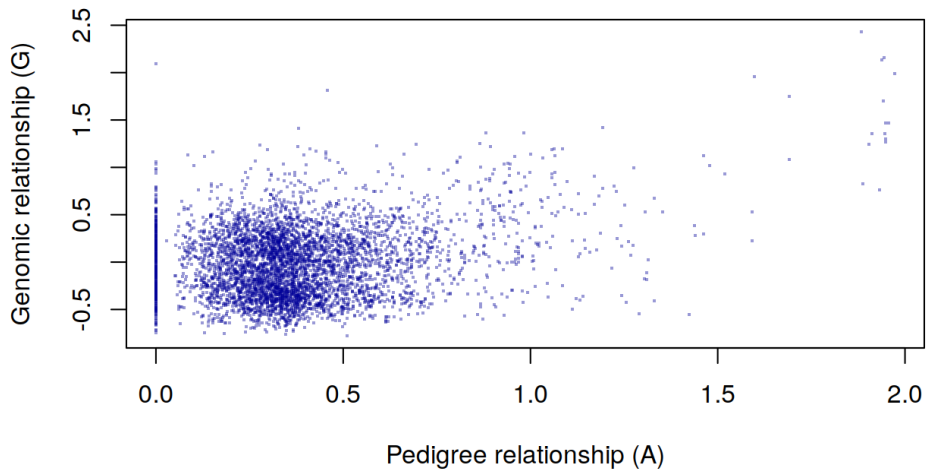
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.34    1.77    2.00    2.00    2.23    2.88

summary(diag(A))

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    1.56    1.98    1.99    1.98    2.00    2.00
```

The diagonal of **A** is close to 2, as expected for inbred lines ($1 + F$ with $F \approx 1$). The diagonal of **G** also has mean 2, but it varies much more among lines (from 1.3 to 2.9): it measures realised inbreeding relative to the genotyped population, whereas the pedigree only gives its expected value. Differences in scale between matrices do not affect predictions, because they are absorbed by the genetic variance, but they do affect the interpretation of σ_g^2. The relationship between both matrices is shown below:

```
up <- upper.tri(A)
set.seed(1); s <- sample(which(up), 5000)
plot(A[s], G[s], pch = ".", cex = 2, col = rgb(0, 0, 0.6, 0.4),
     xlab = "Pedigree relationship (A)", ylab = "Genomic relationship (G)")
```



Pedigree relationships against genomic relationships (off-diagonal elements).

```
cor(A[up], G[up])

## [1] 0.2469
```

The moderate correlation reflects that the pedigree predicts the expected relationship, whereas the markers measure the realised relationship, which varies around the expected one because of Mendelian segregation.

7.3 GBLUP in one environment

The GBLUP model for environment E1 is $\mathbf{y} = \mathbf{1}\mu + \mathbf{g} + \mathbf{e}$, $\mathbf{g} \sim N(\mathbf{0}, \sigma_g^2 \mathbf{G})$. It is fitted with the general engine and with the function `gblup()`, which uses the spectral decomposition of **G** and reduces REML to a one-dimensional optimisation.

```
G <- G + diag(1e-4, nrow(G))      # small numerical regularisation
d1 <- data.frame(gen = ids, y = Y[, 1])
t_general <- system.time(f1 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1,
```

```
trace = FALSE))
y1 <- setNames(Y[, 1], ids)
t_spectral <- system.time(g1 <- gblup(y1, G))
summary(f1)$varcomp
```

```
##                   estimate std.error z.ratio bound
## vm(gen, G)!sigma2   0.2942   0.05462   5.387     P
## R!units!sigma2      0.5551   0.04607  12.049     P
```

```
g1$varcomp
```

```
## sigma2_g sigma2_e       h2
##   0.2942   0.5551   0.3464
```

```
c(general = t_general[3], spectral = t_spectral[3])
```

```
##  general.elapsed spectral.elapsed
##            0.209            0.061
```

```
c(logLik_general = f1$loglik, logLik_spectral = g1$loglik)
```

```
##  logLik_general logLik_spectral
##          -794.5          -794.5
```

```
cor(f1$blups[[1]]$blup, g1$gebv[f1$blups[[1]]$level])
```

```
## [1] 1
```

Both implementations give the same likelihood and the same GEBVs. The estimated genomic heritability is 0.35. The reliability of the GEBVs of the phenotyped lines is obtained directly:

```
summary(f1$blups[[1]]$reliability)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   0.603   0.717   0.754   0.763   0.809   0.964
```

7.4 Predicting unphenotyped lines

The purpose of genomic selection is to predict candidates without phenotypes. If the phenotypes of 120 lines are removed, `vm()` still predicts them because they are in **G**; it is enough to drop their records from the data:

```
set.seed(2024)
test <- sample(ids, 120)
f_tr <- mlreml(y ~ 1, random = ~ vm(gen, G), data = subset(d1, !gen %in% test),
trace = FALSE)
b <- f_tr$blups[[1]]
pred <- b$blup[match(test, b$level)]
c(correlation = cor(pred, Y[test, 1]),
  mean_reliability_test = mean(b$reliability[match(test, b$level)]))
```

```
##           correlation mean_reliability_test
##                0.5457                0.6374
```

7.5 Cross-validation: pedigree, markers and both

The function `cv_gblup()` repeats random k-fold partitions. We compare the predictive ability of the pedigree (**A**), of the markers (**G**) and of a model with both matrices, written in MLREML as two random terms on the same factor.

```
Ar <- A + diag(1e-4, nrow(A))
cvA <- cv_gblup(y1, Ar, folds = 5, reps = 4, seed = 3)
cvG <- cv_gblup(y1, G, folds = 5, reps = 4, seed = 3)
data.frame(matrix = c("A (pedigree)", "G (markers)"),
           mean_r = c(mean(cvA$cor), mean(cvG$cor)),
           sd_r = c(sd(cvA$cor), sd(cvG$cor)))
```

```
##         matrix mean_r   sd_r
## 1 A (pedigree) 0.4157 0.0615
## 2  G (markers) 0.4909 0.0551
```

```
f_AG <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = d1, trace =
FALSE)
summary(f_AG)$varcomp
```

```
##                     estimate std.error z.ratio bound
## vm(gen, Ar)!sigma2   0.1127   0.04147   2.717     P
## vm(gen, G)!sigma2    0.2351   0.05213   4.510     P
## R!units!sigma2       0.4519   0.05102   8.856     P
```

With the same partition for the two-kernel model:

```
set.seed(3); fold <- sample(rep(1:5, length.out = 599))
r_AG <- sapply(1:5, function(k) {
  tr <- d1[fold != k, ]
  f <- mlreml(y ~ 1, random = ~ vm(gen, Ar) + vm(gen, G), data = tr, trace =
FALSE, pev = FALSE)
  te <- ids[fold == k]
  p <- f$blups[[1]]$blup[match(te, f$blups[[1]]$level)] +
f$blups[[2]]$blup[match(te, f$blups[[2]]$level)]
  cor(p, Y[te, 1])
})
mean(r_AG)
```

```
## [1] 0.5161
```

7.6 Multi-environment GBLUP

With the four environments we fit a genomic MET model in which the genetic effect of each line in each environment has covariance $\boldsymbol{\Sigma}_E \otimes \mathbf{G}$ (Burgueño et al., 2012), with environment-specific residual variances:

```
dl <- data.frame(gen = rep(ids, 4), env = factor(rep(paste0("E", 1:4), each =
599)), y = as.vector(Y))
t_met <- system.time(
  fm <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
               data = dl, trace = FALSE, maxit = 100))
t_met[3]
```

```
## elapsed
##   65.96
```

```
summary(fm)
```

```
## MLREML fit (AI-REML)
## Observations: 2396   Fixed-effect columns: 4   Iterations: 15 (converged)
## REML log-likelihood: -3054.2139   AIC: 6136.43   BIC: 6217.35
```

```
##
## Variance parameters:
##                                 estimate std.error z.ratio bound
## us(env):vm(gen, G)!env_E1:E1  0.29972   0.05433    5.52     P
## us(env):vm(gen, G)!env_E2:E1 -0.07022   0.04270   -1.64     P
## us(env):vm(gen, G)!env_E2:E2  0.57991   0.07460    7.77     P
## us(env):vm(gen, G)!env_E3:E1 -0.10855   0.04109   -2.64     P
## us(env):vm(gen, G)!env_E3:E2  0.52893   0.06206    8.52     P
## us(env):vm(gen, G)!env_E3:E3  0.49306   0.06739    7.32     P
## us(env):vm(gen, G)!env_E4:E1 -0.12528   0.03693   -3.39     P
## us(env):vm(gen, G)!env_E4:E2  0.34978   0.04934    7.09     P
## us(env):vm(gen, G)!env_E4:E3  0.35240   0.04828    7.30     P
## us(env):vm(gen, G)!env_E4:E4  0.34639   0.05632    6.15     P
## R!diag(env):units!env_E1      0.55463   0.04568   12.14     P
## R!diag(env):units!env_E2      0.41498   0.03492   11.88     P
## R!diag(env):units!env_E3      0.50242   0.03863   13.01     P
## R!diag(env):units!env_E4      0.56067   0.04425   12.67     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
round(vc_matrix(fm, "us(env):vm(gen, G)")$cor, 2)
```

```
##       E1    E2    E3    E4
## E1  1.00 -0.17 -0.28 -0.39
## E2 -0.17  1.00  0.99  0.78
## E3 -0.28  0.99  1.00  0.85
## E4 -0.39  0.78  0.85  1.00
```

The genetic correlations between environments are more extreme than the phenotypic ones: E2, E3 and E4 are strongly correlated with one another, whereas E1 has negative genetic correlations with the others. This is the structure that the MET model exploits to predict a line in one environment from its data in the other environments (the CV2 scheme of Burgueño et al., 2012).

```
set.seed(5)
hidden <- sample(ids, 120)
dtr <- dl[!(dl$env == "E1" & dl$gen %in% hidden), ]
fm_tr <- mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~
diag(env):units,
                data = dtr, trace = FALSE, maxit = 100, pev = FALSE)
b <- fm_tr$blups[[1]]
p_met <- b$blup[match(paste0("E1:", hidden), b$level)]
y_e1 <- y1; y_e1[hidden] <- NA
p_uni <- gblup(y_e1, G)$gebv[hidden]
c(single_environment = cor(p_uni, Y[hidden, 1]), multi_environment = cor(p_met,
Y[hidden, 1]))
```

```
## single_environment  multi_environment
##             0.3651             0.3438
```

In this case the multi-environment model does not improve on the single-environment model (0.344 versus 0.365). In the CV2 scheme the MET model uses the records of the same lines in the other environments, but E1 has weak and negative genetic correlations with E2–E4, so those records carry little information about E1 and the cost of estimating more parameters is not compensated. When environments are positively correlated, the gain of the MET model in

CV2 is usually large (Burgueño et al., 2012); the result illustrates that the benefit depends on the structure of $\boldsymbol{\Sigma}_E$.

7.7 Practical notes

- `gblup()` is convenient for a single genetic effect with a homogeneous residual, and it is the basis of `cv_gblup()`; with 599 lines it takes a fraction of a second. With several thousand individuals its cost is dominated by the spectral decomposition (Chapter 15).
- `mlreml()` with `vm()` is needed for models with more terms, heterogeneous residuals, several environments or several traits.
- A matrix **G** with more individuals than markers is singular; MLREML does not invert it, but adding a small constant to the diagonal is advisable for numerical stability, as done above.

8 Example 5: animal model with pedigree in dairy cattle

8.1 The data

The `pedigreemm` package (Vazquez et al., 2010) includes `milk`, with 3,397 lactation records of 1,359 Holstein cows in 57 herds, and `pedCowsR`, the pedigree of those cows and their ancestors (6,547 animals). Each record contains 305-day milk, fat and protein yield and the lactation number (1 to 5).

```
library(pedigreemm)
data(milk)
milk$animal <- as.character(milk$id)
milk$milk_t <- milk$milk / 1000    # tonnes
milk$fat_h <- milk$fat / 100       # hundreds of kg
milk$lact <- factor(milk$lact)
table(table(milk$animal))

##
##   1   2   3   4   5
## 337 405 314 207  96
```

8.2 Preparing the pedigree

`pedigree_prepare()` recodes unknown parents, adds parents that do not appear as individuals as founders, detects errors and sorts the pedigree so that parents precede their offspring.

```
p <- pedCowsR
ped <- data.frame(id = p@label, sire = p@label[p@sire], dam = p@label[p@dam])
pp <- pedigree_prepare(ped)
head(pp)

##   id sire  dam sire_i dam_i gen
## 1  1 <NA> <NA>      0     0   0
## 2  2 <NA> <NA>      0     0   0
## 3  3 <NA> <NA>      0     0   0
## 4  4 <NA> <NA>      0     0   0
## 5  5 <NA> <NA>      0     0   0
## 6  6 <NA> <NA>      0     0   0

table(generation = pp$gen)
```

```
## generation
##    0    1    2    3    4    5    6    7
## 2420 1208  777  612  757  562  173   38
```

```
c(animals = nrow(pp), with_sire = sum(pp$sire_i > 0), with_dam = sum(pp$dam_i >
0))
```

```
##   animals with_sire  with_dam
##      6547      2645      3454
```

```
t_A <- system.time(A <- Amatrix(pp))
Fi <- inbreeding_coef(pp)
t_A[3]
```

```
## elapsed
##   0.622
```

```
summary(Fi[Fi > 0])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00049 0.00781 0.01562 0.03744 0.03125 0.25000
```

```
sum(Fi > 0)
```

```
## [1] 31
```

```
all.equal(unname(diag(A) - 1), unname(Fi))
```

```
## [1] TRUE
```

```
Ai <- Ainverse(pp)
c(dim = nrow(Ai), non_zeros = length(Ai@x), density = length(Ai@x) /
nrow(Ai)^2)
```

```
##       dim non_zeros   density
## 6.547e+03 1.460e+04 3.405e-04
```

The dense **A** matrix of 6,547 animals takes about 340 MB. Its inverse, built directly with Henderson's rules, is very sparse. The inbreeding coefficients computed with the Meuwissen and Luo algorithm agree with the diagonal of **A**.

8.3 Animal model for first lactation

The animal model (Henderson, 1984) for first-lactation milk yield is

$$y_{ij} = \mu + h_j + a_i + e_{ij}, \qquad \mathbf{a} \sim N(\mathbf{0}, \sigma_a^2 \mathbf{A}), \qquad \mathbf{h} \sim N(\mathbf{0}, \sigma_h^2 \mathbf{I}),$$

where h_j is the herd effect, here random because several herds have few cows.

```
m1 <- subset(milk, lact == 1)
t_m1 <- system.time(
  a1 <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A), data = m1,
trace = FALSE))
t_m1[3]
```

```
## elapsed
##   2.171
```

```
summary(a1)
```

```
## MLREML fit (AI-REML)
## Observations: 1314   Fixed-effect columns: 1   Iterations: 6 (converged)
## REML log-likelihood: -3605.4877   AIC: 7216.98   BIC: 7232.52
##
## Variance parameters:
##                       estimate std.error z.ratio bound
## id(herd)!sigma2         5.6462    1.3650    4.14     P
## vm(animal, A)!sigma2    0.6750    0.8654    0.78     P
## R!units!sigma2         12.3830    0.9616   12.88     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
vpredict(a1, h2 ~ V2 / (V1 + V2 + V3))
```

```
##    estimate std.error
## h2  0.03609   0.04629
```

The estimated heritability is low and its standard error large, because each cow has a single record and the pedigree relates few phenotyped cows to one another. Breeding values (EBVs) are predicted for all 6,547 animals in the pedigree, including bulls and ancestors without records:

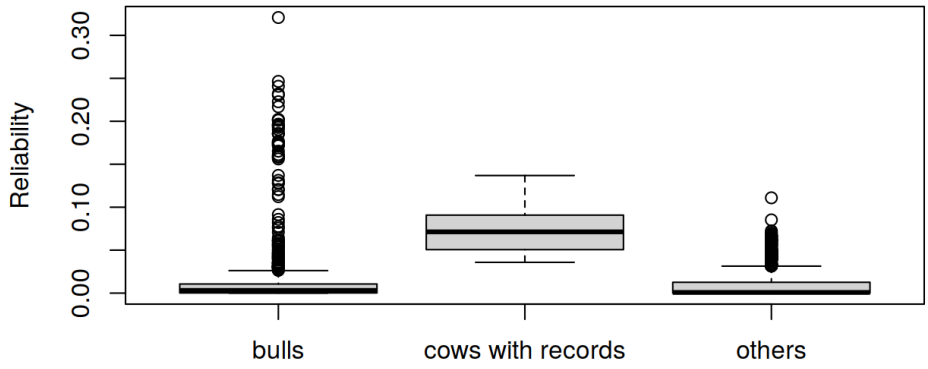
```
ebv <- coef(a1, "random", "vm(animal, A)")
nrow(ebv)
```

```
## [1] 6547
```

```
bulls <- unique(pp$sire[pp$sire_i > 0])
ebv_bulls <- ebv[ebv$level %in% bulls, ]
head(ebv_bulls[order(-ebv_bulls$blup), ], 8)
```

```
##      level   blup    pev     se reliability
## 4742  3749 0.7434 0.5434 0.7372     0.19497
## 5020  1630 0.5761 0.5125 0.7159     0.24076
## 4603  2896 0.5031 0.5393 0.7343     0.20114
## 4743  3754 0.4531 0.5426 0.7366     0.19616
## 5069  2833 0.4063 0.5496 0.7413     0.18586
## 3953  2863 0.2970 0.5449 0.7382     0.19282
## 3655  1424 0.2847 0.6334 0.7959     0.06166
## 4432  1679 0.2505 0.5194 0.7207     0.23050
```

```
group <- ifelse(ebv$level %in% m1$animal, "cows with records",
                ifelse(ebv$level %in% bulls, "bulls", "others"))
boxplot(reliability ~ group, data = cbind(ebv, group), ylab = "Reliability",
xlab = "")
```



Reliability of the EBVs of cows with records, of bulls and of the remaining animals in the pedigree.

Although bulls have no records of their own, their reliabilities are higher than those of cows with a single record, because their EBV accumulates the information of all their daughters. With such a low heritability, however, all reliabilities are modest.

8.4 Repeatability model with all lactations

With several lactations per cow, the genetic value a_i is separated from the permanent environment p_i, common to all records of a cow but not heritable:

$$y_{ijk} = \mu + \ell_k + h_j + a_i + p_i + e_{ijk}, \qquad \mathbf{p} \sim N(\mathbf{0}, \sigma_p^2 \mathbf{I}).$$

In MLREML the permanent environment is simply a second term with the same animal variable and an identity structure:

```
t_rep <- system.time(
  a2 <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, A) + id(animal),
               data = milk, trace = FALSE))
t_rep[3]

## elapsed
##   24.67

summary(a2)$varcomp

##                       estimate std.error z.ratio bound
## id(herd)!sigma2         4.4460    1.0408   4.272     P
## vm(animal, A)!sigma2    0.7982    0.7329   1.089     P
## id(animal)!sigma2       4.7206    0.7970   5.923     P
## R!units!sigma2         10.3955    0.3236  32.121     P

vpredict(a2, h2 ~ V2 / (V1 + V2 + V3 + V4))

##    estimate std.error
## h2   0.0392   0.03597

vpredict(a2, repeatability ~ (V2 + V3) / (V1 + V2 + V3 + V4))
```

```
##                estimate std.error
## repeatability   0.2711   0.02212
```

```
wald(a2)
```

```
##          term df wald.inc   F.inc ddf.inc     p.inc wald.con   F.con ddf.con
## 1 (Intercept)  1   6258.1 6201.56   52.88 1.702e-56   6258.1 6201.56   52.88
## 2        lact  4    157.8   39.39 2425.07 5.312e-32    157.8   39.39 2425.07
##       p.con   p.chisq
## 1 1.702e-56 0.000e+00
## 2 5.312e-32 4.409e-33
```

Repeatability, the correlation between two lactations of the same cow, is much larger than heritability: a large part of the similarity between lactations is due to the permanent environment. With more records per animal, heritability is estimated more precisely than in the single-lactation analysis.

8.5 Maternal effect

The pedigree identifies the dam of every cow. A maternal permanent-environment effect (the environment the dam provides, shared by maternal sisters) is added as a random term with the dam factor:

```
m1$dam <- pp$dam[match(m1$animal, pp$id)]
m1m <- subset(m1, !is.na(dam))
a3 <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A) + id(dam), data =
m1m, trace = FALSE)
a3b <- mlreml(milk_t ~ 1, random = ~ id(herd) + vm(animal, A), data = m1m,
trace = FALSE)
summary(a3)$varcomp
```

```
##                        estimate std.error   z.ratio bound
## id(herd)!sigma2      6.464e+00     1.669 3.872e+00     P
## vm(animal, A)!sigma2 1.457e+00     1.284 1.135e+00     P
## id(dam)!sigma2       1.723e-07     2.568 6.709e-08     B
## R!units!sigma2       1.100e+01     2.676 4.110e+00     P
```

```
lrt(a3b, a3)
```

```
##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.341e-07     0.5 4957 4959 4972 4978
```

The maternal variance is on the boundary (code `B`): there is no evidence of a maternal environmental effect on the milk yield of these cows. The likelihood ratio statistic is zero to numerical precision.

A maternal genetic effect correlated with the direct genetic effect is fitted with the term `str()`, presented in Section 8.7.

8.6 Bivariate analysis of milk and fat

To estimate the genetic correlation between milk and fat yield in first lactation, we fit a bivariate animal model with unstructured covariance matrices for the herd, genetic and residual effects:

$$\mathrm{Var}\begin{pmatrix}\mathbf{a}_M\\ \mathbf{a}_F\end{pmatrix} = \begin{pmatrix}\sigma^2_{a_M} & \sigma_{a_{MF}}\\ \sigma_{a_{MF}} & \sigma^2_{a_F}\end{pmatrix} \otimes \mathbf{A}.$$

```
rm(a2, a3, a3b, Ai, ebv); invisible(gc())
t_biv <- system.time(
  bv <- mlreml(cbind(milk_t, fat_h) ~ trait,
               random = ~ us(trait):id(herd) + us(trait):vm(animal, A),
               residual = ~ us(trait):units, data = m1, trace = FALSE))
t_biv[3]
```

```
## elapsed
##   37.63
```

```
summary(bv)$varcomp
```

```
##                                              estimate std.error z.ratio bound
## us(trait):id(herd)!trait_milk_t:milk_t         5.5995    1.3545  4.1339     P
## us(trait):id(herd)!trait_fat_h:milk_t          1.6135    0.4267  3.7812     P
## us(trait):id(herd)!trait_fat_h:fat_h           0.6589    0.1601  4.1150     P
## us(trait):vm(animal, A)!trait_milk_t:milk_t    0.6726    0.8661  0.7765     P
## us(trait):vm(animal, A)!trait_fat_h:milk_t     0.1022    0.2785  0.3669     P
## us(trait):vm(animal, A)!trait_fat_h:fat_h      0.2399    0.1358  1.7667     P
## R!us(trait):units!trait_milk_t:milk_t         12.3885    0.9624 12.8725     P
## R!us(trait):units!trait_fat_h:milk_t           3.1642    0.3041 10.4068     P
## R!us(trait):units!trait_fat_h:fat_h            1.4939    0.1396 10.7042     P
```

```
vpredict(bv, rg ~ V5 / sqrt(V4 * V6))
```

```
##    estimate std.error
## rg   0.2544     0.538
```

```
vpredict(bv, re ~ V8 / sqrt(V7 * V9))
```

```
##    estimate std.error
## re   0.7355   0.03364
```

```
vpredict(bv, h2_milk ~ V4 / (V1 + V4 + V7))
```

```
##         estimate std.error
## h2_milk  0.03604   0.04644
```

```
vpredict(bv, h2_fat ~ V6 / (V3 + V6 + V9))
```

```
##        estimate std.error
## h2_fat   0.1003   0.05661
```

```
round(vc_matrix(bv, "us(trait):vm(animal, A)")$cor, 3)
```

```
##        milk_t fat_h
## milk_t  1.000 0.254
## fat_h   0.254 1.000
```

```
round(vc_matrix(bv, "residual")$cor, 3)
```

```
##        milk_t fat_h
## milk_t  1.000 0.736
## fat_h   0.736 1.000
```

The parameters are named V1 to V9 in the order of the table: herd (V1–V3), genetic (V4–V6) and residual (V7–V9) covariances. The residual correlation between milk and fat is high and precisely estimated; the genetic correlation has a large standard error, as expected with only 1314 cows.

8.7 Direct–maternal genetic effects

In traits expressed early in life (birth and weaning weight, litter size, survival) and in some traits of the dam's offspring, the phenotype of an animal depends on its own genes and on the maternal ability of its dam, which is also partly genetic. The direct and maternal genetic effects are usually negatively correlated (Willham, 1963; Meyer, 1992). Model (2.17) is fitted with the term `str(animal, dam, K = A)` (Section 2.11), usually together with a maternal permanent-environment effect `id(dam)`. Because the milk records of Section 8.1 include few dams with records of their own (see the end of this section), the estimation is illustrated first with a simulated population whose true parameters are known.

The following function simulates six generations of 600 animals from 300 founders, with 25 sires per generation chosen among the males of the last two generations; direct and maternal breeding values with $\sigma^2_d = 0.40$, $\sigma^2_m = 0.20$ and $\sigma_{dm} = -0.10$ (genetic correlation −0.35); a maternal permanent-environment variance of 0.10; a residual variance of 0.50; and 30 contemporary groups, treated as fixed. Every animal born after the founders has a record, so most dams have records of their own.

```
sim_maternal <- function(seed = 2026, nf = 300, ngen = 6, noff = 600,
                         S = matrix(c(0.40, -0.10, -0.10, 0.20), 2), s2pe =
0.10, s2e = 0.50) {
  set.seed(seed)
  ped <- data.frame(id = paste0("F", 1:nf), sire = NA, dam = NA,
                    sex = rep(c("M", "F"), length.out = nf), gen = 0)
  for (g in 1:ngen) {                                  # sires and dams from the
last two generations
    cand <- ped[ped$gen >= g - 2, ]
    sires <- sample(cand$id[cand$sex == "M"], 25)
    ped <- rbind(ped, data.frame(id = paste0("G", g, "_", 1:noff),
                                 sire = sample(sires, noff, TRUE),
                                 dam = sample(cand$id[cand$sex == "F"], noff,
TRUE),
                                 sex = sample(c("M", "F"), noff, TRUE), gen =
g))
  }
  A <- Amatrix(ped[, 1:3])
  U <- t(chol(A)) %*% matrix(rnorm(2 * nrow(A)), ncol = 2) %*% chol(S)   #
(direct, maternal)
  rownames(U) <- rownames(A)
  d <- ped[ped$gen > 0, ]
  d$cg <- factor(paste(d$gen, sample(1:5, nrow(d), TRUE), sep = "_"))   #
contemporary groups
  pe <- setNames(rnorm(nrow(A), sd = sqrt(s2pe)), rownames(A))
  d$y <- 10 + rnorm(nlevels(d$cg))[d$cg] + U[d$id, 1] + U[d$dam, 2] + pe[d$dam]
+
         rnorm(nrow(d), sd = sqrt(s2e))
  list(data = d, ped = ped, A = A, U = U)
}
sm <- sim_maternal()
dsm <- sm$data; Asim <- sm$A
c(records = nrow(dsm), animals = nrow(sm$A), dams = length(unique(dsm$dam)),
  dams_with_own_record = sum(unique(dsm$dam) %in% dsm$id))

##              records              animals                 dams
dams_with_own_record
```

```
##                 3600                 3900                 1407
1257
```

The model is fitted with the dense engine, with the matrix **A**, and with the sparse engine, with its inverse:

```
t_d <- system.time(
  dm_d <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "dense"))
Aisim <- Ainverse(sm$ped)
t_s <- system.time(
  dm_s <- mlreml(y ~ cg, random = ~ str(id, dam, K = Aisim) + id(dam), data =
dsm,
                 trace = FALSE, engine = "sparse"))
c(dense = t_d[3], sparse = t_s[3], logLik_difference = dm_d$loglik -
dm_s$loglik)
```

```
##     dense.elapsed    sparse.elapsed logLik_difference
##         3.352e+01          1.592e+00         -5.401e-09
```

```
cbind(true = c(0.40, -0.10, 0.20, 0.10, 0.50), summary(dm_d)$varcomp[,
c("estimate", "std.error")])
```

```
##                                  true estimate std.error
## str(id, dam, K = Asim)!id:id      0.4  0.40027   0.06655
## str(id, dam, K = Asim)!dam:id    -0.1 -0.12404   0.04039
## str(id, dam, K = Asim)!dam:dam    0.2  0.27205   0.04846
## id(dam)!sigma2                    0.1  0.04608   0.02658
## R!units!sigma2                    0.5  0.51821   0.03935
```

Both engines reach the same maximum, and the sparse engine is much faster even with 3,600 records. All the estimates are within about two standard errors of the true values. Maternal effects are estimated less precisely than direct effects, as is well known, and the maternal permanent-environment variance is partly confounded with the maternal genetic variance.

```
vc_matrix(dm_d, "str(id, dam, K = Asim)")$cor
```

```
##          id     dam
## id   1.0000 -0.3759
## dam -0.3759  1.0000
```

```
vpredict(dm_d, h2_direct ~ V1 / (V1 + V2 + V3 + V4 + V5))
```

```
##           estimate std.error
## h2_direct   0.3598   0.05505
```

```
vpredict(dm_d, h2_total ~ (V1 + 1.5 * V2 + 0.5 * V3) / (V1 + V2 + V3 + V4 +
V5))
```

```
##          estimate std.error
## h2_total   0.3148   0.03447
```

`vc_matrix()` returns $\widehat{\boldsymbol{\Sigma}}$ and its correlation matrix. The total heritability of Willham (1972), which governs the response to phenotypic selection, is smaller than the direct heritability because the direct–maternal covariance is negative.

```
ebv <- coef(dm_d, "random", "str(id, dam, K = Asim)")
ebv[ebv$level %in% c("G6_1", "G6_2"), ]
```

```
##      effect level    blup    pev     se reliability
## 3330     id  G6_1 -0.3304 0.2288 0.4784      0.4305
## 3681     id  G6_2 -0.1133 0.2282 0.4777      0.4381
## 7230    dam  G6_1  0.2735 0.2311 0.4807      0.1539
## 7581    dam  G6_2  0.1242 0.2306 0.4802      0.1646
```

```
c(r_direct = cor(ebv$blup[ebv$effect == "id"], sm$U[ebv$level[ebv$effect ==
"id"], 1]),
  r_maternal = cor(ebv$blup[ebv$effect == "dam"], sm$U[ebv$level[ebv$effect ==
"dam"], 2]))
```

```
##   r_direct r_maternal
##     0.6712     0.5159
```

```
dm_0 <- mlreml(y ~ cg, random = ~ str(id, dam, K = Asim, cov = "diag") +
id(dam), data = dsm,
               trace = FALSE)
lrt(dm_0, dm_d, boundary = FALSE)
```

```
##   df LR.statistic  p.value  AIC0  AIC1  BIC0  BIC1
## 1  1        14.69 0.000127 10047 10034 10072 10065
```

Each animal receives a direct (`effect = "id"`) and a maternal (`effect = "dam"`) breeding value, including males, whose maternal value is expressed in their daughters. The correlations with the true simulated values show that the maternal values are predicted less accurately. The model with `cov = "diag"` has the direct–maternal covariance fixed at zero; since a covariance of zero is not on the boundary of the parameter space, the likelihood ratio test uses `boundary = FALSE`, and it rejects $\sigma_{dm} = 0$.

Application to the milk data. Of the 1,314 first-lactation records of Section 8.3, 907 have a known dam, but only 14 dams have a first-lactation record of their own. Without records of the dams, the maternal genetic effect is identified only through the relationships among maternal sisters and their relatives, and the direct–maternal covariance is not estimable: with `str(animal, dam, K = A)` the estimated correlation goes to −1 and the algorithm does not converge in 50 iterations. The model without the covariance converges in a few iterations, with the maternal genetic variance on the boundary:

```
c(records = nrow(m1), with_dam = sum(!is.na(m1$dam)),
  dams_with_own_record = sum(unique(na.omit(m1$dam)) %in% m1$animal))
```

```
##              records             with_dam dams_with_own_record
##                 1314                  907                   14
```

```
a5 <- mlreml(milk_t ~ 1, random = ~ id(herd) + str(animal, dam, K = A, cov =
"diag"),
             data = m1, trace = FALSE)
summary(a5)$varcomp
```

```
##                                              estimate std.error   z.ratio
bound
## id(herd)!sigma2                             5.646e+00    1.3654 4.135e+00
P
## str(animal, dam, K = A, cov = "diag")!animal 6.751e-01    0.8682 7.775e-01
P
## str(animal, dam, K = A, cov = "diag")!dam    6.103e-08    0.9978 6.117e-08
B
```

```
## R!units!sigma2                                1.238e+01    1.1259 1.100e+01
P

lrt(a1, a5)

##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  1   -1.944e-07     0.5 7217 7219 7233 7240
```

As for the maternal environmental effect of Section 8.5, these data contain no evidence of a maternal genetic variance for first-lactation milk yield: the log-likelihood is the same as that of the model of Section 8.3.

8.8 Large data sets: the sparse engine

The examples of this chapter have a few thousand records, a size at which the dense engine is still practical. The sparse engine (Section 2.12) solves the mixed model equations instead, and its advantage grows quickly with the size of the data. The repeatability model of Section 8.4 is fitted again with both engines; the sparse engine receives the inverse of $\mathbf{A}$ built by `Ainverse()`, which carries the attribute `INVERSE = TRUE`:

```
Ai <- Ainverse(pp)
t_dense <- system.time(
  a2d <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, A) +
id(animal),
                data = milk, trace = FALSE, engine = "dense"))
t_sparse <- system.time(
  a2s <- mlreml(milk_t ~ lact, random = ~ id(herd) + vm(animal, Ai) +
id(animal),
                data = milk, trace = FALSE, engine = "sparse"))
c(dense = t_dense[3], sparse = t_sparse[3])

##  dense.elapsed sparse.elapsed
##         25.720          0.526

summary(a2s)

## MLREML fit (AI-REML, sparse engine)
## Observations: 3397   Fixed-effect columns: 5   Iterations: 7 (converged)
## REML log-likelihood: -9400.8394   AIC: 18809.68   BIC: 18834.20
##
## Variance parameters:
##                        estimate std.error z.ratio bound
## id(herd)!sigma2          4.4460    1.0410    4.27     P
## vm(animal, Ai)!sigma2    0.7982    0.7329    1.09     P
## id(animal)!sigma2        4.7206    0.7970    5.92     P
## R!units!sigma2          10.3960    0.3236   32.12     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

The output is that of Section 8.4, and the sparse engine needs well under a second instead of about half a minute. The differences between the two engines are of the order of the convergence tolerance, because the iterations follow slightly different numerical paths:

```
e_d <- coef(a2d, "random", "vm(animal, A)"); e_s <- coef(a2s, "random",
"vm(animal, Ai)")
c(logLik = a2d$loglik - a2s$loglik, theta = max(abs(a2d$theta - a2s$theta)),
```

```
  ebv = max(abs(e_d$blup - e_s$blup)), pev = max(abs(e_d$pev - e_s$pev)),
  se_beta = max(abs(sqrt(diag(vcov(a2d))) - sqrt(diag(vcov(a2s))))))
```

```
##    logLik     theta       ebv       pev   se_beta
## 2.910e-10 1.573e-08 1.576e-08 1.891e-08 3.790e-11
```

```
a2s$sparse[c("N", "nnzL")]
```

```
## $N
## [1] 7968
##
## $nnzL
## [1] 106055
```

The mixed model equations of this model have 7,968 unknowns: 5 fixed effects, 57 herds, the genetic values of the 6,547 animals of the pedigree and the permanent-environment effects of the 1,359 cows with records. The bivariate model of Section 8.6, with unstructured covariance matrices for herd, genetic and residual effects and records with a single trait observed, is also fitted by the sparse engine:

```
t_bs <- system.time(
  bvs <- mlreml(cbind(milk_t, fat_h) ~ trait,
                random = ~ us(trait):id(herd) + us(trait):vm(animal, Ai),
                residual = ~ us(trait):units, data = m1, trace = FALSE, engine
= "sparse"))
t_bs[3]
```

```
## elapsed
##   1.655
```

```
c(logLik_dense = bv$loglik, difference = bv$loglik - bvs$loglik)
```

```
## logLik_dense   difference
##   -5.445e+03    1.237e-08
```

```
vpredict(bvs, rg ~ V5 / sqrt(V4 * V6))
```

```
##    estimate std.error
## rg   0.2544     0.538
```

The sparse fit takes less than two seconds (37.6 s with the dense engine in Section 8.6) and reaches the same log-likelihood. Section 15.2 shows the size of the data sets that the sparse engine handles, with simulated data of up to two million records.

9 Example 6: single-step GBLUP with genotyped and non-genotyped individuals

9.1 Setting

In practice, not every individual with pedigree and phenotype is genotyped. Single-step GBLUP (Legarra et al., 2009; Christensen and Lund, 2010) uses, in a single analysis, the pedigree of all individuals and the markers of the genotyped ones, through the matrix **H** of equation (2.16). Non-genotyped individuals benefit from the markers of their genotyped relatives.

To illustrate it with real data we use again the 599 wheat lines of `BGLR`, which have pedigree and markers, and simulate the usual situation: only 360 lines (60 %) are considered genotyped. This allows comparing the pedigree predictions (A-BLUP) and the single-step predictions (H-BLUP) with the real phenotypes of the non-genotyped lines.

```
data(wheat)
ids <- paste0("L", 1:599)
Aw <- wheat.A; dimnames(Aw) <- list(ids, ids)
Mw <- 2 * wheat.X; rownames(Mw) <- ids
yw <- setNames(wheat.Y[, 1], ids)
set.seed(7)
genotyped <- sample(ids, 360)
Gg <- Gmatrix(Mw[genotyped, ], method = "VanRaden")
```

9.2 Aligning G with the pedigree and building H

Before combining the matrices, $\mathbf{G}$ is aligned with $\mathbf{A}_{22}$ (the pedigree block of the genotyped lines) and blended with 5 % of $\mathbf{A}_{22}$:

```
A22 <- Aw[genotyped, genotyped]
Gal <- align_G(Gg, A22)
c(alpha = attr(Gal, "alpha"), beta = attr(Gal, "beta"))

##  alpha   beta
## 0.3799 0.7989

rbind(G_original = c(diag = mean(diag(Gg)), all = mean(Gg)),
      G_aligned = c(mean(diag(Gal)), mean(Gal)),
      A22 = c(mean(diag(A22)), mean(A22)))

##             diag       all
## G_original 2.000 3.550e-17
## G_aligned  1.978 3.799e-01
## A22        1.978 3.799e-01

H <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE)
range(eigen(H, symmetric = TRUE, only.values = TRUE)$values)

## [1] 9.786e-04 2.197e+02
```

After alignment, the means of the diagonal and of all elements of $\mathbf{G}$ match those of $\mathbf{A}_{22}$. The resulting $\mathbf{H}$ is positive definite. If it were not, `bend()` would correct it.

The inverse $\mathbf{H}^{-1}$, used by programs based on the mixed model equations, is obtained with `inverse = TRUE`; the numerical check shows that it is the inverse of $\mathbf{H}$:

```
Hi <- Hmatrix(Aw, Gg, blend = 0.05, align = TRUE, inverse = TRUE)
max(abs(Hi %*% H - diag(599)))

## [1] 1.669e-11
```

9.3 Comparison by validation

The phenotypes of 120 randomly chosen lines (genotyped and non-genotyped) are removed, predicted with A-BLUP and with H-BLUP, and the correlation with the real phenotypes is computed separately for both groups. This is repeated five times.

```
Ar <- Aw + diag(1e-3, 599); Hr <- H + diag(1e-3, 599)
res <- do.call(rbind, lapply(1:5, function(r) {
  set.seed(100 + r)
  test <- sample(ids, 120)
  ytr <- yw; ytr[test] <- NA
  pA <- gblup(ytr, Ar)$gebv
  pH <- gblup(ytr, Hr)$gebv
  ng <- setdiff(test, genotyped); gg <- intersect(test, genotyped)
  data.frame(rep = r, A_non_genot = cor(pA[ng], yw[ng]), H_non_genot =
cor(pH[ng], yw[ng]),
             A_genot = cor(pA[gg], yw[gg]), H_genot = cor(pH[gg], yw[gg]))
}))
round(res, 3)
```

```
##   rep A_non_genot H_non_genot A_genot H_genot
## 1   1       0.424       0.402   0.349   0.412
## 2   2       0.362       0.315   0.226   0.465
## 3   3       0.341       0.353   0.488   0.500
## 4   4       0.344       0.283   0.461   0.415
## 5   5       0.584       0.636   0.387   0.537
```

```
round(colMeans(res[, -1]), 3)
```

```
## A_non_genot H_non_genot     A_genot     H_genot
##       0.411       0.398       0.382       0.466
```

The gain of H-BLUP over the pedigree is concentrated in the genotyped lines, whose realised relationships are known. For the non-genotyped lines, H-BLUP and A-BLUP have on average a similar predictive ability, with differences in both directions across repetitions. In theory, non-genotyped individuals can benefit because their covariances with the genotyped ones are updated through $\mathbf{A}_{12}\mathbf{A}_{22}^{-1}$, but that benefit depends on how closely related they are to the genotyped individuals; in these wheat lines, with a pedigree that already describes the relationships well, it is not observed. This is a common result in the literature: the advantage of single-step for non-genotyped individuals is usually small.

9.4 Single-step with the general engine

`gblup()` was used above for speed, but $\mathbf{H}$ can be used in any `mlreml()` model through `vm()`, for example with heterogeneous residual variances or in a multi-environment model:

```
dss <- data.frame(gen = ids, y = yw)
fss <- mlreml(y ~ 1, random = ~ vm(gen, Hr), data = dss, trace = FALSE)
summary(fss)$varcomp
```

```
##                     estimate std.error z.ratio bound
## vm(gen, Hr)!sigma2   0.3648   0.06446   5.659     P
## R!units!sigma2       0.5186   0.05096  10.176     P
```

```
vpredict(fss, h2 ~ V1 / (V1 + V2))
```

```
##    estimate std.error
## h2   0.4129   0.05974
```

9.5 Recommendations

- Check that the identifiers of $\mathbf{G}$ are in the pedigree and that the pedigree is complete backwards.
- Align $\mathbf{G}$ with $\mathbf{A}_{22}$ (`align = TRUE`) and blend it with a small fraction of $\mathbf{A}_{22}$ (5–10 %).

- In populations with shallow pedigrees, the parameters τ and ω of $\mathbf{H}^{-1}$ can be tuned to reduce prediction bias (Misztal et al., 2014).

10 Example 7: repeated measures and random regression

Many traits are measured several times on the same individual: weight at different ages, milk yield along the lactation, plant height during the season, yield in successive cuts. Treating those observations as independent underestimates standard errors and wastes the information on each individual's trajectory. MLREML offers two complementary approaches: modelling the correlation of the residuals within individuals (residual structures) and modelling individual trajectories with random coefficients (random regression).

10.1 Residual correlation: growth of children

The `Orthodont` data (Potthoff and Roy, 1964; in `nlme`) record the distance from the pituitary to the pterygomaxillary fissure in 27 boys and girls at ages 8, 10, 12 and 14. Several structures are compared for the covariance matrix of the four records of each child.

```
library(nlme)
ort <- as.data.frame(Orthodont)
ort$Subject <- factor(as.character(ort$Subject))
ort$agef <- ort$age
o1 <- mlreml(distance ~ age * Sex, random = ~ Subject, data = ort, trace =
FALSE)
o2 <- mlreml(distance ~ age * Sex, residual = ~ ar1(agef):id(Subject), data =
ort, trace = FALSE)
o3 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
ar1(agef):id(Subject),
             data = ort, trace = FALSE)
o4 <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units,
             data = ort, trace = FALSE)
o5 <- mlreml(distance ~ age * Sex, residual = ~ us(agef):id(Subject), data =
ort, trace = FALSE)
data.frame(model = c("random intercept (compound symmetry)", "AR1",
                     "intercept + AR1", "intercept + residual by sex",
"unstructured"),
           parameters = sapply(list(o1, o2, o3, o4, o5), function(f)
sum(f$free)),
           logLik = sapply(list(o1, o2, o3, o4, o5), `[[`, "loglik"),
           AIC = sapply(list(o1, o2, o3, o4, o5), `[[`, "AIC"))
```

```
##                                   model parameters logLik   AIC
## 1 random intercept (compound symmetry)          2 -216.9 437.8
## 2                                   AR1          2 -222.3 448.6
## 3                         intercept + AR1          3 -216.9 439.8
## 4           intercept + residual by sex          3 -207.6 421.2
## 5                            unstructured         10 -212.3 444.5
```

In `ar1(agef)` the ages are numeric, so MLREML uses the real distances between measurements: the correlation between two records Δ years apart is ρ^{Δ}. Hence the same structure works for measurements at irregular intervals, as `expc()` would. The model with a residual variance specific to each sex is preferred by AIC: girls have a much smaller within-individual variability than boys.

```
summary(o4)
```

```
## MLREML fit (AI-REML)
## Observations: 108   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -207.6102   AIC: 421.22   BIC: 429.15
##
## Variance parameters:
##                                estimate std.error z.ratio bound
## Subject!sigma2                   3.4135    1.0870    3.14     P
## R!diag(Sex):units!Sex_Male       2.7883    0.5711    4.88     P
## R!diag(Sex):units!Sex_Female     0.6104    0.1528    3.99     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

10.2 Linear random regression: sleep deprivation

The `sleepstudy` data (Belenky et al., 2003; in `lme4`) record the average reaction time of 18 people during 10 days of sleep deprivation. Each person has their own line, with correlated random intercept and slope:

$$y_{ij} = (\beta_0 + b_{0i}) + (\beta_1 + b_{1i})\, t_j + e_{ij}, \qquad \begin{pmatrix} b_{0i} \\ b_{1i} \end{pmatrix} \sim N\left(\mathbf{0}, \begin{pmatrix} \sigma_0^2 & \sigma_{01} \\ \sigma_{01} & \sigma_1^2 \end{pmatrix}\right).$$

In MLREML the basis `pol(Days, 1)` generates the columns $(1, t)$ and the structure `us` their covariance matrix; the complete term is `us(pol(Days, 1)):id(Subject)`.

```
data(sleepstudy, package = "lme4")
rr <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE)
summary(rr)
```

```
## MLREML fit (AI-REML)
## Observations: 180   Fixed-effect columns: 2   Iterations: 4 (converged)
## REML log-likelihood: -871.8141   AIC: 1751.63   BIC: 1764.36
##
## Variance parameters:
##                                                 estimate std.error z.ratio
bound
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_0:0  612.090    288.80    2.12
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:0    9.604     46.68    0.21
P
## us(pol(Days, 1)):id(Subject)!pol(Days, 1)_1:1   35.072     14.78    2.37
P
## R!units!sigma2                                 654.940     77.19    8.49
P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
vpredict(rr, cor_int_slope ~ V2 / sqrt(V1 * V3))
```

```
##               estimate std.error
## cor_int_slope  0.06555     0.325
```

The BLUPs of each person's coefficients allow reconstructing their lines. The levels of a term follow the order of its components, with the last one varying fastest. In `us(pol(Days,`

1)):id(Subject) the first component is the polynomial basis and the second the person, so the first 18 BLUPs are the intercepts of the 18 people and the next 18 their slopes. The coefficient matrix is formed by columns:

```
b <- coef(rr, "random")[[1]]
head(b$level, 3)

## [1] "0:308" "0:309" "0:310"

subj <- levels(factor(sleepstudy$Subject))

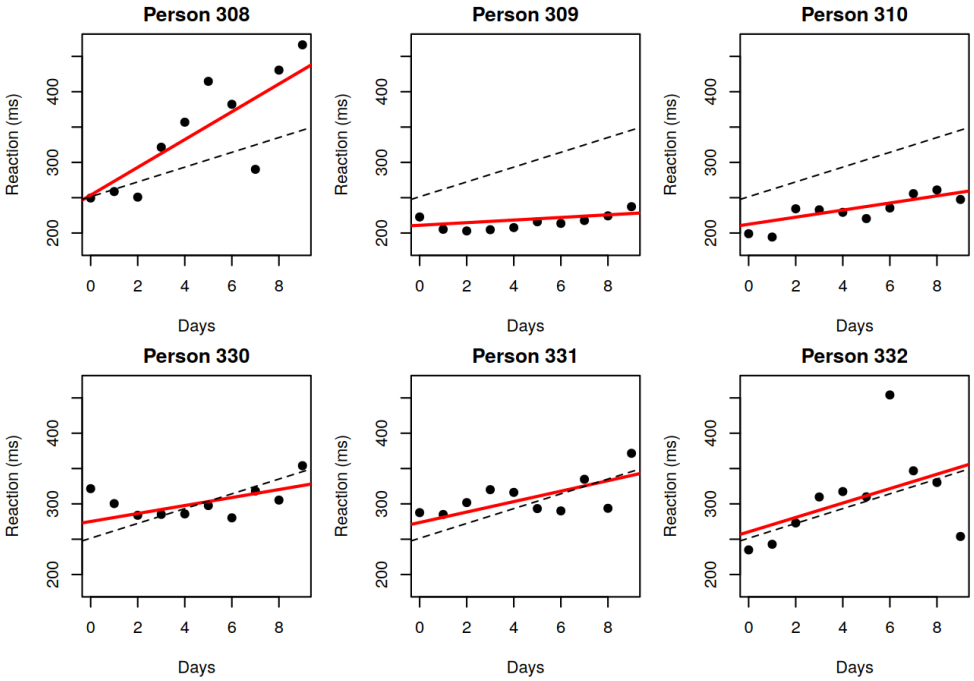
B <- matrix(b$blup, ncol = 2, dimnames = list(subj, c("intercept", "slope")))
head(B)

##     intercept   slope
## 308     2.259  9.1990
## 309   -40.399 -8.6197
## 310   -38.960 -5.4489
## 330    23.691 -4.8143
## 331    22.260 -3.0699
## 332     9.040 -0.2722

head(lme4::ranef(lme4::lmer(Reaction ~ Days + (Days | Subject),
sleepstudy))$Subject)

##     (Intercept)    Days
## 308       2.259  9.1990
## 309     -40.399 -8.6197
## 310     -38.960 -5.4489
## 330      23.691 -4.8144
## 331      22.260 -3.0699
## 332       9.040 -0.2722

op <- par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (s in subj[1:6]) {
  ds <- subset(sleepstudy, Subject == s)
  plot(ds$Days, ds$Reaction, pch = 19, ylim = c(180, 470), main =
paste("Person", s),
       xlab = "Days", ylab = "Reaction (ms)")
  abline(rr$beta[1] + B[s, 1], rr$beta[2] + B[s, 2], col = "red", lwd = 2)
  abline(rr$beta, lty = 2)
}
```



Observed reaction time (points), predicted individual lines (red) and mean line (dashed).

```
par(op)
```

10.3 Random regression with Legendre polynomials: growth curves

In animal breeding, trajectories are modelled with Legendre polynomials (Kirkpatrick et al., 1990; Schaeffer, 2004), which are orthogonal on $[-1,1]$ and give better-conditioned covariance matrices than raw polynomials. The `ChickWeight` data (Crowder and Hand, 1990) contain the weight of 50 chicks fed four diets, measured from birth to day 21.

```
cw <- as.data.frame(ChickWeight)
cw$Chick <- factor(as.character(cw$Chick))
ck1 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 1)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
ck2 <- mlreml(weight ~ Diet + Diet:Time, random = ~ us(leg(Time, 2)):id(Chick),
              residual = ~ diag(Time):units, data = cw, trace = FALSE)
lrt(ck1, ck2, boundary = FALSE)

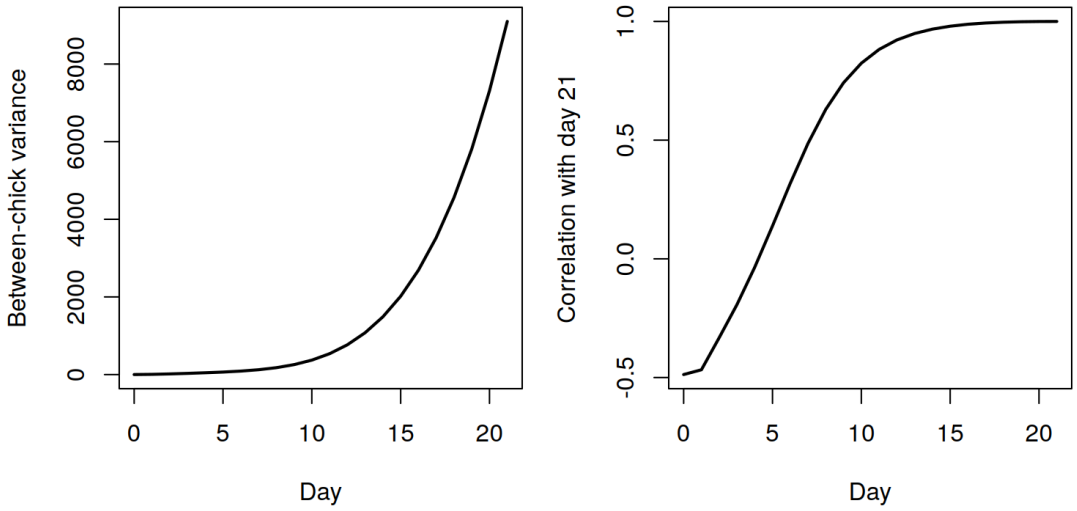
##   df LR.statistic   p.value AIC0 AIC1 BIC0 BIC1
## 1  3        445.3 3.436e-96 4538 4099 4603 4177

summary(ck2)$varcomp[1:6, ]

##                                                 estimate std.error z.ratio bound
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_0:0   1665.1    367.06   4.536     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:0   1542.1    340.91   4.523     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_1:1   1526.5    325.35   4.692     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:0    442.1    105.07   4.208     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:1    484.7    104.72   4.629     P
## us(leg(Time, 2)):id(Chick)!leg(Time, 2)_2:2    175.1     36.42   4.808     P
```

The quadratic model is clearly better. The covariance function between weights at two ages t and t' is $\boldsymbol{\phi}(t)'\mathbf{K}\boldsymbol{\phi}(t')$, where $\boldsymbol{\phi}(t)$ is the vector of Legendre polynomials evaluated at t and $\mathbf{K}$ the covariance matrix of the coefficients. From it we obtain the between-chick variance at each age and the correlation between ages:

```
Kc <- vc_matrix(ck2, "us(leg(Time, 2)):id(Chick)")$cov
tt <- 0:21
Phi <- MLREML:::.legendre(c(tt, range(cw$Time)), 2)[seq_along(tt), ]
Cfun <- Phi %*% Kc %*% t(Phi)
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(tt, diag(Cfun), type = "l", lwd = 2, xlab = "Day", ylab = "Between-chick
variance")
plot(tt, cov2cor(Cfun)[, 22], type = "l", lwd = 2, xlab = "Day", ylab =
"Correlation with day 21")
```



Between-chick variance estimated by random regression (left) and correlation of weight at each age with weight at day 21 (right).

```
par(op)
```

With a pedigree, the same term with `vm(animal, A)` instead of `id(Chick)` gives the additive genetic covariance function, from which heritabilities at each age and genetic correlations between ages are obtained. A complete random regression model in animal breeding typically has a genetic term and a permanent-environment term:

```
mlreml(weight ~ group + leg(age, 3),
       random = ~ us(leg(age, 2)):vm(animal, A) + us(leg(age, 2)):id(animal),
       residual = ~ diag(age_class):units, data = records)
```

11 Example 8: diallel analysis

11.1 The data

The `hayman.tobacco` data set (Hayman, 1954; in `agridat`) is a complete diallel of 8 tobacco varieties (*Nicotiana rustica*): the 64 combinations of female and male parent, including selfs and reciprocal crosses, evaluated in 2 blocks in each of 2 years. The response is the number of days to flowering.

```
data(hayman.tobacco)
dt <- hayman.tobacco
dt$year_f <- factor(dt$year)
str(dt)

## 'data.frame':    256 obs. of  6 variables:
##  $ year  : int  1951 1951 1951 1951 1951 1951 1951 1951 1951 1951 ...
```

```
##  $ block : Factor w/ 2 levels "B1","B2": 1 1 1 1 1 1 1 1 2 2 ...
##  $ male  : Factor w/ 8 levels "G1","G2","G3",..: 1 2 3 4 5 6 7 8 1 2 ...
##  $ female: Factor w/ 8 levels "G1","G2","G3",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ day   : num  27.6 13.6 24.6 31.8 15 18.2 17.4 15.2 30.2 14.2 ...
##  $ year_f: Factor w/ 2 levels "1951","1952": 1 1 1 1 1 1 1 1 1 1 ...
```

```
with(dt, table(female, male))[1:4, 1:4]
```

```
##       male
## female G1 G2 G3 G4
##     G1  4  4  4  4
##     G2  4  4  4  4
##     G3  4  4  4  4
##     G4  4  4  4  4
```

11.2 The model

In a diallel, the value of the cross between female i and male j is decomposed as (Griffing, 1956; Cockerham and Weir, 1977):

$$y_{ijk} = \mu + \text{environment}_k + g_i + g_j + s_{ij} + m_i + r_{ij} + e_{ijk},$$

where g is the general combining ability (GCA) of each parent, the same whether it acts as female or male; $s_{ij} = s_{ji}$ is the specific combining ability (SCA) of the pair, independent of the direction of the cross; m_i is the general maternal effect of the female; and r_{ij} is the specific reciprocal effect of the ordered combination. With random parents, each effect has its own variance.

The technical difficulty is the GCA: each observation has **two** effects of the same factor. In MLREML this is solved with the multi-membership basis `mm(female, male)`, whose design matrix has a 1 in the column of each parent (and a 2 for selfs). The SCA uses a factor with the unordered cross and the reciprocal effects a factor with the ordered cross.

```
dt$cross <- factor(with(dt, paste(pmin(as.character(female),
as.character(male)),
                                  pmax(as.character(female),
as.character(male)), sep = "x")))
dt$cross_ord <- factor(paste(dt$female, dt$male, sep = ">"))
c(unordered_crosses = nlevels(dt$cross), ordered_crosses =
nlevels(dt$cross_ord))
```

```
## unordered_crosses   ordered_crosses
##                36                64
```

```
di <- mlreml(day ~ year_f + year_f:block,
             random = ~ mm(female, male) + cross + female + cross_ord,
             data = dt, trace = FALSE)
summary(di)
```

```
## MLREML fit (AI-REML)
## Observations: 256   Fixed-effect columns: 4   Iterations: 7 (converged)
## REML log-likelihood: -676.2058   AIC: 1362.41   BIC: 1380.06
##
## Variance parameters:
##                           estimate std.error z.ratio bound
## mm(female, male)!sigma2   9.0600    5.1950    1.74     P
## cross!sigma2              3.8401    1.6140    2.38     P
```

```
## female!sigma2              0.4936    0.6835    0.72     P
## cross_ord!sigma2           0.7941    0.9461    0.84     P
## R!units!sigma2             8.3574    0.8597    9.72     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

11.3 Interpretation

The GCA variance is the largest of the genetic variances, indicating that most of the genetic variation in days to flowering is additive and is transmitted predictably to the progeny. The SCA variance reflects non-additive effects (dominance and epistasis) specific to each combination. The maternal and reciprocal effects are small.

Under the usual assumptions (unrelated parents, no epistasis), $\sigma^2_A = 4\sigma^2_{GCA}$ in a non-inbred diploid population and $2\sigma^2_{GCA}$ for fully inbred parents, which is the case for tobacco varieties; the proportion of the genetic variance that is additive is estimated with `vpredict()`:

```
vpredict(di, prop_GCA ~ 2 * V1 / (2 * V1 + V2 + V3 + V4))

##          estimate std.error
## prop_GCA   0.7794    0.1137

vpredict(di, Baker ~ 2 * V1 / (2 * V1 + V2))

##       estimate std.error
## Baker   0.8251    0.1033
```

Baker's ratio (1978), $2\sigma^2_{GCA}/(2\sigma^2_{GCA} + \sigma^2_{SCA})$, close to one indicates that predicting crosses from the GCA of the parents will be accurate.

11.4 Predicted combining abilities

```
gca <- coef(di, "random", "mm(female, male)")
gca[order(gca$blup), c("level", "blup", "se", "reliability")]

##   level    blup   se reliability
## 7    G7 -3.9663 1.29      0.8163
## 8    G8 -2.0061 1.29      0.8163
## 2    G2 -1.6694 1.29      0.8163
## 5    G5 -1.3773 1.29      0.8163
## 6    G6  0.2339 1.29      0.8163
## 4    G4  0.5217 1.29      0.8163
## 1    G1  3.7705 1.29      0.8163
## 3    G3  4.4929 1.29      0.8163
```

Because the response is days to flowering, parents with negative GCA transmit earliness. The predicted value of an unevaluated cross $i \times j$ is obtained by adding the mean, the two GCAs and, when available, the SCA and maternal effects:

```
sca <- coef(di, "random", "cross")
g <- setNames(gca$blup, gca$level); s <- setNames(sca$blup, sca$level)
mu <- mean(di$fixed_fitted)
pred_cross <- function(p1, p2) {
  k <- paste(min(p1, p2), max(p1, p2), sep = "x")
  unname(mu + g[p1] + g[p2] + ifelse(k %in% names(s), s[k], 0))
}
c(G1xG3 = pred_cross("G1", "G3"), G7xG8 = pred_cross("G7", "G8"))
```

```
## G1xG3 G7xG8
## 27.85 10.95
```

11.5 Model comparison

Are the maternal and reciprocal effects needed? Nested models are compared with the likelihood ratio:

```
di0 <- mlreml(day ~ year_f + year_f:block, random = ~ mm(female, male) + cross,
              data = dt, trace = FALSE)
lrt(di0, di, boundary = FALSE)

##   df LR.statistic p.value AIC0 AIC1 BIC0 BIC1
## 1  2        3.974  0.1371 1362 1362 1373 1380
```

With two variances tested simultaneously on the boundary, the null distribution is a mixture of χ^2_0, χ^2_1 and χ^2_2; the χ^2_2 test shown here is conservative.

The same syntax serves for mating designs such as North Carolina I and II, the diallel without reciprocals (Griffing's method 4) or top-crosses, by changing which terms are included. If the parents have a pedigree or markers, `mm()` is not combined directly with `vm()` in this version. The solution is to build the GCA covariance matrix at the cross level: if $\mathbf{Z}_{mm}$ is the multi-membership matrix of the ordered crosses and $\mathbf{K}$ the relationship matrix of the parents, $\mathrm{Var}(g_i + g_j)$ among crosses is $\sigma^2_{GCA}\,\mathbf{Z}_{mm}\mathbf{K}\mathbf{Z}_{mm}'$, and the term is written `vm(cross_ord, Kc)` with `Kc` equal to that matrix:

```
cr <- levels(dt$cross_ord)
p1 <- sub(">.*", "", cr); p2 <- sub(".*>", "", cr)
Zmm <- outer(p1, rownames(K), "==") + outer(p2, rownames(K), "==")
Kc <- Zmm %*% K %*% t(Zmm); dimnames(Kc) <- list(cr, cr)
mlreml(day ~ year_f + year_f:block, random = ~ vm(cross_ord, Kc) + cross, data
= dt)
```

12 Example 9: unreplicated early-generation trial

12.1 The data

In the early stages of a breeding programme, hundreds of new lines are evaluated with a single plot each, interspersed with replicated check varieties that serve to estimate the error and the spatial variation. Cullis, Smith and Coombes (2006) showed that spatial analysis is especially valuable in these trials, because without replication there is no other way to separate genetic merit from plot fertility. The `cullis.earlygen` data (in `agridat`) come from that work: 670 wheat plots in 67 rows by 10 columns, with 532 genotypes. Most appear only once; the checks are replicated. A weed score was also recorded for each plot.

```
data(cullis.earlygen)
eg <- cullis.earlygen
c(plots = nrow(eg), genotypes = nlevels(eg$gen), rows = max(eg$row), columns =
max(eg$col))

##     plots genotypes      rows   columns
##       670       532        67        10

rep_gen <- table(eg$gen)
table(replicates = rep_gen)
```

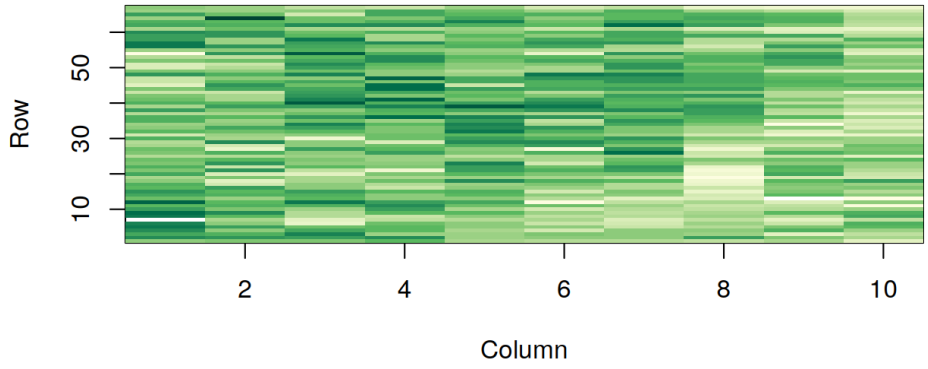
```
## replicates
##   1   3   4   5 121
## 525   1   4   1   1

head(sort(rep_gen, decreasing = TRUE), 6)

##
##    Kite   Banks  Hartog   K1056 Rosella  Vulcan
##     121       5       4       4       4       4

eg$check <- factor(ifelse(rep_gen[as.character(eg$gen)] > 1, "check", "line"))

Ym <- tapply(eg$yield, list(eg$col, eg$row), mean)
image(1:10, 1:67, Ym, col = hcl.colors(30, "YlGn", rev = TRUE), xlab =
"Column", ylab = "Row")
```



Observed yield in the 670 plots (67 rows by 10 columns).

12.2 Models

Three models with the same fixed effects (mean, weed score as a covariate and mean difference between checks and new lines) and random genetic effects are compared:

1. independent residuals;
2. separable AR1 × AR1 residual;
3. AR1 × AR1 residual plus an independent nugget.

```
e1 <- mlreml(yield ~ weed + check, random = ~ gen, data = eg, trace = FALSE)
e2 <- mlreml(yield ~ weed + check, random = ~ gen, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
e3 <- mlreml(yield ~ weed + check, random = ~ gen + units, residual = ~
ar1(row):ar1(col),
             data = eg, trace = FALSE)
data.frame(model = c("independent", "AR1 x AR1", "AR1 x AR1 + nugget"),
           parameters = sapply(list(e1, e2, e3), function(f) sum(f$free)),
           logLik = sapply(list(e1, e2, e3), `[[`, "loglik"),
           AIC = sapply(list(e1, e2, e3), `[[`, "AIC"))

##                 model parameters logLik  AIC
## 1         independent          2  -4897 9798
```

```
## 2          AR1 x AR1          4  -4840 9688
## 3 AR1 x AR1 + nugget          5  -4833 9676
```

```
summary(e3)
```

```
## MLREML fit (AI-REML)
## Observations: 668   Fixed-effect columns: 3   Iterations: 7 (converged)
## REML log-likelihood: -4832.9955   AIC: 9675.99   BIC: 9698.49
##
## Variance parameters:
##                               estimate std.error z.ratio bound
## gen!sigma2                   7.237e+04 1.046e+04    6.92     P
## units!sigma2                 3.274e+04 7.974e+03    4.11     P
## R!ar1(row):ar1(col)!sigma2   5.704e+04 1.123e+04    5.08     P
## R!ar1(row):ar1(col)!row_cor 8.561e-01 3.998e-02   21.41     P
## R!ar1(row):ar1(col)!col_cor 4.027e-01 1.137e-01    3.54     P
##
## Bound codes: P positive/free, B at boundary, F fixed.
```

```
wald(e3)
```

```
##          term df  wald.inc     F.inc ddf.inc     p.inc  wald.con     F.con
ddf.con
## 1 (Intercept)  1 3.521e+03 3.946e+03   10.93 2.480e-15 3.521e+03 3.946e+03
10.93
## 2        weed  1 8.288e+01 7.938e+01  496.09 9.735e-18 8.289e+01 7.940e+01
495.97
## 3       check  1 1.743e-02 1.737e-02  164.50 8.953e-01 1.743e-02 1.737e-02
164.50
##       p.con   p.chisq
## 1 2.480e-15 0.000e+00
## 2 9.693e-18 8.656e-20
## 3 8.953e-01 8.950e-01
```

The spatial models clearly improve the likelihood relative to the model with independent residuals. The weed effect is significant and negative, an example of extraneous variation that should be modelled with covariates when it has been measured.

12.3 Consequences for selection

The aim of an early-generation trial is to select the top fraction of the lines. The practical question is whether the model changes which lines are selected. We compare the BLUPs of the new lines under the independent model and under the spatial model preferred by AIC:

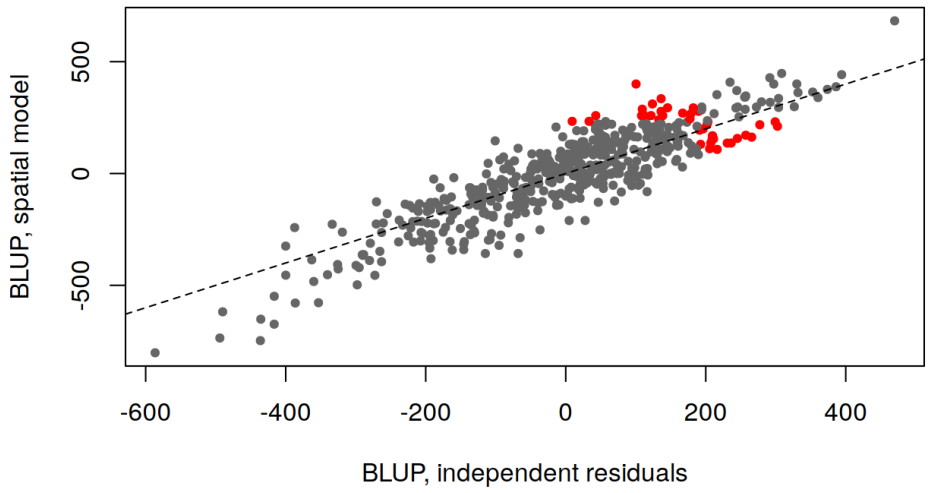
```
b1 <- coef(e1, "random", "gen"); bb <- coef(eb, "random", "gen")
new_lines <- names(rep_gen)[rep_gen == 1]
u1 <- setNames(b1$blup, b1$level)[new_lines]
ub <- setNames(bb$blup, bb$level)[new_lines]
cor(u1, ub, method = "spearman")
```

```
## [1] 0.8612
```

```
k <- round(0.1 * length(new_lines))
sel1 <- names(sort(u1, decreasing = TRUE))[1:k]
selb <- names(sort(ub, decreasing = TRUE))[1:k]
c(selected = k, in_common = length(intersect(sel1, selb)))
```

```
##  selected in_common
##        52        32
```

```
only_one <- setdiff(union(sel1, selb), intersect(sel1, selb))
plot(u1, ub, pch = 20, col = ifelse(new_lines %in% only_one, "red", "grey40"),
     xlab = "BLUP, independent residuals", ylab = "BLUP, spatial model")
abline(0, 1, lty = 2)
```



BLUPs of the new lines with independent residuals against the spatial model. Red points are lines selected (top 10 %) by only one of the two models.

An important part of the selected lines changes with the model. Because the spatial model describes the data better, its predictions are more reliable, which is also reflected in the mean reliability of the BLUPs and in the generalised heritability:

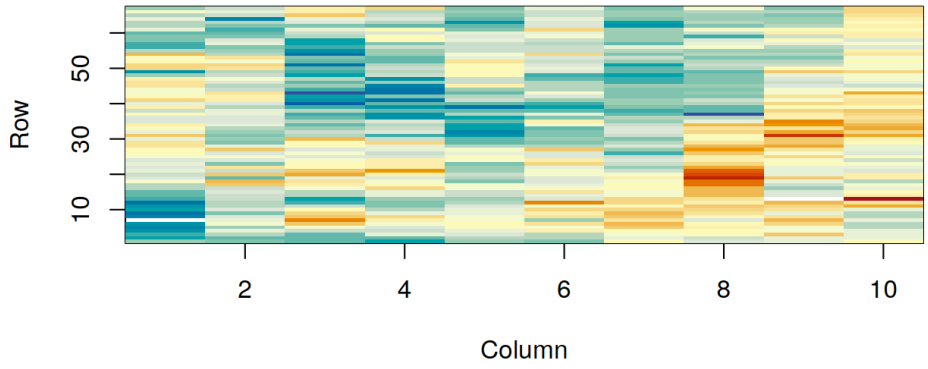
```
H2c <- function(f) {
  pr <- predict(f, "gen", sed = TRUE)
  1 - pr$avsed^2 / (2 * f$theta[1])
}
data.frame(model = c("independent", "spatial"),
           mean_reliability = c(mean(b1$reliability), mean(bb$reliability)),
           H2_Cullis = c(H2c(e1), H2c(eb)))
```

```
##         model mean_reliability H2_Cullis
## 1 independent           0.4025    0.4035
## 2     spatial           0.5767    0.5779
```

12.4 Spatial trend and checks

The estimated spatial variation can be visualised with the residuals of the model after removing fixed and genetic effects. The replicated checks, spread over the whole trial, are what make it estimable.

```
gefe <- bb$blup[match(as.character(eb$data$gen), bb$level)]
esp <- eb$y - eb$fixed_fitted - gefe
Em <- tapply(esp, list(eb$data$col, eb$data$row), mean)
image(1:10, 1:67, Em, col = hcl.colors(30, "RdYlBu"), xlab = "Column", ylab =
"Row")
```



Estimated spatial component (yield minus fixed and genetic effects).

This example shows the workflow recommended by Cullis et al. (2006) for unreplicated trials: model the spatial variation and the measured extraneous factors, treat genotypes as random and base selection on the BLUPs of the best-fitting model.

13 Advanced use and troubleshooting

This chapter gathers techniques that do not belong to a particular type of experiment: how to inspect convergence, how to use starting values and fixed parameters, how to obtain likelihood-based confidence intervals, what to do with boundary estimates and ill-conditioned matrices, and how to reduce memory use. It ends with a table of common problems and their solutions.

13.1 Iteration history

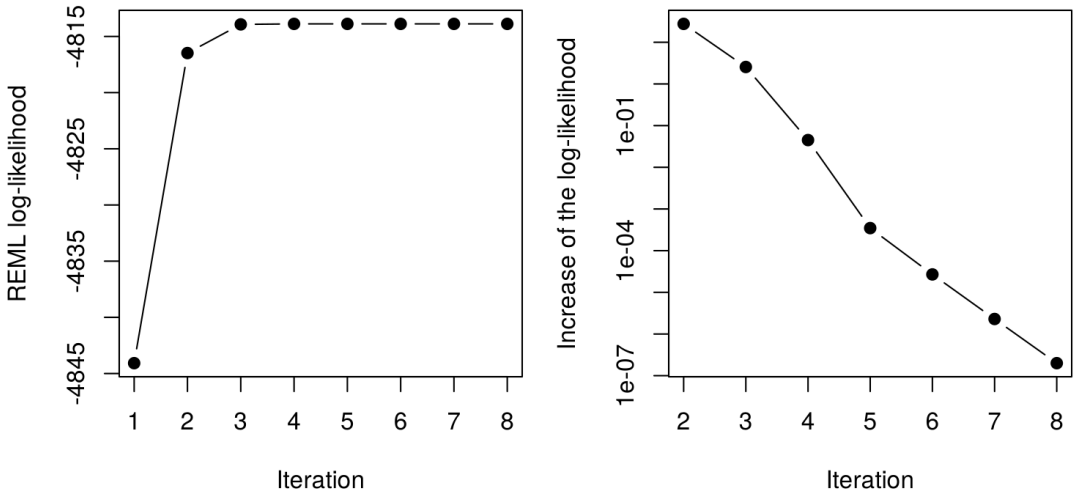
The `history` element of the fitted object stores the log-likelihood of every iteration and the damping factor λ used. A λ of zero indicates a pure AI step; positive values indicate that the pure step was not valid or did not improve the likelihood and had to be shortened.

```
d <- agridat::besag.met
fh <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE, maxit =
100)
head(fh$history, 12)

##   iteration loglik lambda
## 1         1  -4844      0
## 2         2  -4816      0
## 3         3  -4814      0
## 4         4  -4814      0
## 5         5  -4814      0
## 6         6  -4814      0
## 7         7  -4814      0
## 8         8  -4814      0

op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
plot(fh$history$iteration, fh$history$loglik, type = "b", pch = 19,
```

```
     xlab = "Iteration", ylab = "REML log-likelihood")
inc <- diff(fh$history$loglik)
plot(seq_along(inc) + 1, pmax(inc, 1e-8), log = "y", type = "b", pch = 19,
     xlab = "Iteration", ylab = "Increase of the log-likelihood")
```



History of the REML log-likelihood (left) and increase of the likelihood per iteration on a log scale (right) for the FA(1) model.

```
par(op)
```

Near the optimum the AI algorithm converges almost quadratically: the increase of the likelihood falls by several orders of magnitude per iteration. Although three specific variances of this model are on the boundary, the active-set treatment of the boundary (Section 2.4.1) prevents them from blocking the joint step, and the model converges in 8 iterations without damping. A slow and steady decrease of the increases indicates a likelihood that is flat in some direction.

13.2 Starting values and restarts

When a complex model does not converge, it is advisable to restart it from the final values of the previous attempt or from the estimates of a simpler model. Starting values are given as a named vector:

```
ini <- fh$theta                                  # FA(1) estimates
fh2 <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
fa(county, 1):id(gen),
              residual = ~ diag(county):units, data = d, trace = FALSE, start =
ini)
c(iterations = fh2$iterations, logLik = fh2$loglik, converged = fh2$converged)

## iterations     logLik  converged
##          1      -4814          1
```

Starting at the optimum, the algorithm confirms convergence in one or two iterations. This is also the way to verify that a fit has really converged.

To use a simple model as the starting point of a complex one, the common parameters are copied. For example, the variances of a diagonal model serve as initial specific variances of an FA model:

```
fd <- mlreml(yield ~ county, random = ~ county:rep + county:rep:block +
diag(county):id(gen),
             residual = ~ diag(county):units, data = d, trace = FALSE)
vg <- fd$theta[grep("diag\\(county\\):id\\(gen\\)", names(fd$theta))]
names(vg) <- sub("diag\\(county\\):id\\(gen\\)!county_", "fa(county,
1):id(gen)!county_var_", names(vg))
head(vg, 3)
```

```
## fa(county, 1):id(gen)!county_var_C1 fa(county, 1):id(gen)!county_var_C2
##                               57.41                               19.29
## fa(county, 1):id(gen)!county_var_C3
##                               94.35
```

That vector can be passed to `start` (only for the parameters to be initialised; the rest take their default values).

13.3 Fixed parameters and profile likelihood

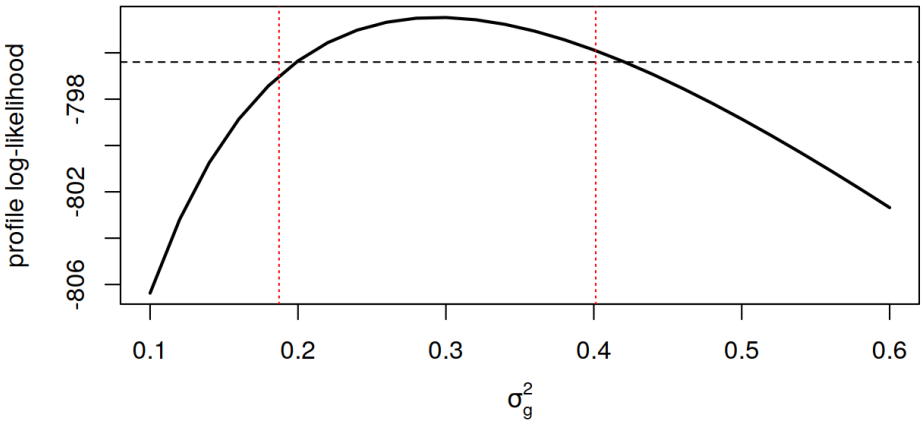
With `fix`, parameters are kept at their starting values. An important application is the **profile likelihood**: a parameter is fixed on a grid of values, the others are estimated and the resulting maximised likelihood is plotted. The 95 % likelihood-based confidence interval contains the values whose profile log-likelihood is within $\chi^2_{1,0.95}/2 = 1.92$ units of the maximum. Unlike the Wald interval (estimate ± 1.96 standard errors), it is not necessarily symmetric and never includes negative values of a variance.

We use the GBLUP model of Chapter 7 for the genetic variance of environment E1:

```
f0 <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, trace = FALSE)
nm <- "vm(gen, G)!sigma2"
grid <- seq(0.10, 0.60, by = 0.02)
prof <- sapply(grid, function(s) {
  mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = setNames(s, nm), fix
= nm,
         trace = FALSE, pev = FALSE)$loglik
})
lim <- range(grid[prof >= max(prof) - qchisq(0.95, 1) / 2])
est <- f0$theta[nm]; se <- f0$varcomp[nm, "std.error"]
rbind(profile = lim, wald = est + c(-1, 1) * 1.96 * se)
```

```
##           [,1]   [,2]
## profile 0.2000 0.4200
## wald    0.1872 0.4013
```

```
plot(grid, prof, type = "l", lwd = 2, xlab = expression(sigma[g]^2), ylab =
"profile log-likelihood")
abline(h = max(prof) - qchisq(0.95, 1) / 2, lty = 2)
abline(v = est + c(-1, 1) * 1.96 * se, lty = 3, col = "red")
```



Profile log-likelihood of the genetic variance (GBLUP, environment E1). The horizontal line marks the threshold of the 95 % interval; the vertical lines, the Wald interval.

The same procedure works for any parameter, including genetic correlations and autoregressive parameters, and is the recommended way to obtain intervals for parameters close to the boundary.

13.4 Known variance components

If the variance components are known (for example, estimated in a large data set or published), all of them can be fixed to obtain only the BLUEs and BLUPs. New generations can thus be predicted with stable parameters without re-estimating them each time:

```
known_vc <- c("vm(gen, G)!sigma2" = 0.30, "R!units!sigma2" = 0.55)
fk <- mlreml(y ~ 1, random = ~ vm(gen, G), data = d1, start = known_vc,
             fix = names(known_vc), trace = FALSE)
summary(fk)$varcomp
```

```
##                    estimate std.error z.ratio bound
## vm(gen, G)!sigma2     0.30        NA      NA     F
## R!units!sigma2        0.55        NA      NA     F
```

```
cor(fk$blups[[1]]$blup, f0$blups[[1]]$blup)
```

```
## [1] 1
```

13.5 Aliased fixed effects

Columns of the fixed-effects matrix that are linear combinations of others are removed automatically, like the aliased effects of ASReml. For example, if the same factor is included twice under different names, or a factor that is a function of another (here, a grouping of the replicates into two fields), the redundant columns are discarded:

```
ja <- agridat::john.alpha
ja$field <- factor(ifelse(ja$rep == "R1", "A", "B"))      # a function of rep
fa_alias <- mlreml(yield ~ rep + field, random = ~ gen, data = ja, trace =
FALSE)
c(possible_columns = ncol(model.matrix(~ rep + field, ja)), estimable_columns =
fa_alias$p)
```

```
##  possible_columns estimable_columns
##                 4                 3

colnames(fa_alias$X)

## [1] "(Intercept)" "repR2"       "repR3"
```

13.6 Parameters on the boundary

A variance estimated at its lower limit (code `B`) means that the likelihood is maximised on the edge of the parameter space. The usual causes are that the effect is really small, that the design does not allow it to be separated from other effects, or that the model is over-parameterised. In those cases:

1. the reported standard error is not valid;
2. the likelihood ratio test must use the mixture of χ^2 distributions (`lrt(..., boundary = TRUE)`);
3. consider removing the term, or replacing a complex structure (`us`) by a more parsimonious one (`fa`, `diag`).

A variance on its bound whose score is negative is held on the bound and removed from the AI system (active set), so that the other parameters converge to their optimum given the boundary; an unstructured matrix that converges to a singular matrix (correlations of ± 1) stops the algorithm with a warning when the steps become negligible.

In factor analytic models, specific variances at zero (Heywood cases) are frequent and do not prevent interpreting the model.

13.7 Ill-conditioned relationship matrices

The dense engine of MLREML does not invert the relationship matrices, so a singular matrix is acceptable as long as $\mathbf{V}$ is positive definite; the sparse engine needs the inverse and therefore a positive definite matrix. However, a matrix with negative eigenvalues (for example, a $\mathbf{G}$ matrix computed with imputed missing data, or a badly constructed $\mathbf{H}$) is not a valid covariance matrix. The function `bend()` corrects it by replacing small or negative eigenvalues:

```
set.seed(1)
Kbad <- cor(matrix(rnorm(30 * 20), 20))          # 30 x 30 with rank 20
Kbad[1, 2] <- Kbad[2, 1] <- 0.99                  # perturbation that makes it
indefinite
range(eigen(Kbad, only.values = TRUE)$values)

## [1] -0.7215  4.4667

Kok <- bend(Kbad, tol = 1e-4)
range(eigen(Kok, only.values = TRUE)$values)

## [1] 0.0004467 4.4667035

max(abs(Kok - Kbad))

## [1] 0.284
```

13.8 Scale of the variables

The algorithm is scale invariant in theory, but in practice convergence is more robust when responses have variances of the order of one. In multivariate models with traits on very

different scales (for example, kilograms of milk and percentage of protein) it is advisable to rescale each trait, as was done in Chapter 8 by expressing milk in tonnes and fat in hundreds of kilograms. Covariances on the original scale are recovered by multiplying by the scale factors.

13.9 Memory and time

- `engine = "sparse"` (Section 2.12) avoids the $n \times n$ matrices altogether; with more than 5,000 records it is chosen automatically when the model is supported.
- `pev = FALSE` skips the computation of the prediction error variances, which is the most expensive part after the fit when a term has many levels (for example, all animals in the pedigree).
- `store = FALSE` does not keep the $n \times n$ matrices needed by `predict()` and by the Kenward–Roger adjustment of `wald()` (with the sparse engine, the Cholesky factor); it greatly reduces the size of the fitted object.
- In a long analysis (cross-validation, simulations) remove fitted objects that are no longer needed and call `gc()`.

13.10 Common problems

Common problems and solutions.

Message or symptom	Likely cause	Solution
`levels of 'x' missing from the vm() matrix`	Some levels of the factor are not in the row names of the matrix.	Check identifiers (spaces, case, prefixes); use `setdiff()` to find them.
`the matrix given to vm() must have rownames`	The relationship matrix has no names.	Assign `dimnames(K) <- list(ids, ids)`.
`term ... has more than one variance-type component`	Two variance structures in one term, e.g. `us(env):diag(gen)`.	Keep only one variance structure; the other must be a correlation (`id`, `vm`, `ar1`…).
`AI-REML did not converge`	Flat likelihood, over-parameterised model or poor starting values.	Increase `maxit`, restart with `start = fit$theta`, simplify the model.
`no improving step found`	The algorithm finds no step that increases the likelihood, usually on the boundary.	Check parameters with code `B`; try other starting values.
`AI-REML stopped because the steps became negligible although the score is not zero`	An unstructured matrix (`us`, `str()`) converges to a singular matrix.	Check the correlations (± 1); use `fa`, `diag` or `str(..., cov = "diag")`.
Variance with code `B` and huge standard error	Parameter on the boundary.	Interpret as zero; use the mixture LRT; consider removing the term.
Genetic correlation equal to ± 1	The `us` matrix is on the boundary of the semidefinite cone.	Normal with few data; consider an `fa` model.
Error in `chol()` or	n too large for dense matrices.	Use `engine =`

Message or symptom	Likely cause	Solution
insufficient memory		`"sparse"` (Section 2.12) or see the strategies in Chapter 15.
`the sparse engine does not support this model (...)`	The model has a structure without a sparse implementation (`cs`, `ar1`, `expc`, `sexp`, `fa`) or an unsupported residual.	Use `engine = "dense"`, or a supported structure.
`the relationship matrix is not positive definite; use bend() or give its inverse`	Sparse engine with a singular relationship matrix.	Bend or blend the matrix (Section 13.7), give its inverse, or use the dense engine.
`REML likelihoods are only comparable with identical fixed effects`	Models with different fixed effects were compared with `lrt()`.	Compare only models with the same fixed part; use Wald tests for fixed effects.

14 Simulation study: properties of the estimators

The comparison with other software (Chapter 15) verifies that MLREML computes the likelihood and its maximum correctly. A different question is how good the REML estimators, their standard errors and the tests are in samples of the size used in practice. This chapter answers with two small simulations that the reader can repeat and modify.

14.1 Complete block design with random genotypes

We simulate 200 experiments with 40 genotypes in 3 complete blocks, with $\sigma^2_g = 1$, $\sigma^2_e = 2$ and fixed block effects. The heritability on a genotype-mean basis is $H^2 = \sigma^2_g / \left(\sigma^2_g + \sigma^2_e / 3\right) = 0.6$.

```
sim_rcbd <- function(seed, ngen = 40, nblk = 3, sg2 = 1, se2 = 2) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:ngen), blk = factor(1:nblk))
  dd$y <- c(0, 0.5, -0.5)[dd$blk] + rnorm(ngen, sd = sqrt(sg2))[dd$gen] +
    rnorm(nrow(dd), sd = sqrt(se2))
  f <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  h <- vpredict(f, H2 ~ V1 / (V1 + V2 / 3))
  c(sg2 = f$theta[[1]], se2 = f$theta[[2]], se_sg2 = f$varcomp$std.error[1],
    H2 = h$estimate, se_H2 = h$std.error, bound = f$varcomp$bound[1] == "B",
    conv = f$converged)
}
res1 <- as.data.frame(t(sapply(1:200, sim_rcbd)))
c(converged = mean(res1$conv), on_boundary = mean(res1$bound))

##   converged on_boundary
##           1           0

truth <- c(sg2 = 1, se2 = 2, H2 = 0.6)
tab1 <- data.frame(parameter = names(truth), true = truth,
                   mean = colMeans(res1[names(truth)]),
                   relative_bias_pct = 100 * (colMeans(res1[names(truth)]) -
```

```
truth) / truth,
                    empirical_sd = sapply(res1[names(truth)], sd))
tab1$mean_se <- c(mean(res1$se_sg2), NA, mean(res1$se_H2))
knitr::kable(tab1, digits = 3, row.names = FALSE,
             caption = "Simulation of 200 complete block experiments: mean,
bias and variability of the REML estimators.")
```

Simulation of 200 complete block experiments: mean, bias and variability of the REML estimators.

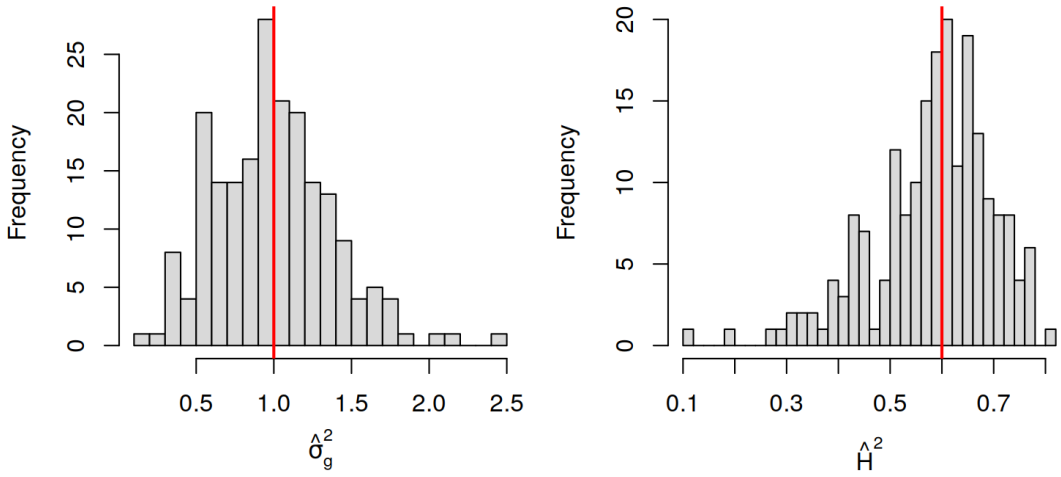
Parameter	true	mean	relative_bias_pct	empirical_sd	mean_se
sg2	1.0	0.997	-0.292	0.378	0.392
se2	2.0	1.990	-0.511	0.326	NA
H2	0.6	0.582	-3.061	0.116	0.116

```
cov95 <- function(est, se, v) mean(est - 1.96 * se <= v & v <= est + 1.96 * se)
c(coverage_sg2 = cov95(res1$sg2, res1$se_sg2, 1), coverage_H2 = cov95(res1$H2,
res1$se_H2, 0.6))

## coverage_sg2  coverage_H2
##         0.95         0.94
```

The REML estimators have a small bias, and the mean standard error obtained from the average information matrix is close to the empirical standard deviation of the estimates, indicating that the asymptotic approximation is reasonable with 40 genotypes. The coverage of the Wald intervals is close to the nominal 95 %. The distribution of $\hat{\sigma}_g^2$ is right-skewed, as expected for a variance estimator:

```
op <- par(mfrow = c(1, 2), mar = c(4, 4, 1, 1))
hist(res1$sg2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(sigma)[g]^2))
abline(v = 1, col = "red", lwd = 2)
hist(res1$H2, breaks = 25, col = "grey85", main = "", xlab =
expression(hat(H)^2))
abline(v = 0.6, col = "red", lwd = 2)
```



Distribution of the REML estimates of the genetic variance (left) and of heritability (right) in 200 simulated experiments. Vertical lines mark the true values.

```
par(op)
```

14.2 Size of the likelihood ratio test on the boundary

To check the boundary correction of `lrt()`, we simulate 200 experiments **without** genetic variance ($\sigma^2_g = 0$) and record the proportion of rejections of $H_0: \sigma^2_g = 0$ at 5 %, with the naive χ^2_1 and with the 50:50 mixture.

```
sim_null <- function(seed) {
  set.seed(seed)
  dd <- expand.grid(gen = factor(1:40), blk = factor(1:3))
  dd$y <- rnorm(nrow(dd), sd = sqrt(2))
  f1 <- mlreml(y ~ blk, random = ~ gen, data = dd, trace = FALSE, pev = FALSE,
store = FALSE)
  f0 <- mlreml(y ~ blk, data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(mixture = lrt(f0, f1, boundary = TRUE)$p.value, naive = lrt(f0, f1,
boundary = FALSE)$p.value,
    boundary = f1$varcomp$bound[1] == "B")
}
pv <- as.data.frame(t(sapply(1:200, sim_null)))
colMeans(pv[c("mixture", "naive")] < 0.05)

## mixture   naive
##   0.020   0.015

mean(pv$boundary)

## [1] 0.49
```

In 49 % of the simulations the genetic variance is estimated on the boundary, close to the half predicted by asymptotic theory. In those simulations the statistic is zero, so both versions of the test are conservative with this sample size: the naive χ^2_1 rejected in 1.5 % of the cases and the 50:50 mixture in 2 %, closer to the nominal 5 %. The mixture is the correct asymptotic approximation; with few levels of the random factor the test remains somewhat conservative, so a significant p value is reliable evidence of genetic variance.

14.3 Spatial model

The second simulation evaluates the AR1 × AR1 model on a 16-row by 12-column grid with 48 genotypes in 4 replicates, with $\rho_r = 0.7$, $\rho_c = 0.4$, spatial variance 1, genetic variance 0.5 and nugget 0.2.

```
sim_sp <- function(seed, rr = 0.7, rc = 0.4) {
  set.seed(seed)
  dd <- expand.grid(row = 1:16, col = 1:12)
  dd$rep <- factor((dd$col - 1) %/% 3 + 1)
  dd$gen <- factor(unlist(lapply(1:4, function(r) sample(1:48))))
  Rr <- rr^abs(outer(1:16, 1:16, "-")); Rc <- rc^abs(outer(1:12, 1:12, "-"))
  xi <- drop(t(chol(kronecker(Rc, Rr))) %*% rnorm(192))       # rows vary
fastest
  dd$y <- rnorm(48, sd = sqrt(0.5))[dd$gen] + xi + rnorm(192, sd = sqrt(0.2))
  f <- mlreml(y ~ rep, random = ~ gen + units, residual = ~ ar1(row):ar1(col),
              data = dd, trace = FALSE, pev = FALSE, store = FALSE)
  c(f$theta, conv = f$converged)
}
res3 <- as.data.frame(t(sapply(1:40, sim_sp)))
names(res3) <- c("sg2", "nugget", "s2_spatial", "rho_row", "rho_col", "conv")
truth3 <- c(sg2 = 0.5, nugget = 0.2, s2_spatial = 1, rho_row = 0.7, rho_col =
```

```
0.4)
data.frame(true = truth3, median = sapply(res3[names(truth3)], median),
           mean = colMeans(res3[names(truth3)]), sd =
sapply(res3[names(truth3)], sd))

##             true median   mean      sd
## sg2          0.5 0.5232 0.5091 0.12644
## nugget       0.2 0.1568 0.1723 0.11259
## s2_spatial   1.0 1.0209 1.0445 0.28329
## rho_row      0.7 0.6573 0.6636 0.09491
## rho_col      0.4 0.3723 0.4015 0.14507
## [1] 0.95

mean(res3$conv)

## [1] 0.95
```

95 % of the fits converged. The autoregressive parameters and the genetic variance are recovered well, with small bias. Separating the nugget from the spatial variance is harder, as is well known in geostatistics: with a grid of this size both components have considerable sampling variability, and their sum is estimated much more precisely than each one separately.

```
c(mean_sum = mean(res3$nugget + res3$s2_spatial), sd_sum = sd(res3$nugget +
res3$s2_spatial))

## mean_sum   sd_sum
##    1.217    0.290
```

These simulations are deliberately small so that they run in a few minutes. For a formal study, increase the number of replicates to 1,000 or more and explore the experiment sizes and parameter values of interest.

15 Numerical validation, performance and limitations

15.1 Comparison with other software

An estimation program must give the same results as established software when both fit the same model. The following table compares the REML log-likelihood of MLREML with that of `lme4` and `nlme` for all models in this guide that those packages can fit, and the function `gblup()` with the general engine. The automatic tests of the package (`tests/testthat`) repeat these comparisons every time the package is checked.

```
library(lme4); library(nlme)
v <- list()
d <- agridat::john.alpha
f <- mlreml(yield ~ rep, random = ~ gen + rep:block, data = d, trace = FALSE)
v[[1]] <- c("Alpha lattice, two variances", "lme4", f$loglik,
            logLik(lmer(yield ~ rep + (1 | gen) + (1 | rep:block), data = d)))
f <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy, trace = FALSE)
v[[2]] <- c("Random regression (us)", "lme4", f$loglik,
            logLik(lmer(Reaction ~ Days + (Days | Subject), sleepstudy)))
f <- mlreml(distance ~ age * Sex, random = ~ Subject, residual = ~
diag(Sex):units, data = ort,
            trace = FALSE)
```

```
v[[3]] <- c("Heterogeneous residual (diag)", "nlme::lme", f$loglik,
            logLik(lme(distance ~ age * Sex, random = ~ 1 | Subject,
                       weights = varIdent(form = ~ 1 | Sex), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ ar1(age):id(Subject), data =
ort, trace = FALSE)
v[[4]] <- c("Autoregressive residual (ar1)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corExp(form = ~ age
| Subject), data = ort)))
f <- mlreml(distance ~ age * Sex, residual = ~ us(age):id(Subject), data = ort,
trace = FALSE)
v[[5]] <- c("Unstructured residual (us)", "nlme::gls", f$loglik,
            logLik(gls(distance ~ age * Sex, correlation = corSymm(form = ~ 1 |
Subject),
                       weights = varIdent(form = ~ 1 | age), data = ort)))
v[[6]] <- c("GBLUP (vm)", "spectral gblup()", f1$loglik, g1$loglik)
tab <- as.data.frame(do.call(rbind, v))
names(tab) <- c("model", "reference", "logLik_MLREML", "logLik_reference")
tab$logLik_MLREML <- as.numeric(tab$logLik_MLREML)
tab$logLik_reference <- as.numeric(tab$logLik_reference)
tab$difference <- signif(tab$logLik_MLREML - tab$logLik_reference, 2)
knitr::kable(tab, digits = 4, caption = "REML log-likelihood of MLREML and of
other software.")
```

REML log-likelihood of MLREML and of other software.

Model	reference	logLik_MLREML	logLik_reference	difference
Alpha lattice, two variances	lme4	-46.6	-46.6	0
Random regression (us)	lme4	-871.8	-871.8	0
Heterogeneous residual (diag)	nlme::lme	-207.6	-207.6	0
Autoregressive residual (ar1)	nlme::gls	-222.3	-222.3	0
Unstructured residual (us)	nlme::gls	-212.3	-212.3	0
GBLUP (vm)	spectral gblup()	-794.5	-794.5	0

The differences are of the order of the convergence tolerance. Variance components, fixed effects, BLUPs and SEDs also agree (Chapters 4 and 10). For structures that no general-purpose R package fits (AR1 × AR1 with nugget, factor analytic, genetic terms with an unstructured structure among environments), the automatic test verifies that the point found is a maximum: perturbing any parameter reduces the likelihood.

The pedigree tools are compared with `pedigreemm`:

```
Fp <- pedigreemm::inbreeding(pedCowsR)
c(max_diff_inbreeding = max(abs(Fp - Fi[p@label])))

## max_diff_inbreeding
##                   0

s <- sample(p@label, 300)
A <- Amatrix(pp)
Ap <- as.matrix(pedigreemm::getA(pedCowsR))[s, s]
c(max_diff_A = max(abs(Ap - A[s, s])))
```

```
## max_diff_A
##   1.11e-16

rm(A); invisible(gc())
```

Additional comparisons. The following code checks further capabilities: (1) the exponential correlation with distances smaller than one against `nlme::gls()`, with which MLREML reaches the same maximum; (2) the Kenward–Roger F statistics and denominator degrees of freedom of the hypotheses tested by `wald()` against those of `pbkrtest`; and (3) the sparse engine against the dense engine.

```
## 1. exponential correlation with distances smaller than one
set.seed(7)
dx <- data.frame(t = rep(c(0, 0.3, 0.7, 1.2, 2), 20), id = factor(rep(1:20,
each = 5)))
dx$y <- 0.5 * dx$t + unlist(lapply(1:20, function(i) cumsum(rnorm(5)) / 2))
fx <- mlreml(y ~ t, residual = ~ expc(t):id(id), data = dx, trace = FALSE)
gx <- gls(y ~ t, correlation = corExp(form = ~ t | id), data = dx)
c(MLREML = fx$loglik, nlme = as.numeric(logLik(gx)))

## MLREML   nlme
## -84.21 -84.21

c(rho_MLREML = unname(fx$theta[2]), rho_nlme = unname(exp(-1 /
coef(gx$modelStruct$corStruct, FALSE))))

## rho_MLREML   rho_nlme
##     0.7511     0.7511

## 2. Kenward-Roger F tests against pbkrtest
kr_row <- function(label, f, l, term) {
  j <- which(f$assign == match(term, attr(f$terms_fixed, "term.labels")))
  L <- chol(solve(vcov(f)))[j, , drop = FALSE]        # incremental hypothesis
of wald()
  w <- wald(f, ddf = "KR"); w <- w[w$term == term, ]
  pk <- KRmodcomp(l, L)$stats
  data.frame(model = label, term = term, F_MLREML = w$F.inc, F_pbkrtest =
pk$Fstat,
             ddf_MLREML = w$ddf.inc, ddf_pbkrtest = pk$ddf)
}
ja <- agridat::john.alpha
oats <- as.data.frame(nlme::Oats)
oats$nitro <- factor(oats$nitro)
oats$Block <- factor(as.character(oats$Block)); oats$Variety <-
factor(as.character(oats$Variety))
f1 <- mlreml(yield ~ rep + gen, random = ~ rep:block, data = ja, trace = FALSE,
tol = 1e-10)
l1 <- lmer(yield ~ rep + gen + (1 | rep:block), data = ja)
f2 <- mlreml(Reaction ~ Days, random = ~ us(pol(Days, 1)):id(Subject), data =
sleepstudy,
             trace = FALSE, tol = 1e-10)
l2 <- lmer(Reaction ~ Days + (Days | Subject), sleepstudy)
f3 <- mlreml(yield ~ Variety * nitro, random = ~ Block + Block:Variety, data =
oats,
             trace = FALSE, tol = 1e-10)
l3 <- lmer(yield ~ Variety * nitro + (1 | Block) + (1 | Block:Variety), oats)
kr <- rbind(kr_row("Alpha lattice", f1, l1, "rep"), kr_row("Alpha lattice", f1,
```

```
l1, "gen"),
            kr_row("Random regression", f2, l2, "Days"),
            kr_row("Split plot (Oats)", f3, l3, "Variety"), kr_row("Split plot
(Oats)", f3, l3, "nitro"),
            kr_row("Split plot (Oats)", f3, l3, "Variety:nitro"))
knitr::kable(kr, digits = 4, row.names = FALSE,
             caption = "Kenward-Roger F statistics and denominator degrees of
freedom of MLREML and pbkrtest.")
```

Kenward-Roger F statistics and denominator degrees of freedom of MLREML and pbkrtest.

Model	term	F_MLREML	F_pbkrtest	ddf_MLREML	ddf_pbkrtest
Alpha lattice	rep	9.2124	9.2124	11.52	11.52
Alpha lattice	gen	5.3628	5.3628	35.50	35.50
Random regression	Days	45.8530	45.8530	17.00	17.00
Split plot (Oats)	Variety	1.4853	1.4853	10.00	10.00
Split plot (Oats)	nitro	37.6856	37.6857	45.00	45.00
Split plot (Oats)	Variety:nitro	0.3028	0.3028	45.00	45.00

The F statistics and the denominator degrees of freedom coincide with those of `pbkrtest` to five or more significant digits. In the split-plot design the method reproduces the classical degrees of freedom of the whole-plot (10) and subplot (45) errors, and $\lambda = 1$.

```
## 3. sparse engine against the dense engine
both <- function(label, ...) {
  fd <- mlreml(..., trace = FALSE, engine = "dense")
  fs <- mlreml(..., trace = FALSE, engine = "sparse")
  bd <- do.call(rbind, fd$blups); bs <- do.call(rbind, fs$blups)
  f <- function(x) sprintf("%.1e", x)
  data.frame(model = label, logLik = round(fd$loglik, 2), logLik_diff =
f(fd$loglik - fs$loglik),
             theta_reldiff = f(max(abs(fd$theta - fs$theta) /
pmax(abs(fd$theta), 1e-8))),
             blup_diff = f(max(abs(bd$blup - bs$blup))), pev_diff =
f(max(abs(bd$pev - bs$pev))))
}
sp <- rbind(
  both("Alpha lattice (two variances)", yield ~ rep, random = ~ gen +
rep:block, data = ja),
  both("Random regression us(leg)", Reaction ~ Days, random = ~ us(leg(Days,
1)):id(Subject),
       data = sleepstudy),
  both("Heterogeneous residual diag(Sex):units", distance ~ age * Sex, random =
~ Subject,
       residual = ~ diag(Sex):units, data = ort),
  both("Split plot (two variances)", yield ~ Variety * nitro, random = ~ Block
+ Block:Variety,
       data = oats))
knitr::kable(sp, row.names = FALSE, align = "lrrrrr",
             caption = "Differences between the dense and the sparse engines
(log-likelihood, maximum relative difference of the variance parameters,
maximum absolute differences of BLUPs and PEVs).")
```

Differences between the dense and the sparse engines (log-likelihood, maximum relative difference of the variance parameters, maximum absolute differences of BLUPs and PEVs).

Model	logLik	logLik_diff	theta_reldiff	blup_diff	pev_diff
Alpha lattice (two variances)	-46.6	-1.8e-12	2.8e-15	6.7e-15	8.3e-17
Random regression us(leg)	-871.8	3.0e-12	1.3e-14	4.0e-13	3.7e-12
Heterogeneous residual diag(Sex):units	-207.6	6.2e-12	6.8e-15	1.8e-13	5.9e-15
Split plot (two variances)	-264.5	-1.3e-12	1.2e-14	6.3e-13	7.1e-13

The two engines give the same results up to rounding error. In addition, the direct–maternal term was checked against a direct maximisation of the likelihood (2.6), with $\mathbf{V}$ built from (2.18) (the estimates agree to the precision of the optimiser and the BLUPs and PEV to 10^{-13}), and the automatic tests of the package verify that with the covariance fixed at zero it reproduces the model with two independent terms `vm(animal, A) + vm(dam, A)`.

15.2 Computing time and memory

The cost of the dense engine of MLREML is dominated by operations with the dense $n \times n$ matrix $\mathbf{V}$. To measure it, a GBLUP model was fitted to simulated data of different sizes on the same computer (one core, OpenBLAS):

```
times <- do.call(rbind, lapply(c(500, 1000, 2000, 3000), function(n) {
  set.seed(n)
  Msim <- matrix(rbinom(n * 1000, 2, 0.3), n); rownames(Msim) <- paste0("i",
1:n)
  Ks <- Gmatrix(Msim) + diag(1e-3, n)
  ds <- data.frame(id = rownames(Msim), y = drop(Msim %*% rnorm(1000, 0, 0.05))
+ rnorm(n))
  t1 <- system.time(fs <- mlreml(y ~ 1, random = ~ vm(id, Ks), data = ds, trace
= FALSE))[3]
  t2 <- system.time(gblup(setNames(ds$y, ds$id), Ks))[3]
  data.frame(n = n, iterations = fs$iterations, sec_mlreml = t1,
             sec_per_iteration = t1 / (fs$iterations + 2), sec_gblup = t2,
             memory_V_GB = round(3 * 8 * n^2 / 1e9, 2))
}))
knitr::kable(times, digits = 2, row.names = FALSE,
             caption = "Fitting time of a GBLUP model by number of
observations.")
```

Fitting time of a GBLUP model by number of observations.

n	iterations	sec_mlreml	sec_per_iteration	sec_gblup	memory_V_GB
500	4	0.12	0.02	0.04	0.01
1000	3	0.53	0.11	0.24	0.02
2000	4	4.05	0.68	1.47	0.10
3000	3	9.25	1.85	17.63	0.22

The time per iteration grows approximately with n^3 and memory with n^2 (the memory column estimates three double-precision $n \times n$ matrices: $\mathbf{V}$, $\mathbf{P}$ and a working matrix). With 8 GB of memory, MLREML comfortably fits models with about 6,000 records; with 16 GB, about

9,000. In a model with a single random term the AI algorithm converges in very few iterations, so `mlreml()` is as fast as `gblup()` or faster. The cost of `gblup()` is dominated by the spectral decomposition, which is several times more expensive than a Cholesky factorisation; its advantage appears when the general model needs many iterations or when the decomposition can be reused.

The number of parameters has a linear influence: each parameter requires forming its derivative matrix and a trace, at a cost of $O(n^2)$. Models with many parameters (`us` with many environments, `fa` with several factors) also need more iterations.

Sparse engine. To measure the sparse engine, dairy-type populations were simulated with ten overlapping generations, artificial-insemination sires used across herds, dams within herds, records on all animals with known parents and contemporary groups defined by herd and year (fixed effects, about 20 records each). The additive animal model `y ~ hy`, `random = ~ vm(id, Ai)`, and the direct–maternal model with `random = ~ str(id, dam, K = Ai)` were fitted on the same computer as above (one core, OpenBLAS, 8 GB of memory), together with a model without pedigree for repeated records of cows in herds, `y ~ year`, `random = ~ herd + cow`, with four records per cow. The code is installed with the package: `system.file("scale", "sparse_scale.R", package = "MLREML")`.

Sparse engine: size of the problem, computing time and memory (elements of the factor in its supernodal storage, which includes the explicit zeros of the supernodes).

Model	Records	Animals or cows	Equations	Elements of the factor (millions)	Iterations	Time (s)	Peak memory (GB)
Animal model	23,750	25,000	26,234	1.5	4	3.6	0.34
Animal model	47,500	50,000	52,468	3.8	4	12.3	0.65
Animal model	95,000	100,000	104,908	10.7	4	20.1	0.77
Animal model	237,500	250,000	262,287	49.4	5	125.1	2.36
Direct–maternal	23,750	25,000	51,234	4.0	8	20.0	0.54
Direct–maternal	47,500	50,000	102,468	11.1	5	48.6	1.19
Direct–maternal	95,000	100,000	204,908	34.4	8	134.5	2.15
Repeated records	1,000,000	250,000	250,358	3.0	5	26.5	1.43
Repeated records	2,000,000	500,000	500,691	6.0	7	63.2	2.56

The estimates were close to the simulated values in all cases (for example, 0.304 and 0.644 for the additive and residual variances of the animal model with 100,000 records, simulated with 0.30 and 0.60 plus a maternal component that the additive model does not include; and 2.00, 1.00 and 2.99 for the herd, cow and residual variances of the model with one million records, simulated with 2, 1 and 3). Time and memory are governed by the number of non-zero elements of the Cholesky factor of the mixed model equations rather than by the number of

records. In the animal models the pedigree and the contemporary groups link many animals and the factor fills in: from 100,000 to 250,000 records its size was multiplied by 4.6 and the time by 6. With 8 GB of memory, the largest animal models that could be fitted with this population structure were the additive model with 250,000 records and the direct–maternal model with 100,000 records; the next sizes tried (500,000 and 250,000 records) exceeded the memory. A model whose random effects are nested, such as repeated records of cows within herds, has a factor with few non-zero elements, and the sparse engine fitted it with one and two million records in about one minute. Animal models with millions of records therefore need a computer with several times more memory than the one used here or, for routine national evaluations, the specialised programs with iterative solvers (Section 15.4).

15.3 Strategies for large data sets

- **Sparse engine.** With the structures it supports, `engine = "sparse"` analyses all the records in a single step (Section 2.12); use `Ainverse()` for pedigree relationships, and remember that its cost depends on the fill of the Cholesky factor, which grows with the number of fixed and random effects connected by the records.
- **Two-stage analysis.** First fit each trial or environment separately and then analyse the adjusted means (BLUEs) with their weights (Smith et al., 2001; Piepho et al., 2012). The means reduce the number of observations to one per genotype and environment.
- **Data compression.** Summarise individuals into groups (for example, by quantiles of the phenotype) and fit a weighted model to the group means.
- **Subsamples to estimate variances.** Estimate the components on a representative subsample and fix them (`start` and `fix`) to predict the complete data set with `gblup()`.
- **More memory and a parallel BLAS.** With multithreaded OpenBLAS or MKL, the time per iteration decreases almost in proportion to the number of cores.

15.4 Limitations of version 1.0.0

1. **Scale.** The dense engine is limited to a few thousand records. On a computer with 8 GB of memory the sparse engine fits animal models with a few hundred thousand records and models with nested random effects with millions of records (Section 15.2), because its memory grows with the fill of the Cholesky factor; it is not intended to replace the specialised programs (ASReml, BLUPF90, DMU, WOMBAT) for national evaluations with tens of millions of records, solvers by iteration on data or single-step genomic models with hundreds of thousands of genotyped animals. Structures with correlation parameters (`cs`, `ar1`, `expc`, `sexp`) and factor analytic structures are only available in the dense engine.
2. **Covariance between terms.** Two random terms can be correlated only through `str()`, which requires that their factors share the levels of the same matrix **K** (for example, direct and maternal genetic effects); correlations between terms with different relationship matrices or with different bases are not available. An intercept and a slope are correlated within a single random-regression term.
3. **Degrees of freedom.** Kenward–Roger degrees of freedom are available for the dense engine; the sparse engine uses $n - p$, and the Satterthwaite approximation is not implemented.
4. **Distributions.** Only normal responses are fitted; there are no generalised linear mixed models.
5. **Structures.** Some ASReml-specific structures are not available, such as `corgh` with separate variances, ARMA, random cubic splines or `at()` for conditional terms, although `diag()` and `us()` cover most uses of `at()`. The correlation of spatial models is common to all environments when `diag(env):ar1(row):ar1(col)` is used.

6. **Standard errors on the boundary.** As in ASReml, the standard errors of parameters at their limit are not valid.

These limitations are the development priorities for future versions, in particular the extension of the sparse engine to the structures with correlation parameters and to factor analytic models, and the Kenward–Roger adjustment for the sparse engine.

16 Function summary

The following table summarises all exported functions. The complete documentation of each one, with its arguments, values and examples, is in the reference manual included as an appendix at the end of this document, and is also available in R with `?function_name`.

MLREML functions.

Function	Purpose
`mlreml()`	Fits a linear mixed model by AI-REML (dense or sparse engine).
`summary()`, `print()`	Variance components, likelihood, AIC and BIC; fixed effects with standard errors.
`coef()`	Fixed effects (`type = "fixed"`) or BLUPs with PEV, standard error and reliability (`type = "random"`).
`fitted()`, `residuals()`, `vcov()`, `logLik()`	Standard R methods.
`wald()`	Incremental and conditional Wald tests for fixed terms, with Kenward–Roger or residual denominator degrees of freedom.
`predict()`	Predicted means, standard errors and SED matrix for a classification.
`vpredict()`	Functions of the variance parameters with delta-method standard errors.
`lrt()`	Likelihood ratio test, with boundary correction.
`vc_matrix()`	Covariance and correlation matrix of a `diag`, `us` or `fa` component or of a `str()` term; loadings and specific variances.
`variogram()`	Sample variogram of the residuals of a field trial.
`plot()`	Diagnostic and convergence plots.
`pedigree_prepare()`	Cleans, completes and sorts a pedigree.
`inbreeding_coef()`	Inbreeding coefficients (Meuwissen and Luo, 1992).
`Amatrix()`	Dense additive relationship matrix (tabular method).
`Ainverse()`	Sparse inverse of **A** (Henderson's rules with inbreeding), usable directly in `vm()` and `str()`.
`snp_qc()`	Marker quality control (MAF, call rate per marker and per individual).
`Gmatrix()`	Genomic relationship matrix (VanRaden, Yang or scaled).
`align_G()`	Aligns **G** with $\mathbf{A}_{22}$.
`Hmatrix()`	Single-step matrix **H** or its inverse.

Function	Purpose
`bend()`	Turns a symmetric matrix into a positive definite one.
`gblup()`	GBLUP by spectral rotation.
`cv_gblup()`	k-fold cross-validation of GBLUP.

16.1 Syntax equivalences with ASReml-R

For ASReml-R users, the following table translates the most common constructs.

Translation of ASReml-R syntax.

ASReml-R (version 4)	MLREML
`asreml(fixed = y ~ rep, random = ~ gen, data = d)`	`mlreml(y ~ rep, random = ~ gen, data = d)`
`random = ~ vm(animal, ainv)`	`random = ~ vm(animal, A)` (matrix $\mathbf{A}$) or `random = ~ vm(animal, Ainverse(ped))`
`random = ~ fa(env, 2):id(gen)`	Same
`random = ~ us(env):vm(gen, Ginv)`	`random = ~ us(env):vm(gen, G)`
`residual = ~ dsum(~ units | env)`	`residual = ~ diag(env):units`
`residual = ~ ar1(row):ar1(col)`	Same
`random = ~ str(~ animal + dam, ~ us(2):vm(animal, ainv))`	`random = ~ str(animal, dam, K = A)`
`random = ~ leg(age, 2):id(animal)` with `us()`	`random = ~ us(leg(age, 2)):id(animal)`
`random = ~ and(male)` together with `female`	`random = ~ mm(female, male)`
`fixed = cbind(y1, y2) ~ trait, residual = ~ id(units):us(trait)`	`cbind(y1, y2) ~ trait, residual = ~ us(trait):units`
`summary(fit)$varcomp`	`summary(fit)$varcomp`
`wald(fit)`	`wald(fit)` (Kenward–Roger denominator degrees of freedom by default)
`predict(fit, classify = "gen")`	`predict(fit, classify = "gen")`
`vpredict(fit, h2 ~ V1/(V1+V2))`	Same
`lrt(fit0, fit1)`	Same

The most important difference is that `vm()` receives the relationship matrix itself, because the dense engine of MLREML works with the marginal form of the model; its inverse can also be given when it carries the attribute `INVERSE = TRUE`, as the result of `Ainverse()`. With the dense engine the matrices may be singular; the sparse engine needs them positive definite.

16.2 Quick guide to models

The following templates cover the most common analyses. Variable names are illustrative.

```
## Complete blocks, random genotypes
mlreml(y ~ rep, random = ~ gen, data = d)

## Alpha lattice or incomplete blocks, fixed genotypes
mlreml(y ~ rep + gen, random = ~ rep:block, data = d)

## Row x column spatial analysis with nugget and linear trend
```

```r
mlreml(y ~ rep + row_num, random = ~ gen + units,
       residual = ~ ar1(row):ar1(column), data = d)

## Irregular spatial data (coordinates)
mlreml(y ~ 1, random = ~ gen + units, residual = ~ sexp(x, y), data = d)

## MET: genetic and residual variances by environment
mlreml(y ~ env, random = ~ env:rep + diag(env):id(gen),
       residual = ~ diag(env):units, data = d)

## MET: two-factor factor analytic model
mlreml(y ~ env, random = ~ env:rep + fa(env, 2):id(gen),
       residual = ~ diag(env):units, data = d, maxit = 200)

## GBLUP and multi-environment GBLUP
mlreml(y ~ 1, random = ~ vm(gen, G), data = d)
mlreml(y ~ env, random = ~ us(env):vm(gen, G), residual = ~ diag(env):units,
data = d)

## Pedigree + markers (two kernels)
mlreml(y ~ 1, random = ~ vm(gen, A) + vm(gen, G), data = d)

## Single-step
H <- Hmatrix(A, G); mlreml(y ~ 1, random = ~ vm(id, H), data = d)

## Animal model, repeatability and maternal effect
mlreml(y ~ herd + sex, random = ~ vm(animal, A), data = d)
mlreml(y ~ herd + parity, random = ~ vm(animal, A) + id(animal), data = d)
mlreml(y ~ herd, random = ~ vm(animal, A) + id(dam), data = d)

## Direct-maternal animal model with maternal permanent environment
mlreml(y ~ herd + sex, random = ~ str(animal, dam, K = A) + id(dam), data = d)

## Bivariate with genetic and residual correlations
mlreml(cbind(y1, y2) ~ trait + trait:herd, random = ~ us(trait):vm(animal, A),
       residual = ~ us(trait):units, data = d)

## Repeated measures with autoregressive residual
mlreml(y ~ trt * time, random = ~ subject, residual = ~ ar1(time):id(subject),
data = d)

## Random regression
mlreml(y ~ group + age, random = ~ us(leg(age, 2)):vm(animal, A) +
       us(leg(age, 2)):id(animal), residual = ~ diag(age_class):units, data =
d)

## Diallel (GCA, SCA, maternal, reciprocal)
mlreml(y ~ block, random = ~ mm(female, male) + cross + female + cross_ord,
data = d)

## Known variance components (BLUE and BLUP only)
mlreml(y ~ 1, random = ~ vm(gen, G), data = d,
       start = c("vm(gen, G)!sigma2" = 0.3, "R!units!sigma2" = 0.6),
       fix = c("vm(gen, G)!sigma2", "R!units!sigma2"))

## Large data set: sparse engine with the inverse of A
Ai <- Ainverse(ped)
mlreml(y ~ hys + sex, random = ~ vm(animal, Ai) + id(animal), data = d, engine
= "sparse")
```

16.3 Post-fit functions

```
summary(fit)                              # variance components
coef(fit, "random", "vm(gen, G)")         # BLUP, PEV, reliability
wald(fit, ddf = "KR")                     # tests of fixed effects (Kenward-Roger)
predict(fit, classify = "env:gen")        # predicted means and SED
vpredict(fit, h2 ~ V1 / (V1 + V2))        # functions of components
vc_matrix(fit, "us(env):vm(gen, G)")      # covariance and correlation matrices
lrt(fit0, fit1)                           # likelihood ratio
variogram(fit, "row", "column")           # sample variogram
plot(fit)                                 # diagnostics
```

References

Baker, R.J. (1978). Issues in diallel analysis. *Crop Science* 18, 533–536.

Bates, D., Mächler, M., Bolker, B., Walker, S. (2015). Fitting linear mixed-effects models using lme4. *Journal of Statistical Software* 67(1), 1–48.

Belenky, G. et al. (2003). Patterns of performance degradation and restoration during sleep restriction and subsequent recovery. *Journal of Sleep Research* 12, 1–12.

Besag, J., Higdon, D. (1999). Bayesian analysis of agricultural field experiments. *Journal of the Royal Statistical Society B* 61, 691–746.

Burgueño, J., de los Campos, G., Weigel, K., Crossa, J. (2012). Genomic prediction of breeding values when modeling genotype × environment interaction using pedigree and dense molecular markers. *Crop Science* 52, 707–719.

Butler, D.G., Cullis, B.R., Gilmour, A.R., Gogel, B.J., Thompson, R. (2017). *ASReml-R Reference Manual Version 4*. VSN International, Hemel Hempstead.

Chen, Y., Davis, T.A., Hager, W.W., Rajamanickam, S. (2008). Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software* 35(3), 22.

Christensen, O.F., Lund, M.S. (2010). Genomic prediction when some animals are not genotyped. *Genetics Selection Evolution* 42, 2.

Christensen, O.F., Madsen, P., Nielsen, B., Ostersen, T., Su, G. (2012). Single-step methods for genomic evaluation in pigs. *Animal* 6, 1565–1571.

Cockerham, C.C., Weir, B.S. (1977). Quadratic analyses of reciprocal crosses. *Biometrics* 33, 187–203.

Crossa, J. et al. (2010). Prediction of genetic values of quantitative traits in plant breeding using pedigree and molecular markers. *Genetics* 186, 713–724.

Crowder, M.J., Hand, D.J. (1990). *Analysis of Repeated Measures*. Chapman and Hall, London.

Cullis, B.R., Smith, A.B., Coombes, N.E. (2006). On the design of early generation variety trials with correlated data. *Journal of Agricultural, Biological and Environmental Statistics* 11, 381–393.

Erisman, A.M., Tinney, W.F. (1975). On computing certain elements of the inverse of a sparse matrix. *Communications of the ACM* 18, 177–179.

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *Journal of Agricultural, Biological and Environmental Statistics* 2, 269–293.

Gilmour, A.R., Gogel, B.J., Cullis, B.R., Welham, S.J., Thompson, R. (2015). *ASReml User Guide Release 4.1*. VSN International, Hemel Hempstead.

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440–1450.

Griffing, B. (1956). Concept of general and specific combining ability in relation to diallel crossing systems. *Australian Journal of Biological Sciences* 9, 463–493.

Halekoh, U., Højsgaard, S. (2014). A Kenward–Roger approximation and parametric bootstrap methods for tests in linear mixed models — the R package pbkrtest. *Journal of Statistical Software* 59(9).

Hayman, B.I. (1954). The analysis of variance of diallel tables. *Biometrics* 10, 235–244.

Henderson, C.R. (1975). Best linear unbiased estimation and prediction under a selection model. *Biometrics* 31, 423–447.

Henderson, C.R. (1976). A simple method for computing the inverse of a numerator relationship matrix used in prediction of breeding values. *Biometrics* 32, 69–83.

Henderson, C.R. (1984). *Applications of Linear Models in Animal Breeding*. University of Guelph.

John, J.A., Williams, E.R. (1995). *Cyclic and Computer Generated Designs*. Chapman and Hall, London.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *Journal of Dairy Science* 78, 449–456.

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709–1723.

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983–997.

Kirkpatrick, M., Lofsvold, D., Bulmer, M. (1990). Analysis of the inheritance, selection and evolution of growth trajectories. *Genetics* 124, 979–993.

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *Journal of Dairy Science* 92, 4656–4663.

Lin, L., Yang, C., Meza, J.C., Lu, J., Ying, L., E, W. (2011). SelInv — an algorithm for selected inversion of a sparse symmetric matrix. *ACM Transactions on Mathematical Software* 37(4), 40.

Meuwissen, T.H.E., Hayes, B.J., Goddard, M.E. (2001). Prediction of total genetic value using genome-wide dense marker maps. *Genetics* 157, 1819–1829.

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genetics Selection Evolution* 24, 305–313.

Meyer, K. (1992). Variance components due to direct and maternal effects for growth traits of Australian beef cattle. *Livestock Production Science* 31, 179–204.

Misztal, I., Legarra, A., Aguilar, I. (2014). Using recursion to compute the inverse of the genomic relationship matrix. *Journal of Dairy Science* 97, 3943–3952.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *Journal of Dairy Science* 76, 1479–1483.

Patterson, H.D., Thompson, R. (1971). Recovery of inter-block information when block sizes are unequal. *Biometrika* 58, 545–554.

Pérez, P., de los Campos, G. (2014). Genome-wide regression and prediction with the BGLR statistical package. *Genetics* 198, 483–495.

Piepho, H.P., Möhring, J., Schulz-Streeck, T., Ogutu, J.O. (2012). A stage-wise approach for the analysis of multi-environment trials. *Biometrical Journal* 54, 844–860.

Pinheiro, J.C., Bates, D.M. (2000). *Mixed-Effects Models in S and S-PLUS*. Springer, New York.

Potthoff, R.F., Roy, S.N. (1964). A generalized multivariate analysis of variance model useful especially for growth curve problems. *Biometrika* 51, 313–326.

Quaas, R.L. (1976). Computing the diagonal elements and inverse of a large numerator relationship matrix. *Biometrics* 32, 949–953.

Schaeffer, L.R. (2004). Application of random regression models in animal breeding. *Livestock Production Science* 86, 35–45.

Self, S.G., Liang, K.Y. (1987). Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *Journal of the American Statistical Association* 82, 605–610.

Smith, A.B., Cullis, B.R. (2018). Plant breeding selection tools built on factor analytic mixed models for multi-environment trial data. *Euphytica* 214, 143.

Smith, A.B., Cullis, B.R., Thompson, R. (2001). Analyzing variety by environment data using multiplicative mixed models and adjustments for spatial field trend. *Biometrics* 57, 1138–1147.

Smith, A.B., Cullis, B.R., Thompson, R. (2005). The analysis of crop cultivar breeding and evaluation trials: an overview of current mixed model approaches. *Journal of Agricultural Science* 143, 449–462.

Stram, D.O., Lee, J.W. (1994). Variance components testing in the longitudinal mixed effects model. *Biometrics* 50, 1171–1177.

Takahashi, K., Fagan, J., Chen, M.S. (1973). Formation of a sparse bus impedance matrix and its application to short circuit study. *Proceedings of the 8th Power Industry Computer Applications (PICA) Conference*, Minneapolis, 63–69.

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *Journal of Dairy Science* 91, 4414–4423.

Vazquez, A.I., Bates, D.M., Rosa, G.J.M., Gianola, D., Weigel, K.A. (2010). Technical note: an R package for fitting generalized linear mixed models in animal breeding. *Journal of Animal Science* 88, 497–504.

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Australian and New Zealand Journal of Statistics* 46, 325–347.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18–27.

Willham, R.L. (1972). The role of maternal effects in animal breeding: III. Biometrical aspects of maternal effects in animals. *Journal of Animal Science* 35, 1288–1293.

Wright, K. (2024). *agridat: Agricultural Datasets*. R package, CRAN.

Appendix: function reference manual

This appendix reproduces the help page of every function in the package. In R, it is available with `?function_name`.

16.4 `MLREML-package`: MLREML: linear mixed models by Average-Information REML

16.4.0.1 Description

MLREML fits linear mixed models for plant and animal breeding by restricted maximum likelihood with the Average Information algorithm. The main function is `mlreml`. Two computing engines are available: a dense engine based on the variance matrix of the observations, which supports every variance structure, and a sparse engine based on the mixed model equations, the sparse Cholesky factorisation and the Takahashi sparse inverse, for data sets with hundreds of thousands to millions of records. User guides in English and in Spanish with the theory and worked examples with real data are installed with the package: `system.file("doc", "MLREML-guide-en.pdf", package = "MLREML")` and `system.file("doc", "MLREML-guide-es.pdf", package = "MLREML")`.

16.4.0.2 Details

Main functions:

> `mlreml`: model fitting (including direct-maternal models with the `str()` term and the sparse engine).
>
> `summary.mlreml`, `wald` (with Kenward-Roger denominator degrees of freedom), `predict.mlreml`, `vpredict`, `lrt`, `vc_matrix`, `variogram`: inference.
>
> `pedigree_prepare`, `Amatrix`, `Ainverse`, `inbreeding_coef`, `snp_qc`, `Gmatrix`, `align_G`, `Hmatrix`, `bend`: relationship matrices.
>
> `gblup`, `cv_gblup`: fast GBLUP.

16.4.0.3 Author(s)

Maintainer: Osval A. Montesinos-Lopez maintainer@example.org

16.5 `mlreml`: Fit a linear mixed model by Average-Information REML

16.5.0.1 Description

`mlreml` fits the linear mixed model

$$y = X\beta + \sum_k Z_k\, u_k + e, \quad u_k \sim N(0, G_k), \quad e \sim N(0, R)$$

by restricted maximum likelihood (REML) with the Average Information (AI) algorithm. The covariance matrices G_k and R are specified with an ASReml-like syntax as direct (Kronecker) products of variance structures.

16.5.0.2 Usage

```
mlreml(
  fixed,
  random = NULL,
  residual = NULL,
  data,
  start = NULL,
  fix = NULL,
  maxit = 50,
  tol = 1e-06,
  pev = TRUE,
  trace = TRUE,
  store = TRUE,
  engine = c("auto", "dense", "sparse")
)
```

16.5.0.3 Arguments

Argument	Description
`fixed`	two-sided formula for the fixed effects.
`random`	one-sided formula with the random terms, or `NULL`.
`residual`	one-sided formula for the residual structure.
`data`	data frame.
`start`	optional named vector of starting values (names as in `summary(fit)$varcomp`).
`fix`	character vector with names of variance parameters to be held fixed at their starting values.
`maxit`	maximum number of AI iterations.
`tol`	convergence tolerance for the change of the REML log-likelihood.
`pev`	logical; compute prediction error variances of the BLUPs.
`trace`	logical; print the iteration history.
`store`	logical; keep the matrices needed by `predict.mlreml` and by the Kenward-Roger adjustment of `wald`.
`engine`	`"auto"`, `"dense"` or `"sparse"` (see the section on computing engines).

16.5.0.4 Value

An object of class `"mlreml"`.

16.5.0.5 Specifying random terms and residuals

Each random term is a product of components separated by `:`. Each component is a structure function applied to a factor, to a polynomial basis of a covariate or to a multi-membership factor:

`id(f)` or `f`
independent effects, identity matrix.

`vm(f, K)`
effects correlated according to a known matrix `K` (pedigree A, genomic G, single-step H, kinship). `K` must have rownames that contain all levels of `f`; it may contain additional levels (individuals without records), whose BLUPs are also returned. `K` may also be given through its inverse, flagged with the attribute `INVERSE = TRUE` (as returned by `Ainverse`); the sparse engine then uses it directly.

`diag(f)`
independent effects with a different variance for each level of `f`.

`us(f)`
unstructured (fully parameterised) covariance matrix.

`fa(f, k)`
factor analytic structure with `k` factors, $\Lambda\Lambda' + \Psi$.

`cs(f)`
compound symmetry correlation.

`ar1(f)`
first-order autoregressive correlation; if the levels of `f` are numeric, the actual distances are used, so gaps are allowed.

`expc(x)`
exponential correlation $\rho^{|x_i - x_j|}$ for an irregularly spaced numeric covariate.

`sexp(x, y)`
isotropic exponential correlation $\rho^{d_{ij}}$ on two-dimensional coordinates (irregular spatial data).

`pol(x, k)`, `leg(x, k)`
raw and normalised Legendre polynomial bases of order `k` (random regression); usually combined with `us()` or `diag()`.

`mm(f1, f2)`
multi-membership factor (for example, general combining ability of the two parents in a diallel).

`units`
the observation units (one level per record).

A term whose components are all correlation structures (`id`, `vm`, `cs`, `ar1`, `expc`, `sexp`) receives a scale variance. At most one variance structure (`diag`, `us`, `fa`) is allowed per term, and it carries the scale.

The `residual` formula uses the same syntax; the default is `~ units` (independent residuals with a common variance) and, for multivariate models, `~ us(trait):units`.

16.5.0.6 Correlated effects of several factors (direct-maternal models)

The term `str(f1, f2, ..., K = A, cov = "us")` defines $m \geq 2$ random effects that share the levels of the matrix `K` and are correlated with each other: with $u = ({u_1}', \ldots, {u_m}')'$ and $Z = [Z_1, \ldots, Z_m]$,

$$Var(u) = \Sigma \otimes K,$$

where Σ is an unstructured (`cov = "us"`) or diagonal (`cov = "diag"`) $m \times m$ matrix. The usual application is the direct-maternal animal model, `str(animal, dam, K = A)`, in which Z_1 links each record to the animal and Z_2 to its dam, and Σ contains the direct and maternal genetic variances and their covariance. A missing value of a factor of `str()` (for example an unknown dam) means that the record receives no effect from it; such records are kept. If `K` is omitted the levels are independent ($K = I$).

16.5.0.7 Computing engines

The `"dense"` engine works with the $n \times n$ matrix $V = Var(y)$ and supports every structure; its cost grows with n^3 and its memory with n^2, so it is meant for up to a few thousand records. The `"sparse"` engine works with Henderson's mixed model equations, a sparse Cholesky factorisation (CHOLMOD, through the Matrix package) and the sparse (Takahashi) inverse, so that its cost depends on the number of non-zero elements rather than on n^2; it handles hundreds of thousands to millions of records. It supports random terms built from `id`, `vm`, `diag` and `us` components (with factors, `pol`, `leg` or `mm` bases) and `str()` terms, and the residual structures `units`, `diag(f):units` and, in multivariate models, `us(trait):units`. `engine = "auto"` (default) uses the sparse engine when there are more than 5000 records and the model is supported, and the dense engine otherwise. Both engines give the same estimates, BLUPs and prediction error variances up to numerical precision.

16.5.0.8 Multivariate models

If the left-hand side of `fixed` is `cbind(y1, y2, ...)`, the data are stacked internally, a factor `trait` is created, and `units` identifies the original records, so that `us(trait):units` is the residual covariance matrix between traits measured on the same record. Records with some traits missing are kept for the traits observed.

16.5.0.9 References

Gilmour, A.R., Thompson, R., Cullis, B.R. (1995). Average information REML: an efficient algorithm for variance parameter estimation in linear mixed models. *Biometrics* 51, 1440-1450.

Johnson, D.L., Thompson, R. (1995). Restricted maximum likelihood estimation of variance components for univariate animal models using sparse matrix techniques and average information. *J. Dairy Sci.* 78, 449-456.

Misztal, I., Perez-Enciso, M. (1993). Sparse matrix inversion for restricted maximum likelihood estimation of variance components by expectation-maximization. *J. Dairy Sci.* 76, 1479-1483.

Willham, R.L. (1963). The covariance between relatives for characters composed of components contributed by related individuals. *Biometrics* 19, 18-27.

16.5.0.10 Examples

```
set.seed(1)
d <- data.frame(block = factor(rep(1:4, each = 30)),
                gen = factor(rep(1:30, 4)))
```

```r
d$y <- 10 + rnorm(4)[d$block] + rnorm(30, sd = 1.5)[d$gen] + rnorm(120)
fit <- mlreml(y ~ block, random = ~ id(gen), data = d)
summary(fit)
```

16.6 summary.mlreml: Print and summary methods for mlreml fits

16.6.0.1 Description

Print and summary methods for mlreml fits

16.6.0.2 Usage

```r
## S3 method for class 'mlreml'
summary(object, ...)

## S3 method for class 'summary.mlreml'
print(x, ...)

## S3 method for class 'mlreml'
print(x, ...)

## S3 method for class 'mlreml'
fitted(object, ...)

## S3 method for class 'mlreml'
residuals(object, ...)

## S3 method for class 'mlreml'
vcov(object, ...)

## S3 method for class 'mlreml'
logLik(object, ...)
```

16.6.0.3 Arguments

Argument	Description
object, x	an object of class "mlreml".
...	further arguments (ignored).

16.6.0.4 Value

summary returns a list with the variance components (varcomp), the fixed effects with standard errors (coef.fixed), the log-likelihood and information criteria.

16.7 coef.mlreml: Extract fixed and random effects

16.7.0.1 Description

Extract fixed and random effects

16.7.0.2 Usage

```r
## S3 method for class 'mlreml'
coef(object, type = c("fixed", "random"), term = NULL, ...)

ranef_mlreml(object, term = NULL)

fixef_mlreml(object)
```

16.7.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`type`	`"fixed"` or `"random"`.
`term`	for random effects, the label of a random term (as in `names(object$blups)`); `NULL` returns all terms.
`...`	ignored.

16.7.0.4 Value

A named vector (fixed effects) or a data frame (BLUPs with prediction error variance, standard error and reliability).

16.8 `wald`: Wald tests for fixed effects

16.8.0.1 Description

Computes incremental and conditional Wald statistics for every term of the fixed formula, with F tests whose denominator degrees of freedom are obtained by the method of Kenward and Roger (1997) or, as a large-sample approximation, by the residual degrees of freedom $n - rank(X)$.

16.8.0.2 Usage

```
wald(object, ddf = c("auto", "KR", "residual"))
```

16.8.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`ddf`	`"auto"` (Kenward-Roger for fits of the dense engine stored with `store = TRUE`, residual degrees of freedom otherwise), `"KR"` or `"residual"`.

16.8.0.4 Details

The incremental statistic of a term tests it adjusted for the terms that precede it in the fixed formula (sequential, "type I" tests). The conditional statistic tests a term adjusted for all other terms that do not contain it, respecting the principle of marginality as ASReml does (equivalent to "type II" tests): the main effect `A` in `y ~ A * B` is adjusted for `B` but not for `A:B`. Both statistics are computed from the Cholesky factor of the inverse of the covariance matrix of the fixed effects, with the columns reordered for the conditional statistic (the term after the terms that do not contain it and before the terms that contain it).

With the Kenward-Roger method, the statistic of each hypothesis $L\beta = 0$ is $F = \lambda\left(L\hat{\beta}\right)'(L\Phi_A L')^{-1}\left(L\hat{\beta}\right)/q$, where Φ_A is the covariance matrix of the fixed effects adjusted for the uncertainty of the estimated variance parameters, and the denominator degrees of freedom m and the scale λ are obtained by matching the moments of F with those of an $F(q, m)$ distribution. The expected information matrix is used; the second derivatives of V are included for structures that are not linear in their parameters (`ar1`, `expc`, `sexp`, `fa`, products of parameterised components). Variance parameters that are fixed or on the boundary of the parameter space are treated as known.

16.8.0.5 Value

A data frame with one row per fixed term: numerator degrees of freedom (`df`); incremental Wald statistic, F statistic, denominator degrees of freedom and p-value (`wald.inc`, `F.inc`, `ddf.inc`, `p.inc`); the same for the conditional test (`wald.con`, `F.con`, `ddf.con`, `p.con`); and the p-value of the conditional Wald statistic referred to a chi-square distribution (`p.chisq`). The attribute `"ddf"` gives the method used.

16.8.0.6 References

Kenward, M.G., Roger, J.H. (1997). Small sample inference for fixed effects from restricted maximum likelihood. *Biometrics* 53, 983-997.

Halekoh, U., Hojsgaard, S. (2014). A Kenward-Roger approximation and parametric bootstrap methods for tests in linear mixed models - the R package pbkrtest. *J. Stat. Softw.* 59(9).

16.8.0.7 Examples

```
set.seed(2)
d <- data.frame(trt = gl(3, 20), blk = gl(4, 5, 60))
d$y <- c(0, 1, 2)[d$trt] + rnorm(4)[d$blk] + rnorm(60)
fit <- mlreml(y ~ trt, random = ~ blk, data = d, trace = FALSE)
wald(fit)
wald(fit, ddf = "residual")
```

16.9 `predict.mlreml`: Predicted means with standard errors and standard errors of differences

16.9.0.1 Description

Forms predictions for the levels of one or more factors (`classify`), averaging the fixed effects over the levels of the other fixed factors (simple averaging) and evaluating covariates at their mean. Random terms whose factors are all contained in `classify` are included as BLUPs; the remaining random terms are ignored (their expectation is zero). The standard errors and the matrix of standard errors of differences (SED) are computed from the joint prediction error variance of fixed and random effects, following Welham et al. (2004).

16.9.0.2 Usage

```
## S3 method for class 'mlreml'
predict(object, classify, sed = TRUE, ...)
```

16.9.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit (fitted with `store = TRUE`).
`classify`	character, for example `"gen"` or `"env:gen"`.
`sed`	logical; return the SED matrix.
`...`	ignored.

16.9.0.4 Value

A list with `pvals` (predicted values and standard errors), `sed` (matrix) and `avsed` (average SED).

16.9.0.5 References

Welham, S., Cullis, B., Gogel, B., Gilmour, A., Thompson, R. (2004). Prediction in linear mixed models. *Aust. N. Z. J. Stat.* 46, 325-347.

16.10 `vpredict`: Functions of variance parameters with delta-method standard errors

16.10.0.1 Description

Evaluates a function of the variance parameters (for example a heritability or a genetic correlation) and its approximate standard error from the inverse of the Average Information matrix, as the `vpredict` function of ASReml-R. Parameters are referred to as `V1`, `V2`, ... in the order of `summary(fit)$varcomp`.

16.10.0.2 Usage

```
vpredict(object, formula)
```

16.10.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`formula`	a formula such as `h2 ~ V2 / (V2 + V3)`.

16.10.0.4 Value

A data frame with the estimate and its standard error.

16.10.0.5 Examples

```
set.seed(3)
d <- data.frame(fam = gl(40, 5))
d$y <- rnorm(40, sd = 1)[d$fam] + rnorm(200, sd = 2)
fit <- mlreml(y ~ 1, random = ~ fam, data = d, trace = FALSE)
vpredict(fit, h2 ~ 4 * V1 / (V1 + V2))
```

16.11 `lrt`: Likelihood ratio test between nested models

16.11.0.1 Description

Compares two models with the same fixed effects that differ in their variance structure. With `boundary = TRUE` the p-value uses a 50:50 mixture of chi-square distributions, appropriate when a single variance parameter is tested on the boundary of the parameter space.

16.11.0.2 Usage

```
lrt(fit0, fit1, boundary = TRUE)
```

16.11.0.3 Arguments

Argument	Description
`fit0`	reduced model.
`fit1`	full model.
`boundary`	logical; use the mixture distribution.

16.11.0.4 Value

A data frame with the statistic, degrees of freedom and p-value.

16.12 `vc_matrix`: Estimated covariance and correlation matrix of a structured component

16.12.0.1 Description

Returns the estimated covariance matrix of a `us`, `diag` or `fa` component (for example the genetic covariance matrix between environments or traits), its correlation matrix and, for `fa`, the loadings and specific variances.

16.12.0.2 Usage

```
vc_matrix(object, term)
```

16.12.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`term`	label of the term (as in `object$varcomp$term`), or `"residual"`.

16.12.0.4 Value

A list with `cov` and `cor` (and `loadings`, `psi` for factor analytic structures).

16.13 `variogram`: Sample variogram of residuals of a field trial

16.13.0.1 Description

Computes the empirical semi-variogram of the residuals by absolute row and column displacement, the standard diagnostic of spatial analyses (Gilmour, Cullis and Verbyla, 1997).

16.13.0.2 Usage

```
variogram(object, row, col, plot = TRUE, resid = NULL)
```

16.13.0.3 Arguments

Argument	Description
`object`	an `"mlreml"` fit.
`row, col`	names of the numeric row and column variables in the data.
`plot`	logical; draw a perspective plot.
`resid`	optional vector of residuals (default: conditional residuals).

16.13.0.4 Value

A data frame with row lag, column lag, semi-variance and number of pairs.

16.13.0.5 References

Gilmour, A.R., Cullis, B.R., Verbyla, A.P. (1997). Accounting for natural and extraneous variation in the analysis of field experiments. *JABES* 2, 269-293.

16.14 `plot.mlreml`: Diagnostic plots

16.14.0.1 Description

Residuals against fitted values, normal quantile plot and histogram of the residuals, and the history of the REML log-likelihood.

16.14.0.2 Usage

```
## S3 method for class 'mlreml'
plot(x, ...)
```

16.14.0.3 Arguments

Argument	Description
x	an "mlreml" fit.
...	passed to plot.

16.15 pedigree_prepare: Prepare a pedigree

16.15.0.1 Description

Recodes unknown parents, adds parents that do not appear as individuals (as founders), checks for errors and sorts the pedigree so that parents precede their offspring.

16.15.0.2 Usage

```
pedigree_prepare(ped)
```

16.15.0.3 Arguments

Argument	Description
ped	data frame whose first three columns are individual, sire (father) and dam (mother). Unknown parents can be coded as NA, 0, "0", "-" or "".

16.15.0.4 Value

A data frame with columns id, sire, dam (character, NA for unknown), sire_i, dam_i (integer positions, 0 for unknown) and gen (generation number).

16.15.0.5 Examples

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
pedigree_prepare(ped)
```

16.16 inbreeding_coef: Inbreeding coefficients (Meuwissen and Luo, 1992)

16.16.0.1 Description

Inbreeding coefficients (Meuwissen and Luo, 1992)

16.16.0.2 Usage

```
inbreeding_coef(ped)
```

16.16.0.3 Arguments

Argument	Description
ped	a pedigree as returned by pedigree_prepare, or any data frame accepted by it.

16.16.0.4 Value

A named vector of inbreeding coefficients.

16.16.0.5 References

Meuwissen, T.H.E., Luo, Z. (1992). Computing inbreeding coefficients in large populations. *Genet. Sel. Evol.* 24, 305-313.

16.17 `Amatrix`: Numerator (additive) relationship matrix

16.17.0.1 Description

Builds the dense additive relationship matrix A with the tabular method. Suitable for up to roughly 15,000 individuals; for larger pedigrees use the sparse inverse `Ainverse`.

16.17.0.2 Usage

```
Amatrix(ped)
```

16.17.0.3 Arguments

Argument	Description
`ped`	pedigree (see `pedigree_prepare`).

16.17.0.4 Value

A symmetric matrix with individual names as dimnames.

16.17.0.5 Examples

```
ped <- data.frame(id = c("A3", "A4", "A5"), sire = c("A1", "A1", "A3"),
                  dam = c("A2", "A2", "A4"))
Amatrix(ped)
```

16.18 `Ainverse`: Sparse inverse of the additive relationship matrix

16.18.0.1 Description

Henderson's rules with inbreeding (Quaas, 1976), using the inbreeding coefficients of Meuwissen and Luo (1992).

16.18.0.2 Usage

```
Ainverse(ped)
```

16.18.0.3 Arguments

Argument	Description
`ped`	pedigree (see `pedigree_prepare`).

16.18.0.4 Value

A sparse symmetric matrix (`Matrix` package) with the attribute `INVERSE = TRUE`, so that it can be given directly to `vm()` or `str()` in `mlreml` (the sparse engine uses it as it is; the dense engine inverts it).

16.19 `snp_qc`: Quality control of a SNP matrix

16.19.0.1 Description

Quality control of a SNP matrix

16.19.0.2 Usage

```
snp_qc(M, maf = 0.05, callrate = 0.9, ind_callrate = 0.9)
```

16.19.0.3 Arguments

Argument	Description
`M`	matrix of genotypes (individuals in rows), coded 0/1/2 or -1/0/1; missing

Argument	Description
	values as NA.
maf	minimum minor-allele frequency.
callrate	minimum call rate per marker.
ind_callrate	minimum call rate per individual.

16.19.0.4 Value

A list with the filtered matrix (M, coded 0/1/2), allele frequencies and a report of the removed markers and individuals.

16.20 Gmatrix: Genomic relationship matrix

16.20.0.1 Description

Genomic relationship matrix

16.20.0.2 Usage

```
Gmatrix(M, method = c("VanRaden", "Yang", "scaled"), p = NULL)
```

16.20.0.3 Arguments

Argument	Description
M	matrix of genotypes (individuals in rows) coded 0/1/2 or -1/0/1. Missing values are replaced by the marker mean.
method	"VanRaden" (VanRaden 2008, method 1), "Yang" (standardised markers, VanRaden method 2) or "scaled" (centred and scaled markers divided by the number of markers, ZZ'/p, the kernel of GBLUP as ridge regression).
p	optional vector of base allele frequencies.

16.20.0.4 Value

A symmetric matrix with the rownames of M.

16.20.0.5 References

VanRaden, P.M. (2008). Efficient methods to compute genomic predictions. *J. Dairy Sci.* 91, 4414-4423.

16.21 align_G: Align a genomic matrix with the pedigree matrix

16.21.0.1 Description

Rescales G as $\alpha + \beta G$ so that the averages of its diagonal and of all its elements equal those of A22 (Christensen et al., 2012).

16.21.0.2 Usage

```
align_G(G, A22)
```

16.21.0.3 Arguments

Argument	Description
G	genomic relationship matrix.
A22	pedigree relationships of the genotyped individuals.

16.21.0.4 Value

The aligned matrix, with attributes `alpha` and `beta`.

16.22 `Hmatrix`: Single-step relationship matrix H and its inverse

16.22.0.1 Description

Combines the pedigree matrix A of all individuals with the genomic matrix G of the genotyped individuals (Legarra et al., 2009; Aguilar et al., 2010). G is optionally blended with A22 and aligned to it before combining.

16.22.0.2 Usage

```
Hmatrix(A, G, blend = 0.05, align = TRUE, tau = 1, omega = 1, inverse = FALSE)
```

16.22.0.3 Arguments

Argument	Description
`A`	pedigree relationship matrix of all individuals (dense).
`G`	genomic relationship matrix of the genotyped individuals; its rownames must be in `rownames(A)`.
`blend`	weight of A22 in $(1-w)G + wA_{22}$.
`align`	logical; align G to A22 with `align_G`.
`tau`, `omega`	scaling factors of G^{-1} and A_{22}^{-1} in the inverse.
`inverse`	logical; return H^{-1} instead of H.

16.22.0.4 Value

H (or its inverse) with the dimnames of A.

16.22.0.5 References

Legarra, A., Aguilar, I., Misztal, I. (2009). A relationship matrix including full pedigree and genomic information. *J. Dairy Sci.* 92, 4656-4663.

16.23 `bend`: Make a symmetric matrix positive definite (bending)

16.23.0.1 Description

Make a symmetric matrix positive definite (bending)

16.23.0.2 Usage

```
bend(K, tol = 1e-04)
```

16.23.0.3 Arguments

Argument	Description
`K`	symmetric matrix.
`tol`	minimum eigenvalue after bending (relative to the largest).

16.23.0.4 Value

The bent matrix.

16.24 `gblup`: Fast GBLUP by spectral decomposition

16.24.0.1 Description

Fits $y = X\beta + g + e, g \sim N(0, K\sigma_g^2), e \sim N(0, I\sigma_e^2)$ by exact REML using the eigen-decomposition of the relationship matrix of the phenotyped individuals (Kang et al., 2008), which reduces REML to a one-dimensional optimisation. Predictions (GEBVs) are returned for every individual in `K`, phenotyped or not. The results are identical to `mlreml(y ~ ..., random = ~ vm(id, K))` but much faster.

16.24.0.2 Usage

```
gblup(y, K, X = NULL, id = names(y))
```

16.24.0.3 Arguments

Argument	Description
`y`	numeric vector of phenotypes, named by individual, or with the individual identifiers given in `id`.
`K`	relationship matrix with rownames.
`X`	optional fixed-effects design matrix (default: intercept).
`id`	optional character vector of identifiers for `y`.

16.24.0.4 Value

A list with `varcomp` ($\sigma_g^2, \sigma_e^2, h^2$), `beta`, `gebv` (named vector for all individuals in `K`) and `loglik`.

16.24.0.5 References

Kang, H.M. et al. (2008). Efficient control of population structure in model organism association mapping. *Genetics* 178, 1709-1723.

16.25 `cv_gblup`: K-fold cross-validation of GBLUP

16.25.0.1 Description

Randomly partitions the phenotyped individuals into `folds` groups, predicts each group from the others with `gblup` and reports the Pearson correlation between predicted and observed values.

16.25.0.2 Usage

```
cv_gblup(y, K, folds = 5, reps = 1, seed = 1)
```

16.25.0.3 Arguments

Argument	Description
`y`	named vector of phenotypes.
`K`	relationship matrix.
`folds`	number of folds.
`reps`	number of repetitions of the random partition.
`seed`	random seed.

16.25.0.4 Value

A data frame with one row per repetition and fold.